

一、linux入门

TIPS: 本门课用到的所有额外软件都在同一个 [网盘链接](#) 下面，你可以一次把所有的软件都下载下来，到课程对应部分直接使用即可。

1.1 Linux环境搭建

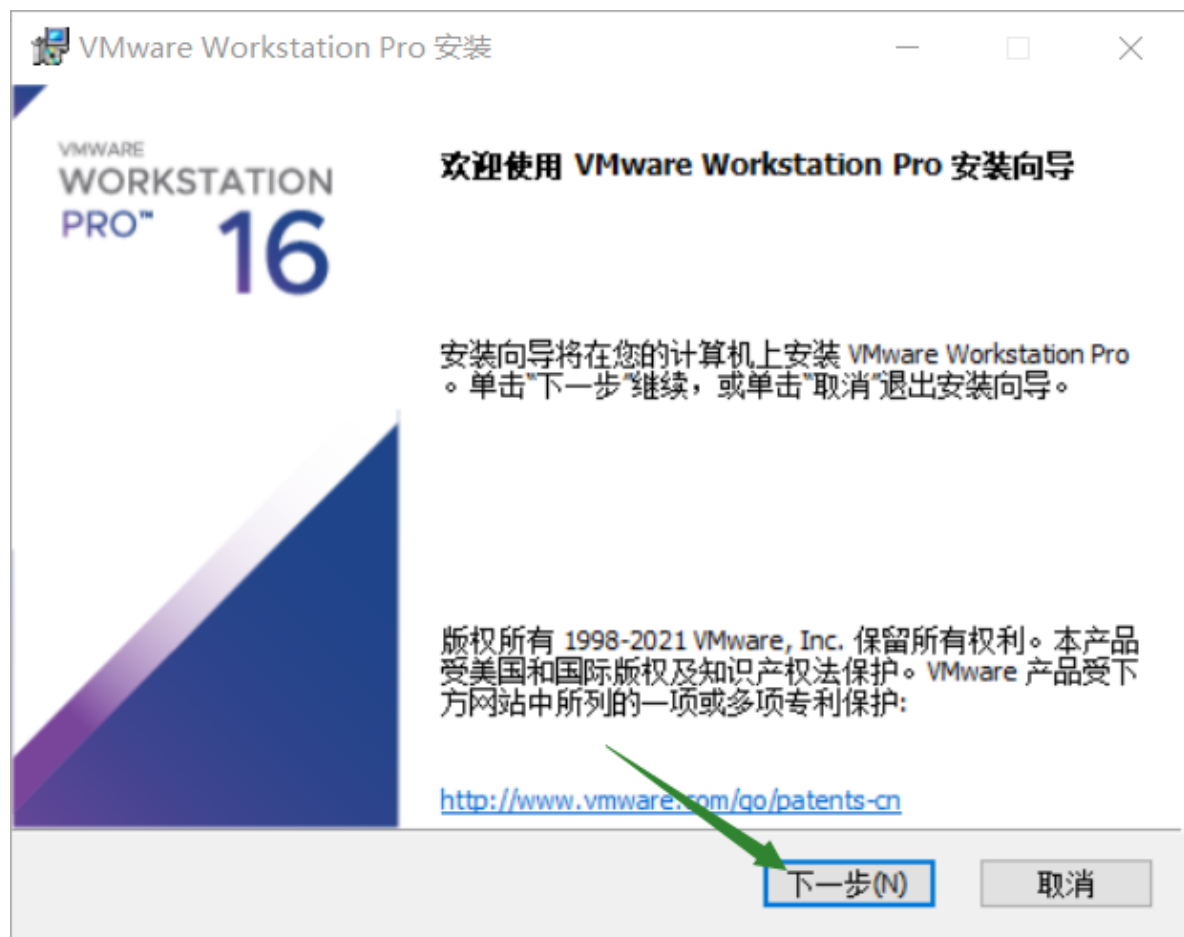
1> 虚拟机安装

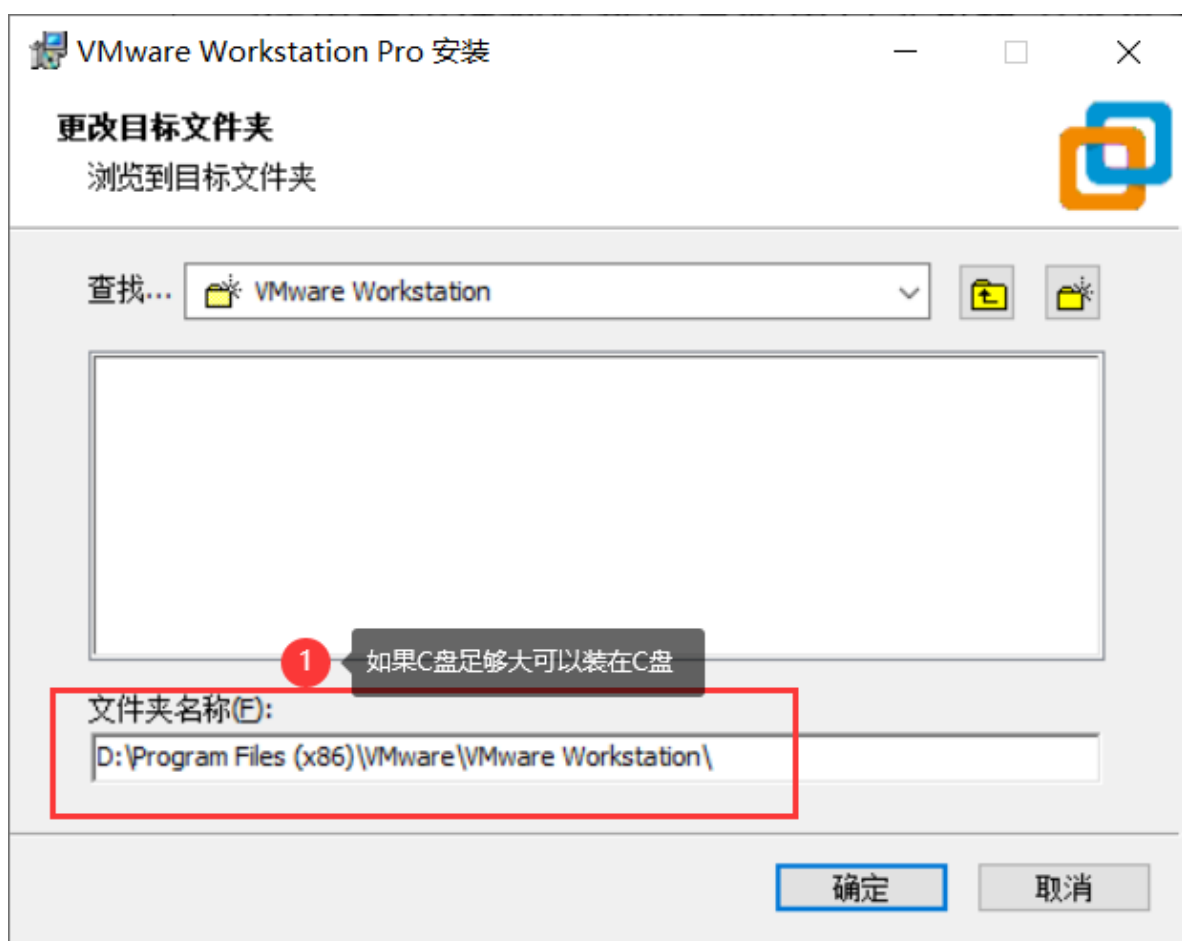
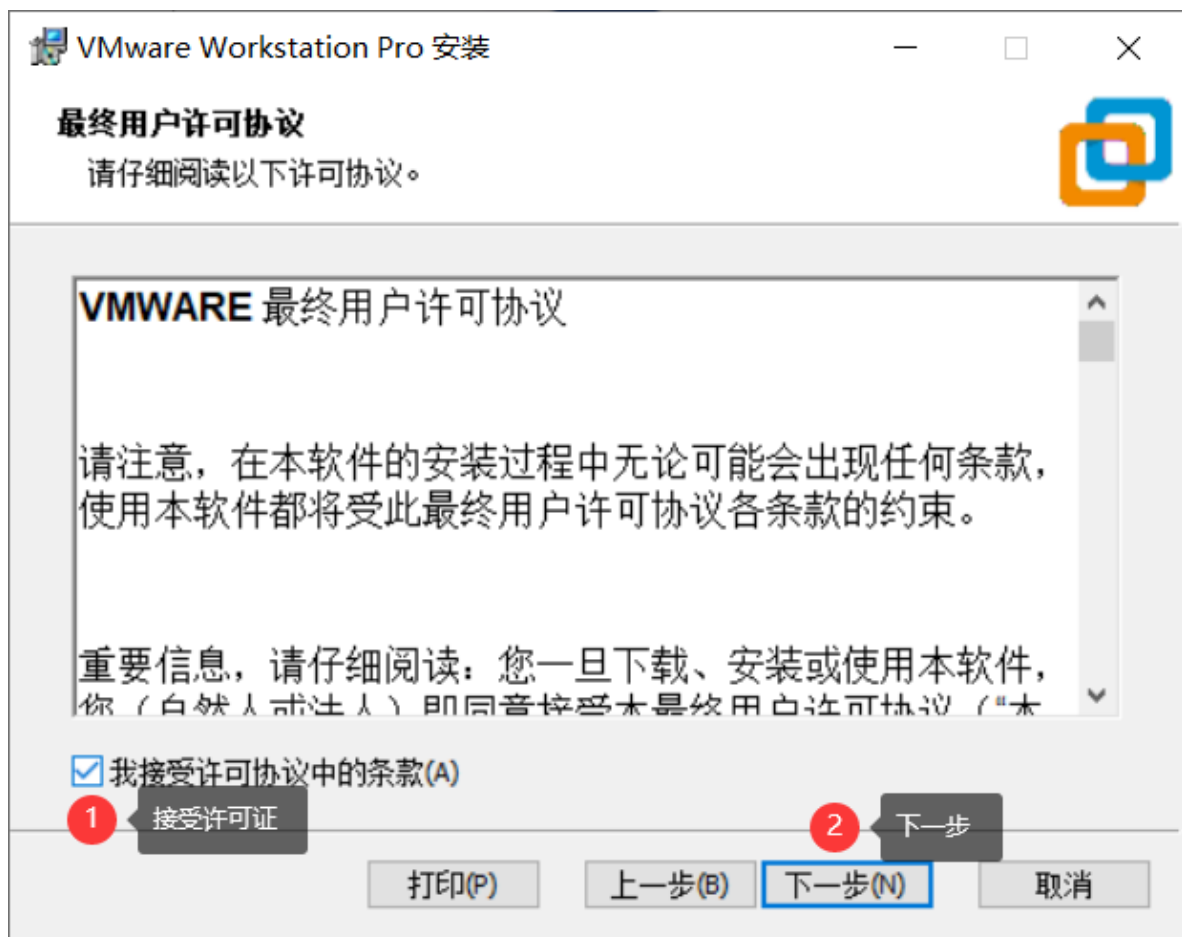
资料链接: https://pan.baidu.com/s/1zUUQbv0m2yJtr_a445bWVA?pwd=gbau 提取码: gbau

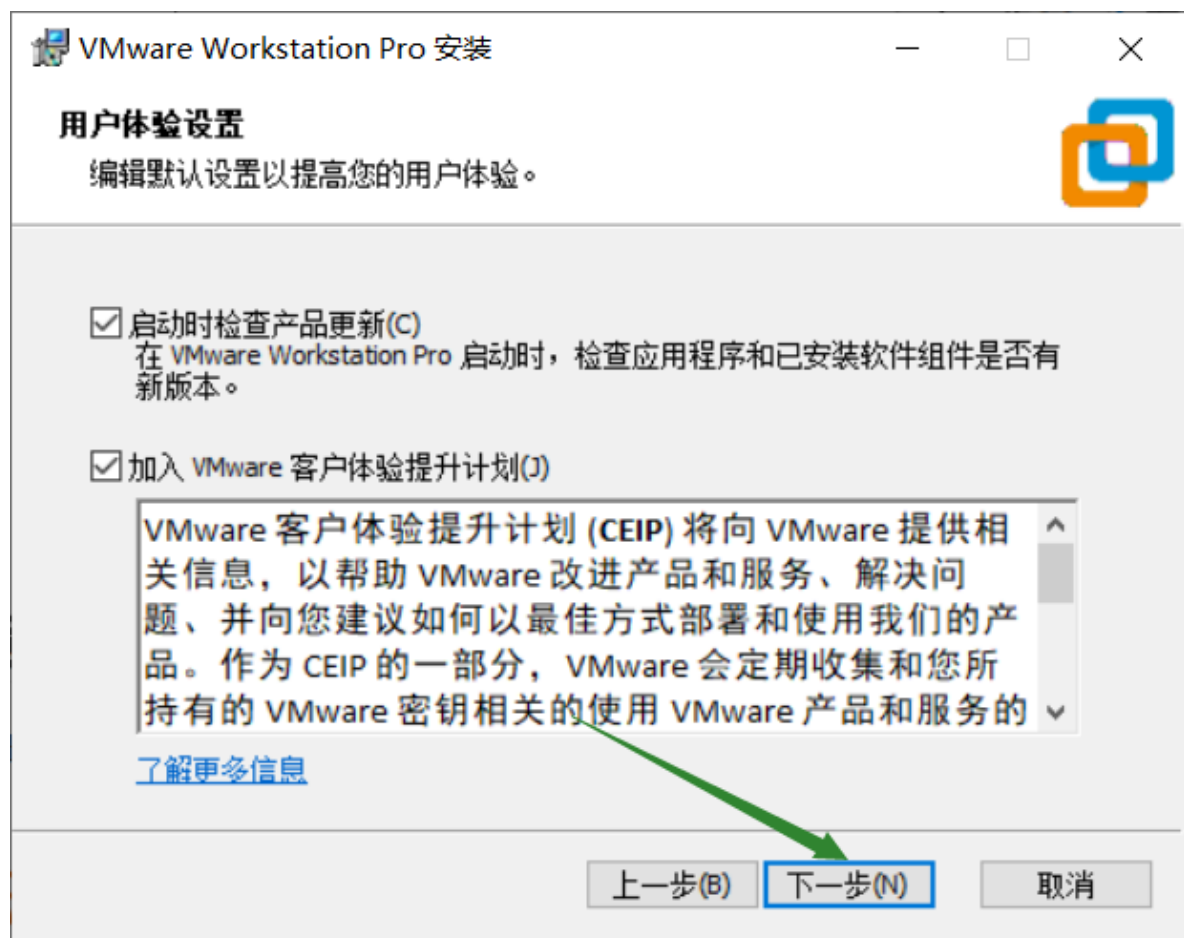
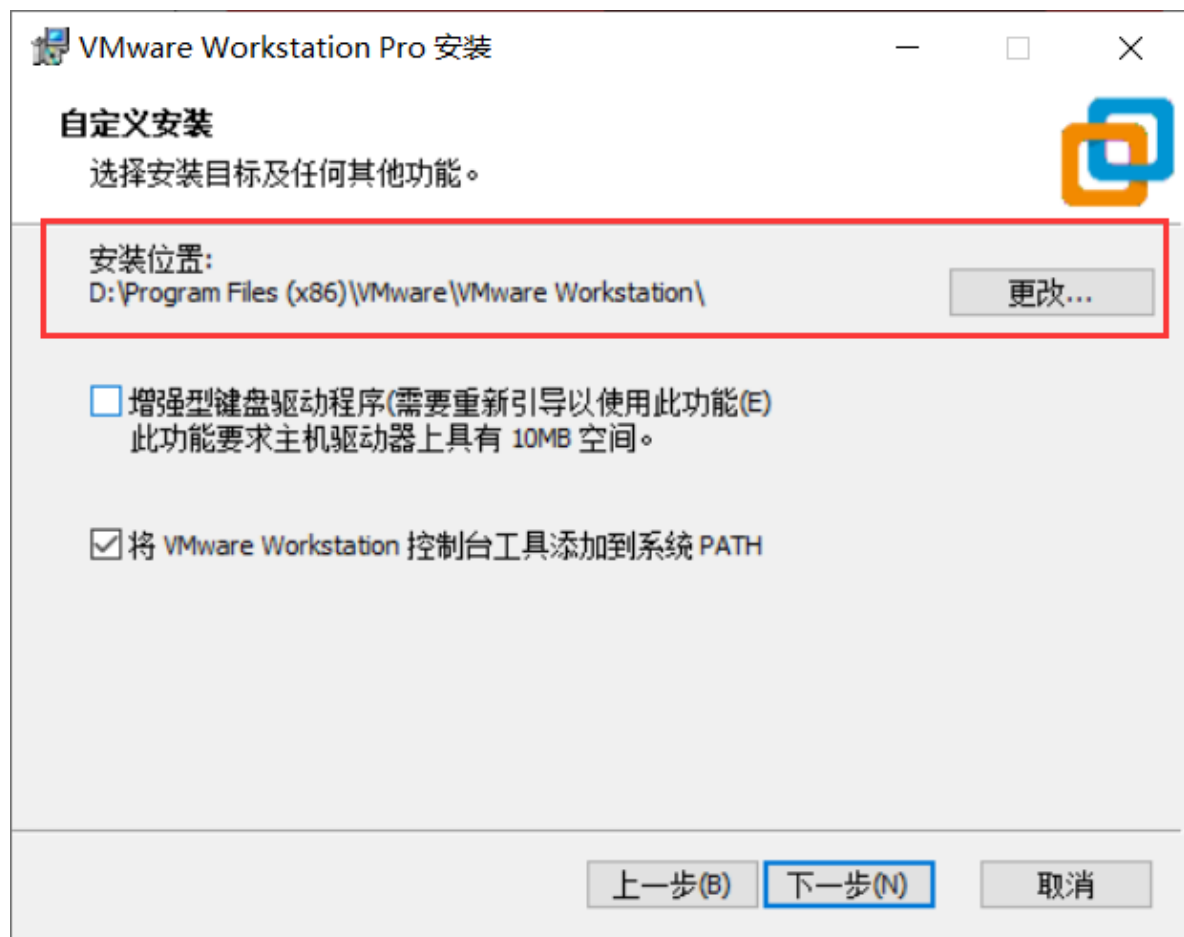
1> 在给定的链接中，找到如下的文件夹进行下载

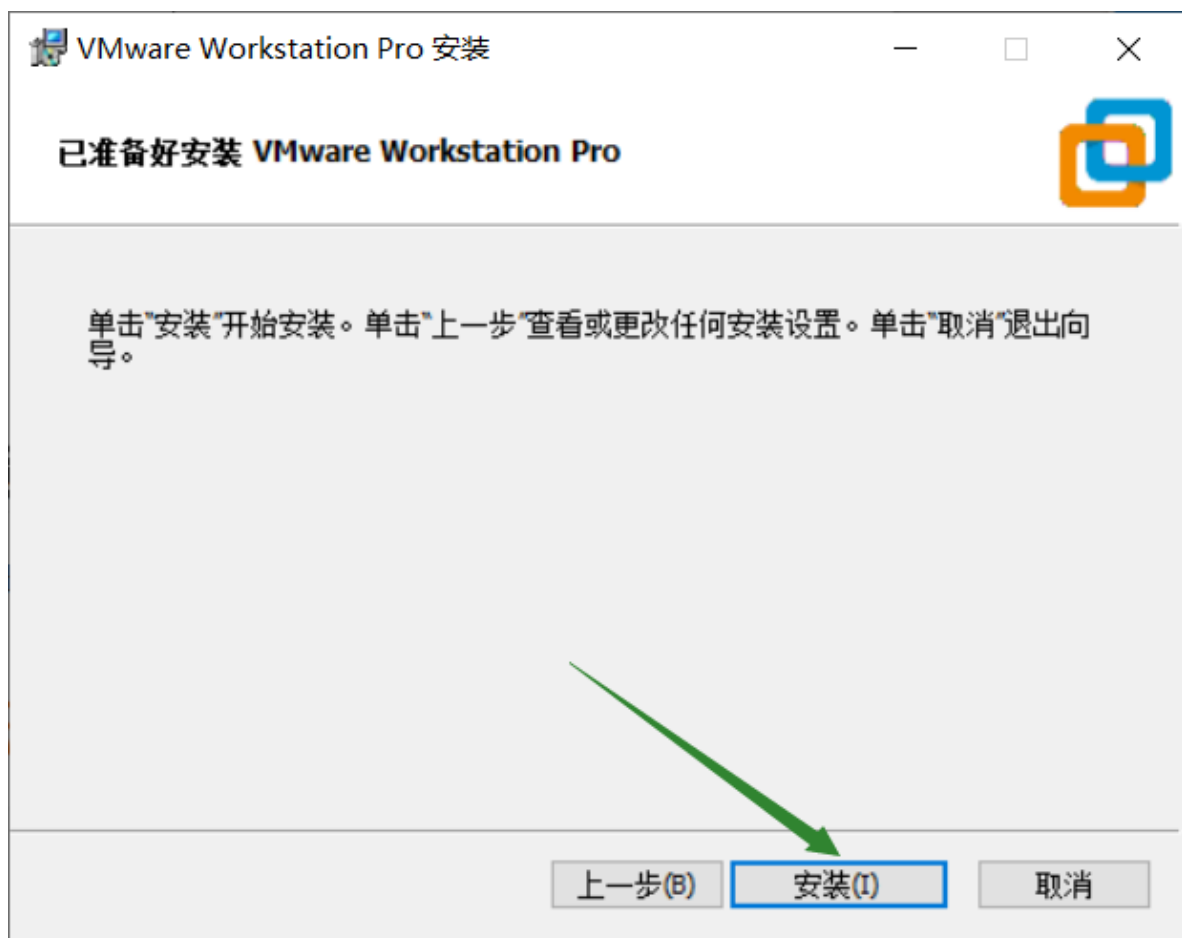
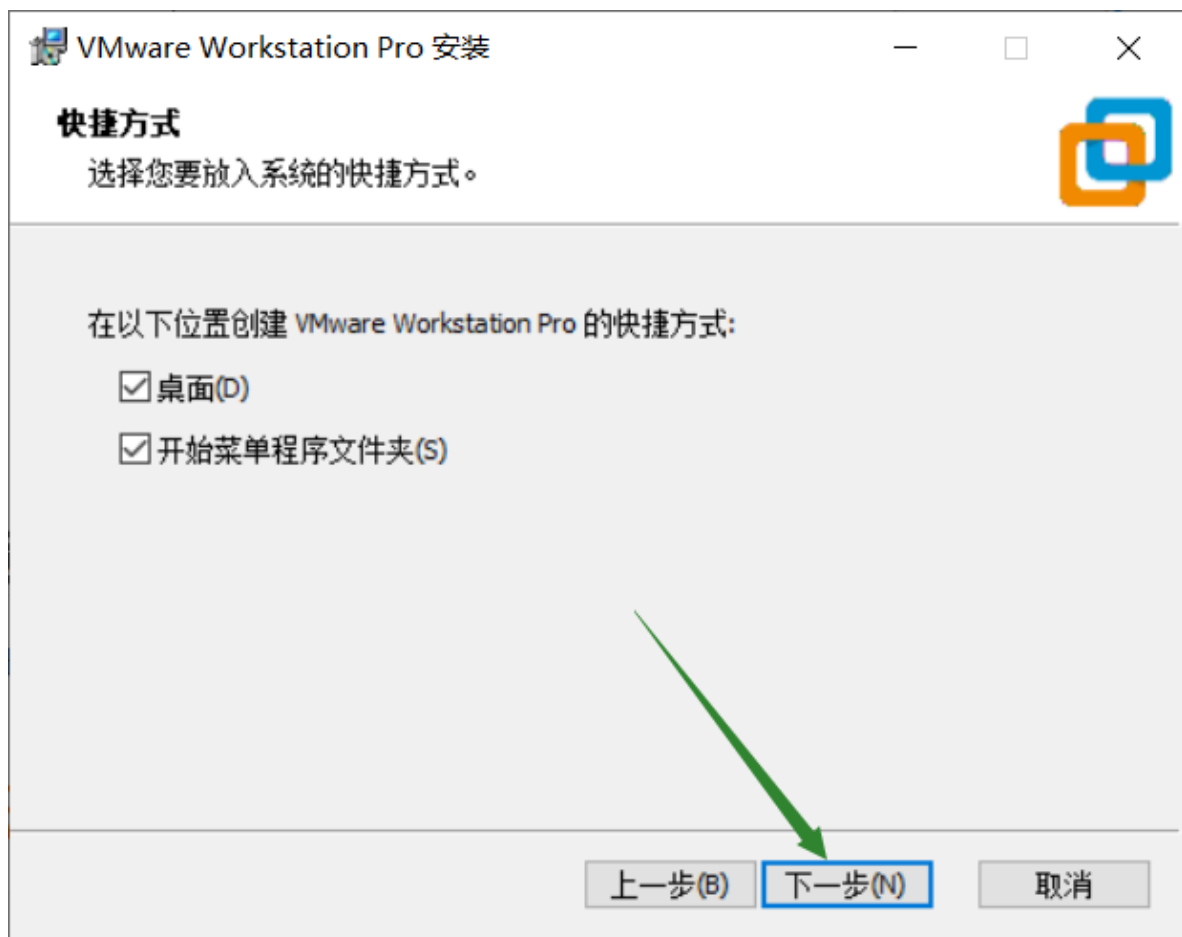
☐	 注册码.txt	122B	2024-03-03 21:50
☐	 VMware-workstation-full-16.1.1-17801498.exe	621.5M	2024-03-03 21:50

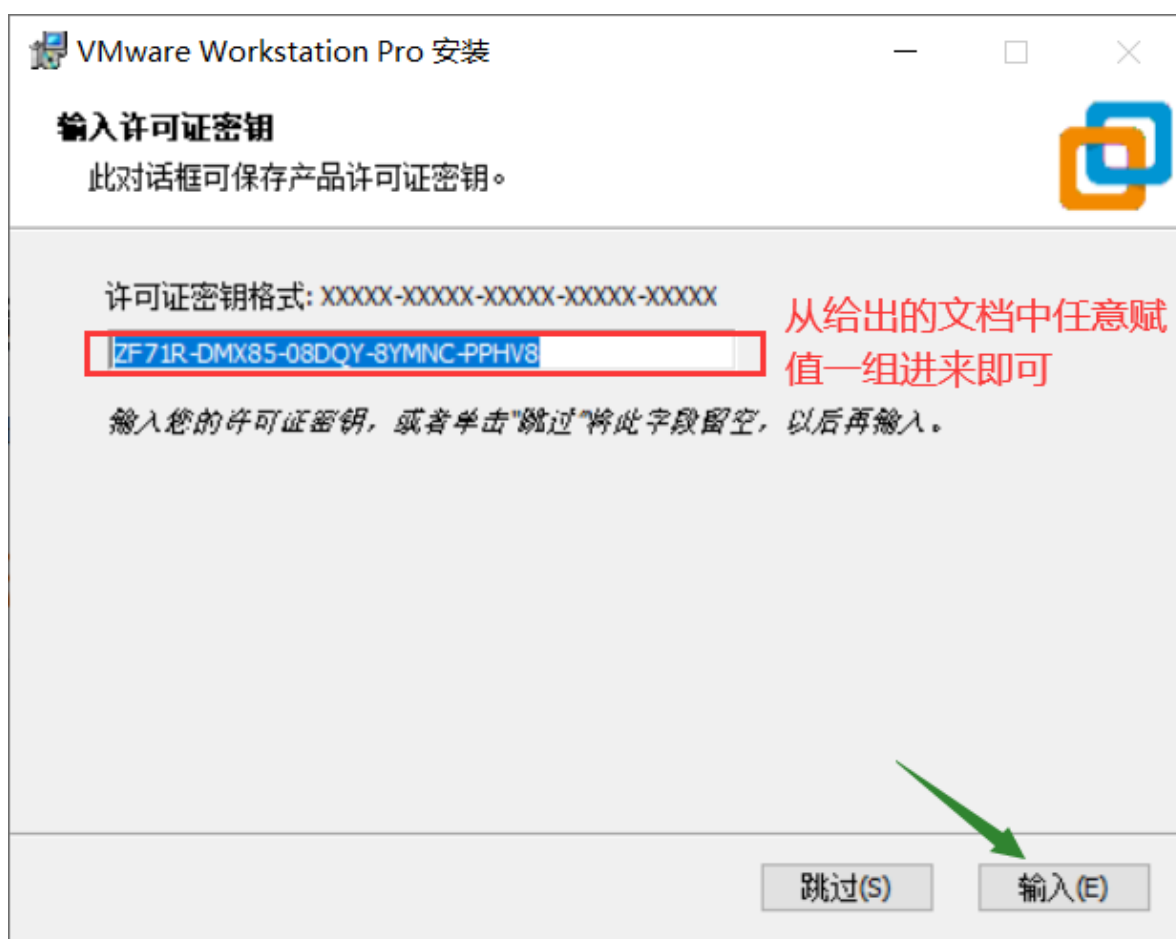
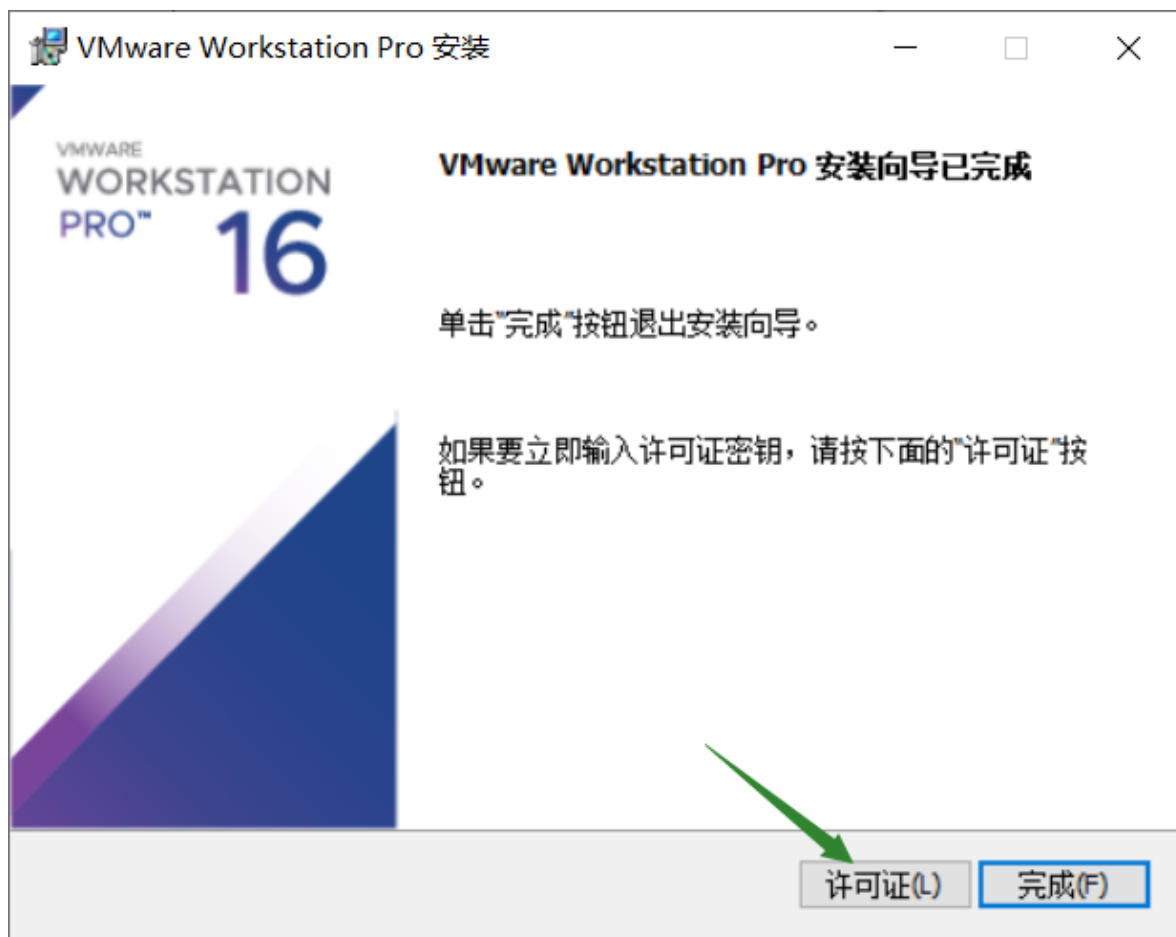
2> 安装虚拟机

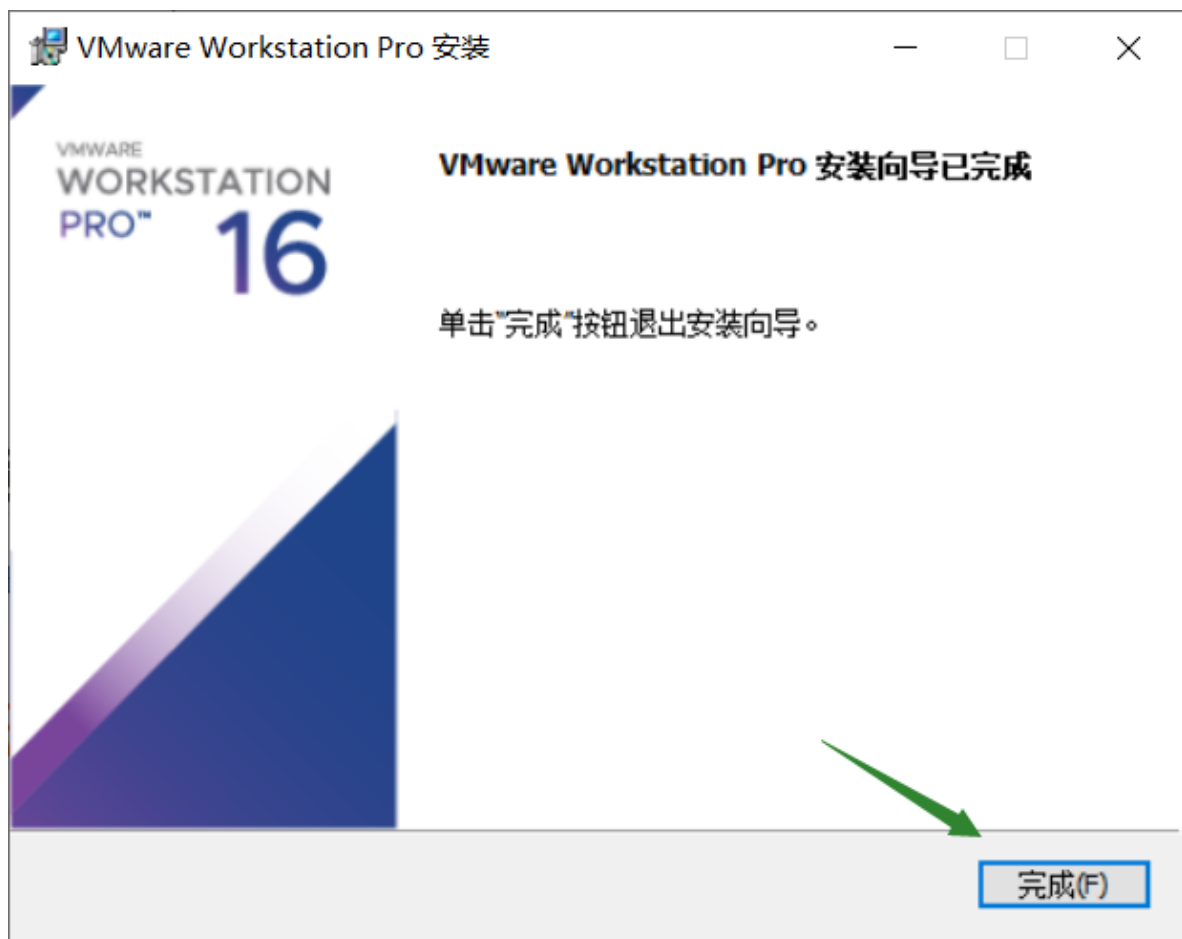




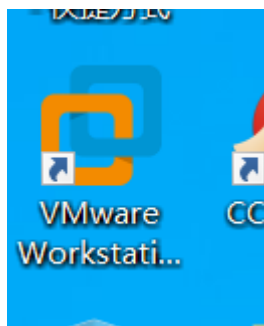








4> 当桌面上出现如下图标时，表示已经安装虚拟机成功



2> 安装CentOS

Centos下载地址：https://mirrors.aliyun.com/centos-vault/8.3.2011/isos/x86_64/?spm=a2c6h.25603864.0.0.989d2ce7LN79OV

Index of /centos-vault/8.3.2011/isos/x86_64/

File Name	File Size	Date
Parent directory/	-	-
CHECKSUM	485.0 B	2020-12-08 21:59
CHECKSUM.asc	1.3 KB	2021-04-02 00:23
CentOS-8.3.2011-x86_64-boot.iso	683.0 MB	2020-11-19 05:01
CentOS-8.3.2011-x86_64-boot.iso.manifest	635.0 B	2020-11-19 05:13
CentOS-8.3.2011-x86_64-boot.torrent	27.2 KB	2020-12-05 00:01
CentOS-8.3.2011-x86_64-dvd1.iso	8.6 GB	2020-11-19 05:43
CentOS-8.3.2011-x86_64-dvd1.iso.manifest	455.0 KB	2020-11-19 05:43
CentOS-8.3.2011-x86_64-dvd1.torrent	372.5 KB	2020-12-05 00:01
CentOS-8.3.2011-x86_64-minimal.iso	1.7 GB	2020-11-19 05:42
CentOS-8.3.2011-x86_64-minimal.iso.manifest	92.5 KB	2020-11-19 05:42

在给定网盘链接中找到：

已选中1个文件/文件夹		修改时间
	XShell安装包	2024-03-03 21:56
	VMware WorkStation16	2024-03-03 21:50
	CentOS-8.3.2011-x86_64-minimal.iso	2025-04-07 14:53
	网络调试工具-飞机.zip	2024-07-20 11:38
	机械臂.zip	2024-04-16 21:39
	tftp服务器.7z	2024-04-16 21:39
	抓包工具.7z	2024-04-16 21:39
	CentOS-7-x86_64-DVD-2009.iso	2024-03-03 22:02
	VSCodeUserSetup-x64-1.87.0.exe	2024-03-03 21:55
	x86_64-8.1.0-release-win32-seh-rt_v6-rev0.7z	2024-03-03 21:55

1> 安装步骤

WORKSTATION 16 PRO™



选择客户机操作系统

此虚拟机中将安装哪种操作系统？

客户机操作系统

- ☐ Microsoft Windows(W)
- ☒ Linux(L)
- ☐ VMware ESX(X)
- ☐ 其他(O)

版本(V)

CentOS 8 64 位

帮助

< 上一步(B)

下一步(N) >

取消

命名虚拟机

您希望该虚拟机使用什么名称？

虚拟机名称(V):

CentOS 8 64 位 mini 版本

位置(L):

C:\Users\CC\Documents\Virtual Machines\CentOS 8 64 位 mini 版

浏览(R)...

在“编辑”>“首选项”中可更改默认位置。

不建议更改路径

< 上一步(B)

下一步(N) >

取消

指定磁盘容量

磁盘大小为多少？

虚拟机的硬盘作为一个或多个文件存储在主机的物理磁盘中。这些文件最初很小，随着您向虚拟机中添加应用程序、文件和数据而逐渐变大。

最大磁盘大小 (GB)(S): 40.0

针对 CentOS 8 64 位 的建议大小: 20 GB

☒ 将虚拟磁盘存储为单个文件(O)☐ 将虚拟磁盘拆分成多个文件(M)

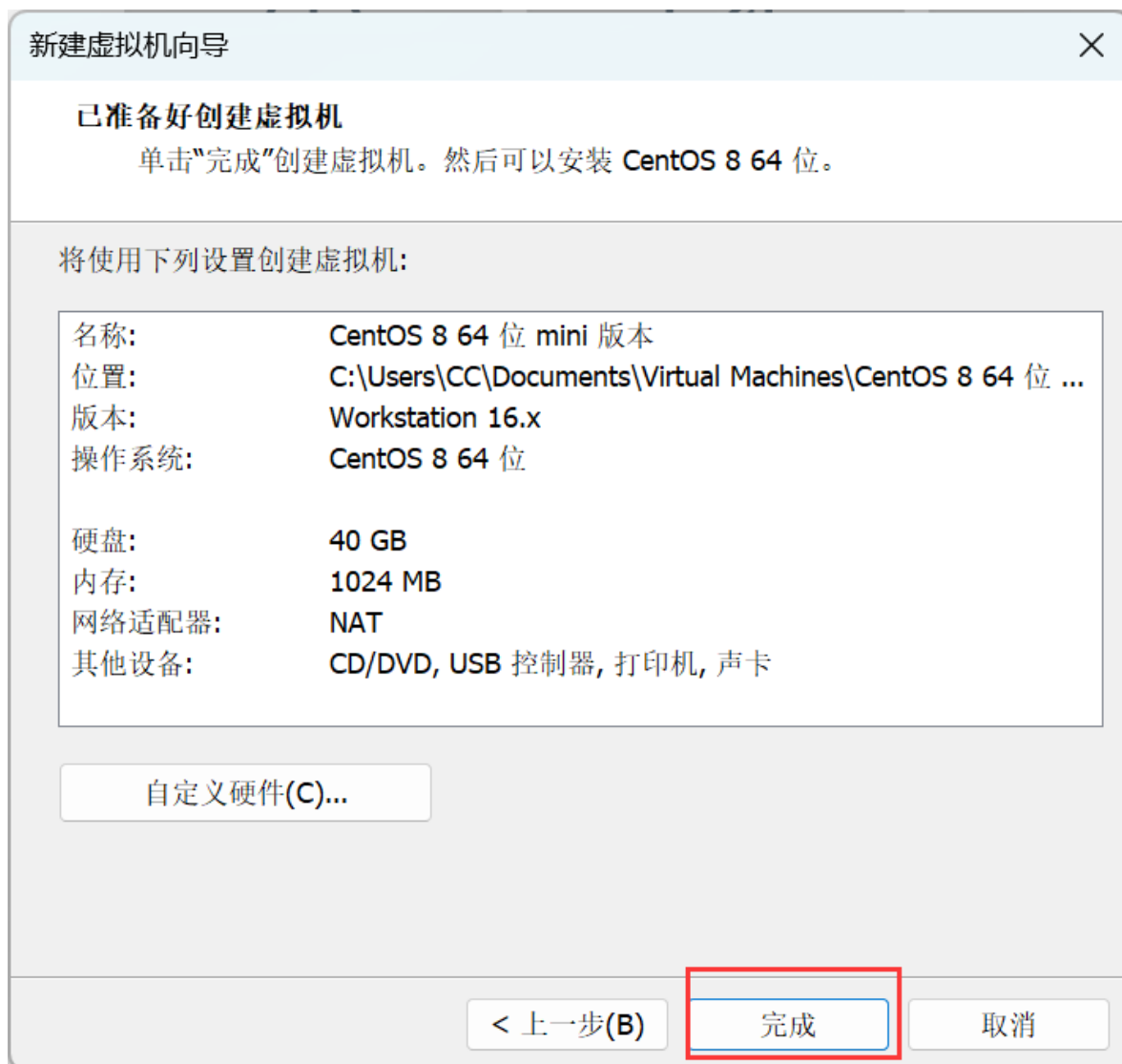
拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

帮助

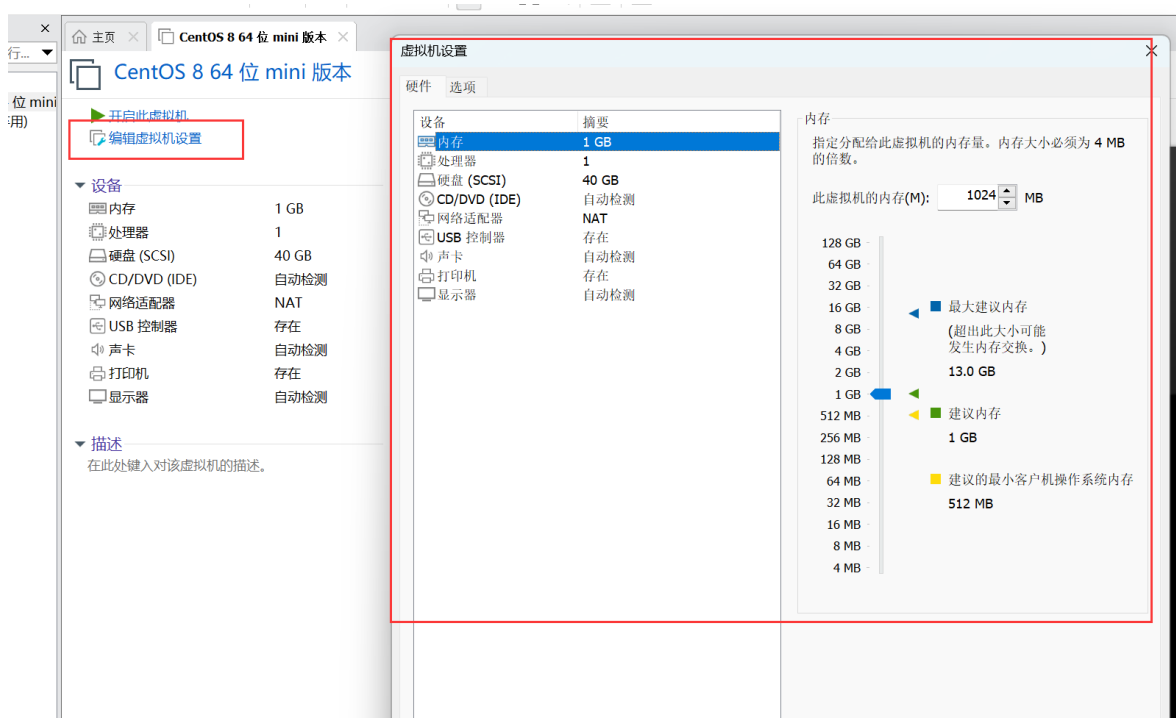
< 上一步(B)

下一步(N) >

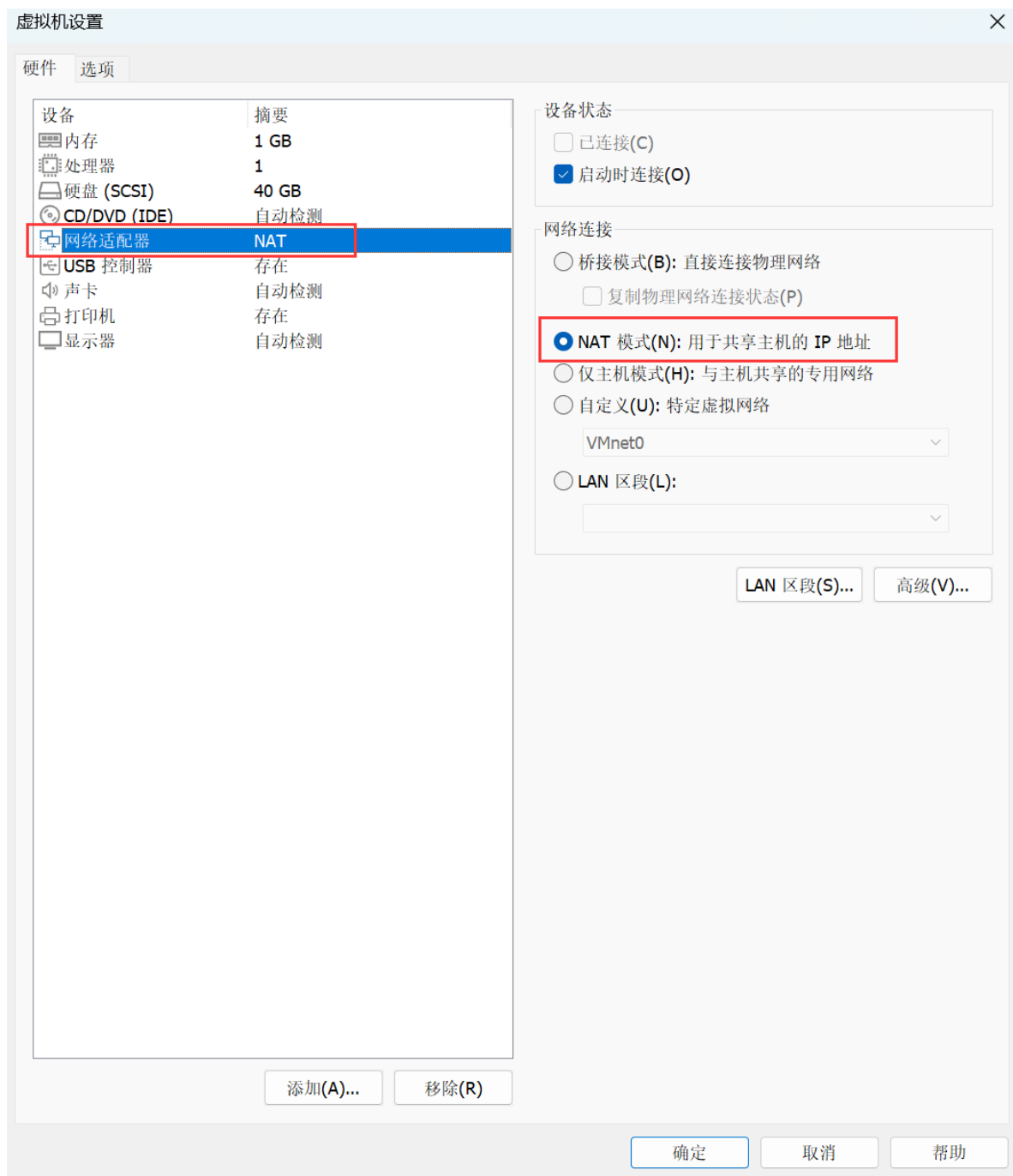
取消



2> 设置虚拟机

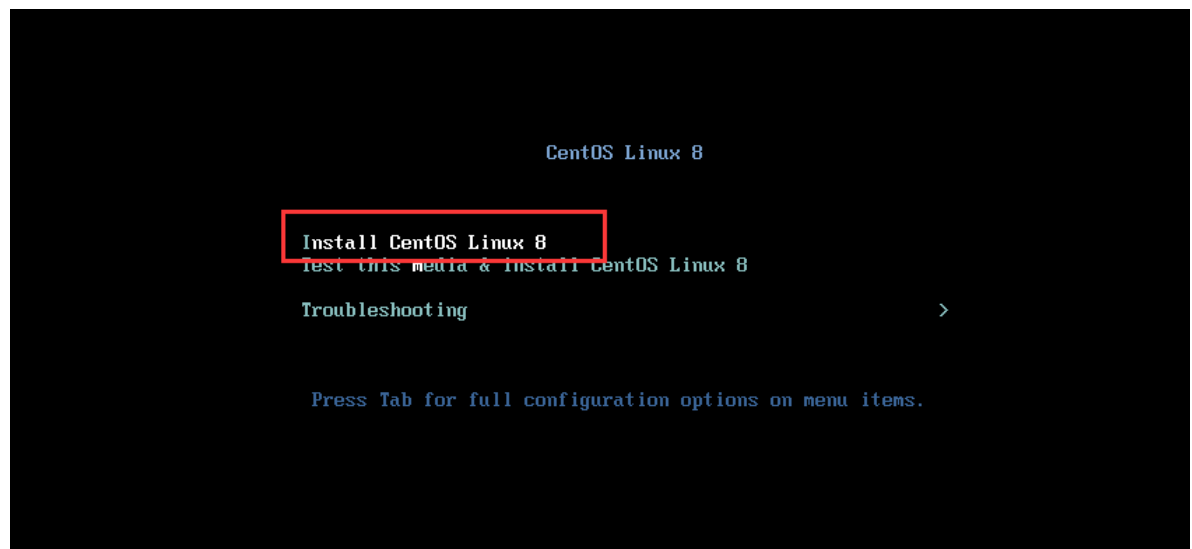
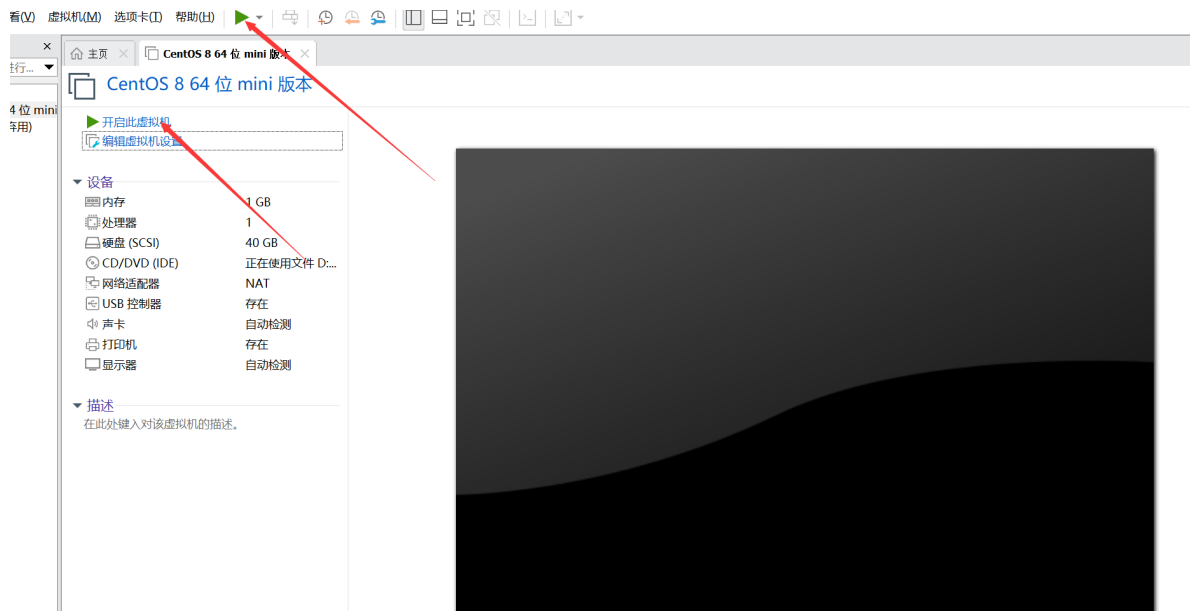






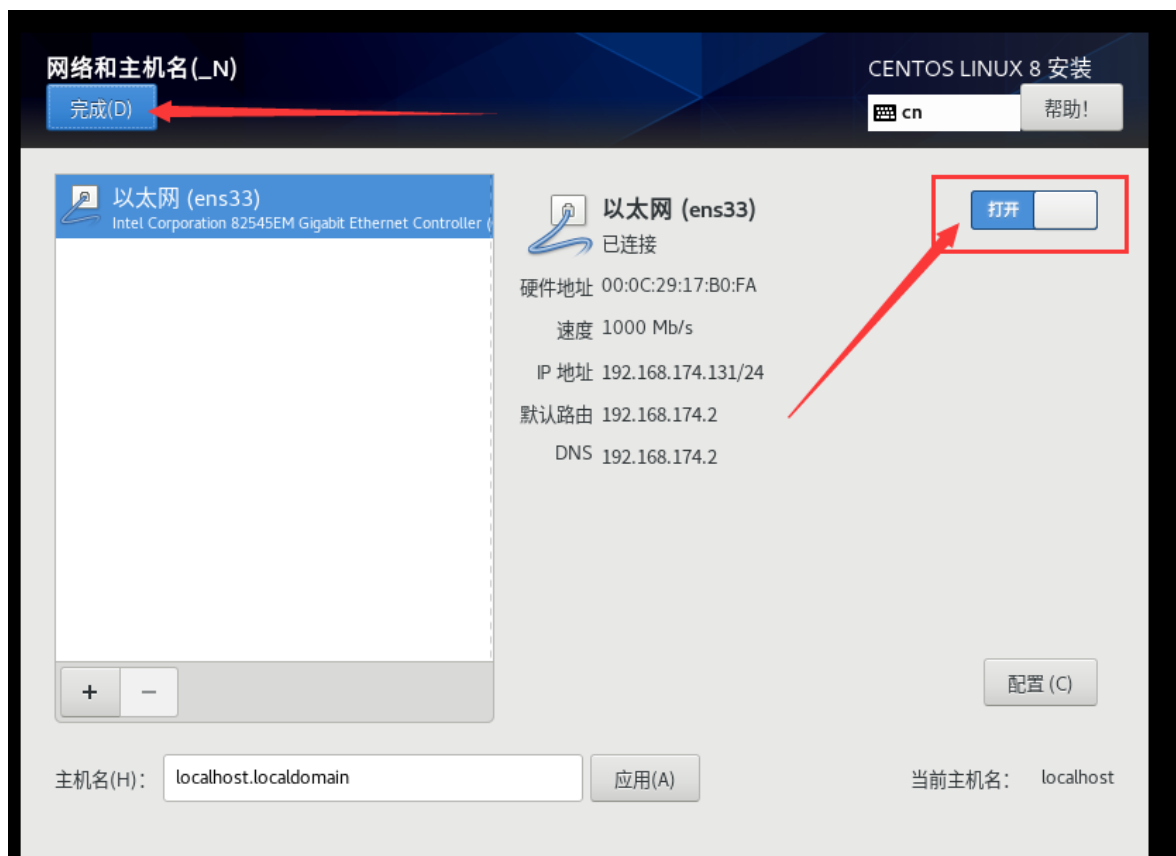
此时虚拟机配置就已经完成了

3> 打开虚拟机，安装linux环境











创建用户

完成(D)

CENTOS LINUX 8 安装

cn帮助!

全名(F)

zpp

用户名

用户名(U)

zpp

☐ 将此用户设为管理员(M)

☒ 需要密码才能使用该帐户(R)

密码(P)

....

用户密码

过短

确认密码(C)

....|

高级(A)...

密码长度太短 必须按两次完成按钮进行确认。

CentOS

安装信息摘要

CENTOS LINUX 8 安装

cn帮助!

键盘(K)

汉语

安装源(I)

本地介质

安装目的地(D)

已选择自动分区

语言支持 (L)

简体中文 (中国)

软件选择 (S)

服务器

KDUMP

已启用 Kdump

时间和日期(T)

美洲/纽约 时区

网络和主机名(N)

有线 (ens33) 已连接

安全策略 (S)

没有找到内容

用户设置

根密码(R)

已经设置 root 密码

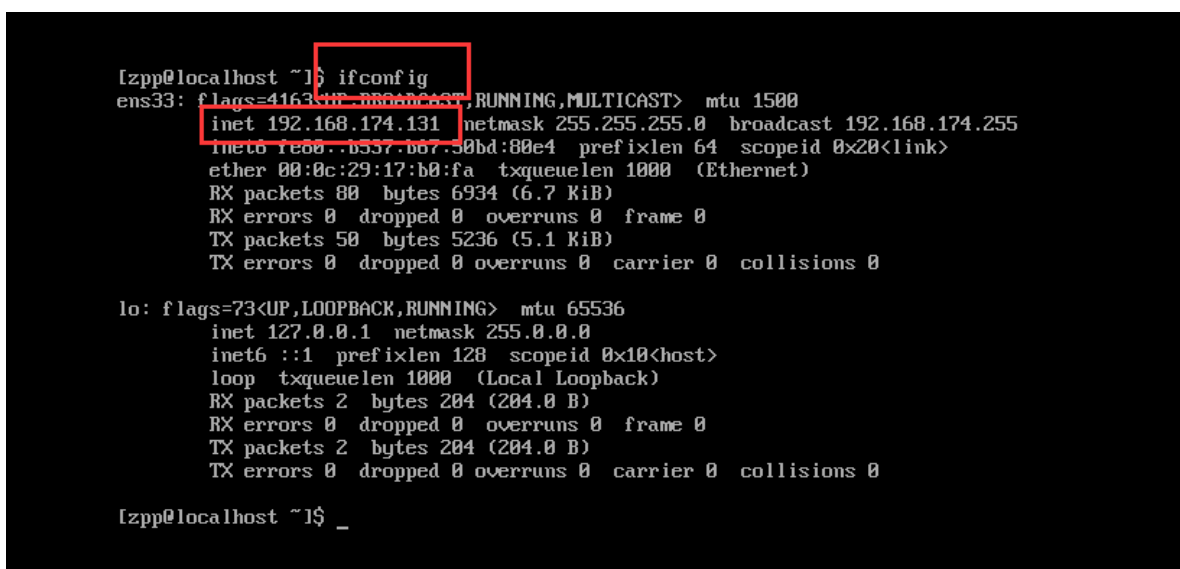
创建用户(U)

将创建用户 zpp

退出(Q)

开始安装(B)

在点击“开始安装”按钮前我们并不会操作您的磁盘。



4> 在普通用户中，设置管理员权限 更改sudoers文件

```
[zpp@localhost ~]$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.174.131 netmask 255.255.255.0 broadcast 192.168.174.255
    inet6 fe80::b537:b67:59bd:80e4 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:17:b0:fa txqueuelen 1000 (Ethernet)
    RX packets 80 bytes 6934 (6.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 50 bytes 5236 (5.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2 bytes 204 (204.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2 bytes 204 (204.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[zpp@localhost ~]$ sudo yum install gcc-c++ -y
```

使用管理员权限安装相关软件

```
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

[sudo] password for zpp:
zpp is not in the sudoers file. This incident will be reported.
[zpp@localhost ~]$ _
```

输入管理员密码

zpp用户不在sudoers文件中

切换到管理员用户下：su root

```
#2) Think before you type.
#3) With great power comes great responsibility.

[sudo] password for zpp:
zpp is not in the sudoers file. This incident will be reported.
[zpp@localhost ~]$ su root
Password:
[root@localhost zpp]# _
```

普通用户

从普通用户向管理员用户切换

管理员用户

更改sudoers文件的权限，加上写权限：chmod u+w /etc/sudoers

```
Password:
[root@localhost zpp]# chmod u+w /etc/sudoers
[root@localhost zpp]#
```

修改该文件中的内容：vi /etc/sudoers

```

## Sudoers allows particular users to run various commands as
## the root user, without needing the root password.
##
## Examples are provided at the bottom of the file for collections
## of related commands, which can then be delegated out to particular
## users or groups.
##
## This file must be edited with the 'visudo' command.

## Host Aliases
## Groups of machines. You may prefer to use hostnames (perhaps using
## wildcards for entire domains) or IP addresses instead.
# Host_Alias    FILESERVERS = fs1, fs2
# Host_Alias    MAILSERVERS = smtp, smtp2

## User Aliases
## These aren't often necessary, as you can use regular groups
## (ie, from files, LDAP, NIS, etc) in this file - just use %groupname
## rather than USERALIAS
# User_Alias    ADMINS = jsmith, mikem

## Command Aliases
## These are groups of related commands...

## Networking
# Cmd_Alias    NETWORKING = /sbin/route, /sbin/ifconfig, /bin/ping, /sbin/dhclient, /usr/bin/net, /sbin/ipables, /usr/bin/rfcomm, /usr/bin/wvdial, /sbin/lwconfig, /sbin/mii-tool

## Installation and management of software
# Cmd_Alias    SOFTWARE = /bin/rpm, /usr/bin/up2date, /usr/bin/yum

## Services
# Cmd_Alias    SERVICES = /sbin/service, /sbin/chkconfig, /usr/bin/systemctl start, /usr/bin/systemctl stop, /usr/bin/systemctl reload, /usr/bin/systemctl restart, /usr/bin/systemctl status, /usr/bin/systemctl enable, /usr/bin/systemctl disable

## Updating the locate database
# Cmd_Alias    LOCATE = /usr/bin/updatedb

## Storage
# Cmd_Alias    STORAGE = /sbin/fdisk, /sbin/sfdisk, /sbin/parted, /sbin/partprobe, /bin/mount, /bin/umount

## Delegating permissions
# Cmd_Alias    DELEGATING = /usr/sbin/visudo, /bin/chown, /bin/chmod, /bin/chgrp

## Processes
# Cmd_Alias    PROCESSES = /bin/nice, /bin/kill, /usr/bin/kill, /usr/bin/killall
"/etc/sudoers" 12BL, 432BC

```

文件内容

使用上下键找到以 root ALL= ... 开头的一行内容

```

##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)    ALL

## Allows members of the 'sys' group to run networking, software,
## service management apps and more.
# %sys ALL = NETWORKING, SOFTWARE, SERVICES, STORAGE, DELEGATING, PROCESSES, LOCATE, DRIVERS

## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)    ALL

```

将光标放在该行，按 yy 键，然后按 p 键

在复制出来的那行中，按 a 键进入编辑模式，将该行中的root改成自己的用户名 zpp

```

## which machines (the sudoers file can be shared between multiple
## systems).
## Syntax:
##
##      user    MACHINE=COMMANDS
##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)    ALL
zpp_    ALL=(ALL)    ALL

## Allows members of the 'sys' group to run networking, software,
## service management apps and more.
# %sys ALL = NETWORKING, SOFTWARE, SERVICES, STORAGE, DELEGATING, PROCESSES, LOCATE, DRIVERS

## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)    ALL

## Same thing without a password

```

按Esc键后 ----> shift + : 后输入wq保存并退出即可

```

Defaults    env_reset
Defaults    env_keep = "COLORS DISPLAY HOSTNAME HISTSIZE KDEDIR LS_COLORS"
Defaults    env_keep += "MAIL PS1 PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE"
Defaults    env_keep += "LC_COLLATE LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES"
Defaults    env_keep += "LC_MONETARY LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE"
Defaults    env_keep += "LC_TIME LC_ALL LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY"

#
# Adding HOME to env_keep may enable a user to run unrestricted
# commands via sudo.
#
# Defaults    env_keep += "HOME"

Defaults    secure_path = /sbin:/bin:/usr/sbin:/usr/bin

## Next comes the main part: which users can run what software on
## which machines (the sudoers file can be shared between multiple
## systems).
## Syntax:
##
##      user    MACHINE=COMMANDS
##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)    ALL
zpp     ALL=(ALL)    ALL

## Allows members of the 'sys' group to run networking, software,
## service management apps and more.
# %sys ALL = NETWORKING, SOFTWARE, SERVICES, STORAGE, DELEGATING, PROCESSES, LOCATE, DRIVERS

## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)    ALL

## Same thing without a password
# %wheel    ALL=(ALL)    NOPASSWD: ALL

## Allows members of the users group to mount and unmount the
## cdrom as root
# %users    ALL=/sbin/mount /mnt/cdrom, /sbin/umount /mnt/cdrom

## Allows members of the users group to shutdown this system
# %users    localhost=/sbin/shutdown -h now

## Read drop in files from /etc/sudoers.d (the # here does not mean a comment)
:uq

```

将sudoers文件的写权限去除：chmod u-w /etc/sudoers

回到普通用户权限：指令exit 或者 ctrl + D快捷键

5> 切换linux的镜像源

默认官方的centos 8镜像源已经不可用了，需要切换到国内的镜像源（阿里云）

```

[root@localhost zpp]# chmod u-w /etc/sudoers
[root@localhost zpp]# exit
exit
[zpp@localhost ~]$ sudo yum install gcc-c++ -y
[sudo] password for zpp:
CentOS Linux 8 - AppStream
Errors during downloading metadata for repository 'appstream':
- Curl error (6): Could not resolve host name for http://mirrorlist.centos.org/?release=8&arch=x86_64&repo=appStream&infra=stock [Could not resolve host: mirrorlist.centos.org]
Error: Failed to download metadata for repo 'appstream': Cannot prepare internal mirrorlist: Curl error (6): Could not resolve host name for http://mirrorlist.centos.org/?release=8&arch=x86_64&repo=appStream&infra=stock [Could not resolve host: mirrorlist.centos.org]
[zpp@localhost ~]$ _

```

切换步骤

备份原有的仓库配置：

```

sudo mkdir /etc/yum.repos.d/backup    #创建一个用于存放备份的文件夹
sudo mv /etc/yum.repos.d/CentOS-*    /etc/yum.repos.d/backup    #备份文件

```

```

[zpp@localhost ~]$ sudo mkdir /etc/yum.repos.d/backup
[zpp@localhost ~]$ sudo mv /etc/yum.repos.d/CentOS-* /etc/yum.repos.d/backup
[zpp@localhost ~]$

```

创建一个存放备份文件的文件夹

将原有的镜像文件放入到备份文件中

下载阿里云的镜像文件

```
sudo curl -o /etc/yum.repos.d/CentOS-Base.repo
https://mirrors.aliyun.com/repo/Centos-vault-8.5.2111.repo
```

```
lzp@localhost ~]$ sudo mkdir /etc/yum.repos.d/backup
lzp@localhost ~]$ sudo mv /etc/yum.repos.d/CentOS-* /etc/yum.repos.d/backup
lzp@localhost ~]$ sudo curl -o /etc/yum.repos.d/CentOS-Base.repo https://mirrors.aliyun.com/repo/Centos-vault-8.5.2111.repo
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 2495 100 2495 0 0 17206 0 --:--:-- --:--:-- --:--:-- 17206
lzp@localhost ~]$
```

清理缓存并重建元数据

```
sudo dnf clean all #清除缓存
sudo dnf makecache #创建元数据
```

```
lzp@localhost ~]$ sudo mkdir /etc/yum.repos.d/backup
lzp@localhost ~]$ sudo mv /etc/yum.repos.d/CentOS-* /etc/yum.repos.d/backup
lzp@localhost ~]$ sudo curl -o /etc/yum.repos.d/CentOS-Base.repo https://mirrors.aliyun.com/repo/Centos-vault-8.5.2111.repo
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 2495 100 2495 0 0 17206 0 --:--:-- --:--:-- --:--:-- 17206
lzp@localhost ~]$ sudo dnf clean all
0 files removed
lzp@localhost ~]$ sudo dnf makecache
CentOS-8.5.2111 - Base - mirrors.aliyun.com 5.3 MB/s | 4.6 MB 00:00
CentOS-8.5.2111 - Extras - mirrors.aliyun.com 63 kB/s | 10 kB 00:00
CentOS-8.5.2111 - AppStream - mirrors.aliyun.com 5.7 MB/s | 8.4 MB 00:01
Metadata cache created.
lzp@localhost ~]$
```

6> 有关网络问题

如何判断自己的虚拟机是否有外网，如果出现下面的情况说明有网

```
lzp@localhost ~]$ ping www.baidu.com
PING www.a.shifen.com (180.101.49.44) 56(84) bytes of data:
64 bytes from 180.101.49.44 (180.101.49.44): icmp_seq=1 ttl=128 time=15.4 ms
64 bytes from 180.101.49.44 (180.101.49.44): icmp_seq=2 ttl=128 time=156 ms
64 bytes from 180.101.49.44 (180.101.49.44): icmp_seq=3 ttl=128 time=14.6 ms
64 bytes from 180.101.49.44 (180.101.49.44): icmp_seq=4 ttl=128 time=13.4 ms
64 bytes from 180.101.49.44 (180.101.49.44): icmp_seq=5 ttl=128 time=14.0 ms
```

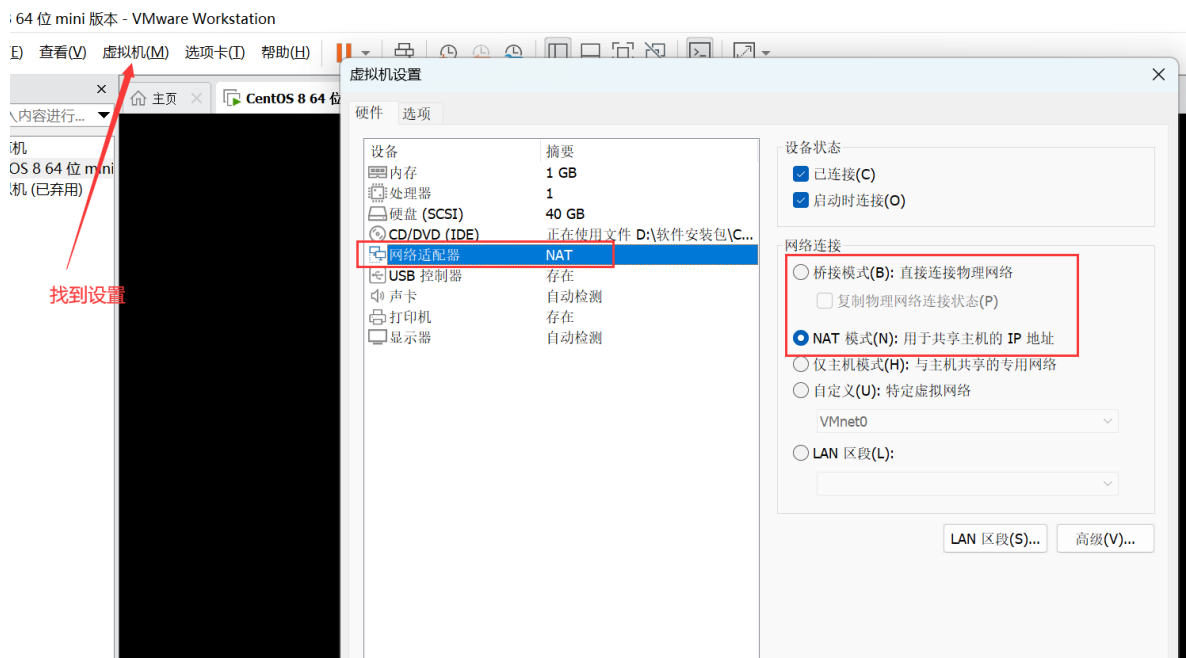
- 1、一直没有ip地址
- 2、虚拟机没有网
- 3、设置后有ip地址，但是关机重启后就没有了

解决方法

- 1、看看虚拟机是否没有连接网络



2、上述连接后还没有网络，查看网络配置器，切换网络模式



3、如果上述操作还不行，需要配置网卡配置

```
cat /etc/sysconfig/network-scripts/ifcfg-ens33
```

```

tazpp@localhost ~1$ cat /etc/sysconfig/network-scripts/ifcfg-ens33
TYPE=Ethernet
PROXY_METHOD=none
BROWSER_ONLY=no
BOOTPROTO=dhcp
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6_INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
IPV6_ADDR_GEN_MODE=stable-privacy
NAME=ens33
UUID=854a1eb9-4371-406c-96c2-cd5f784844b8
DEVICE=ens33
ONBOOT=yes
tazpp@localhost ~1$ _

```

```
sudo vi /etc/sysconfig/network-scripts/ifcfg-ens33
```

按照更改sudoers文件的内容形式改成将上面相关文件信息更改后，保存退出 --->重启虚拟机即可

4、检查IP地址是否存在

```

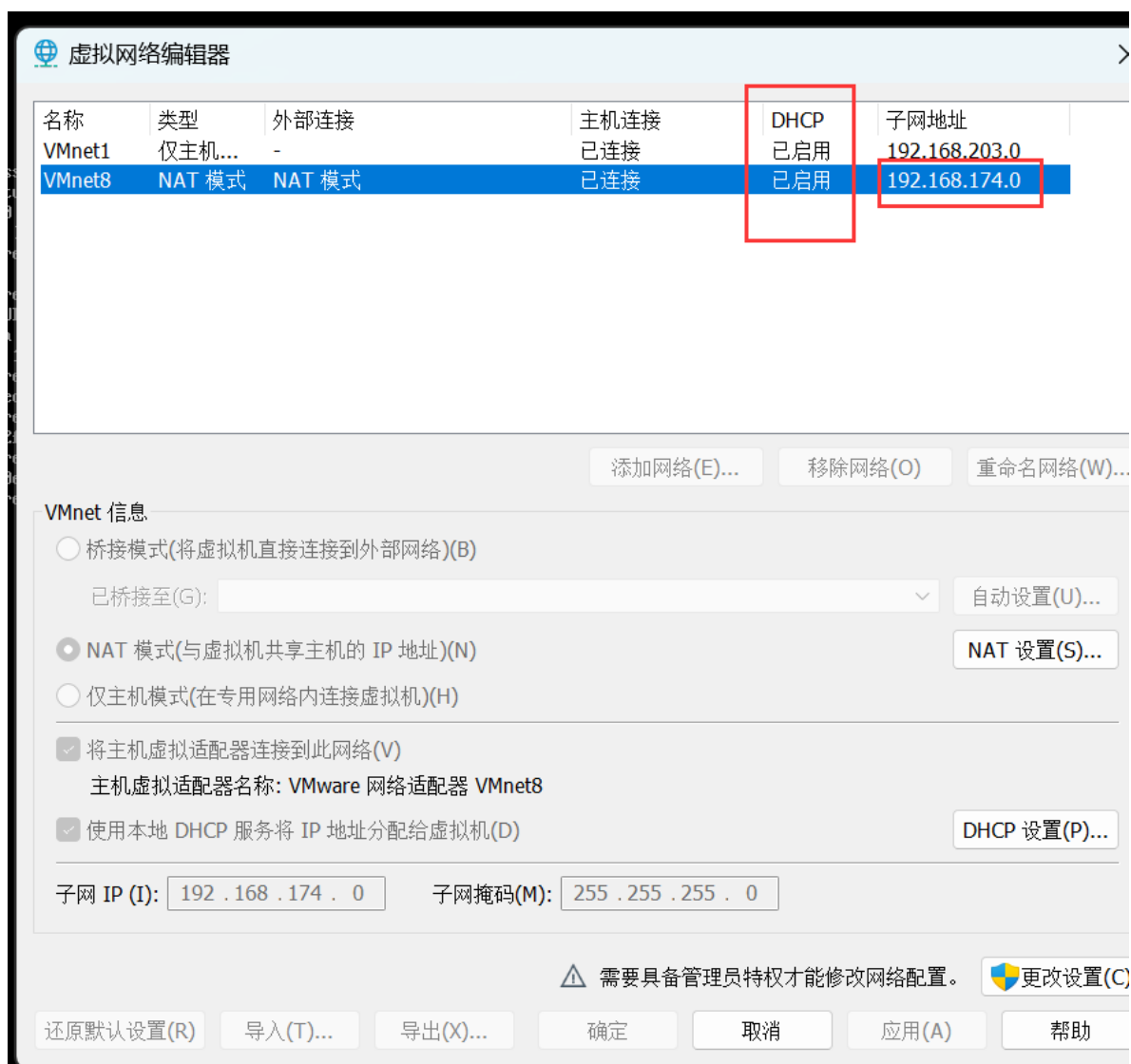
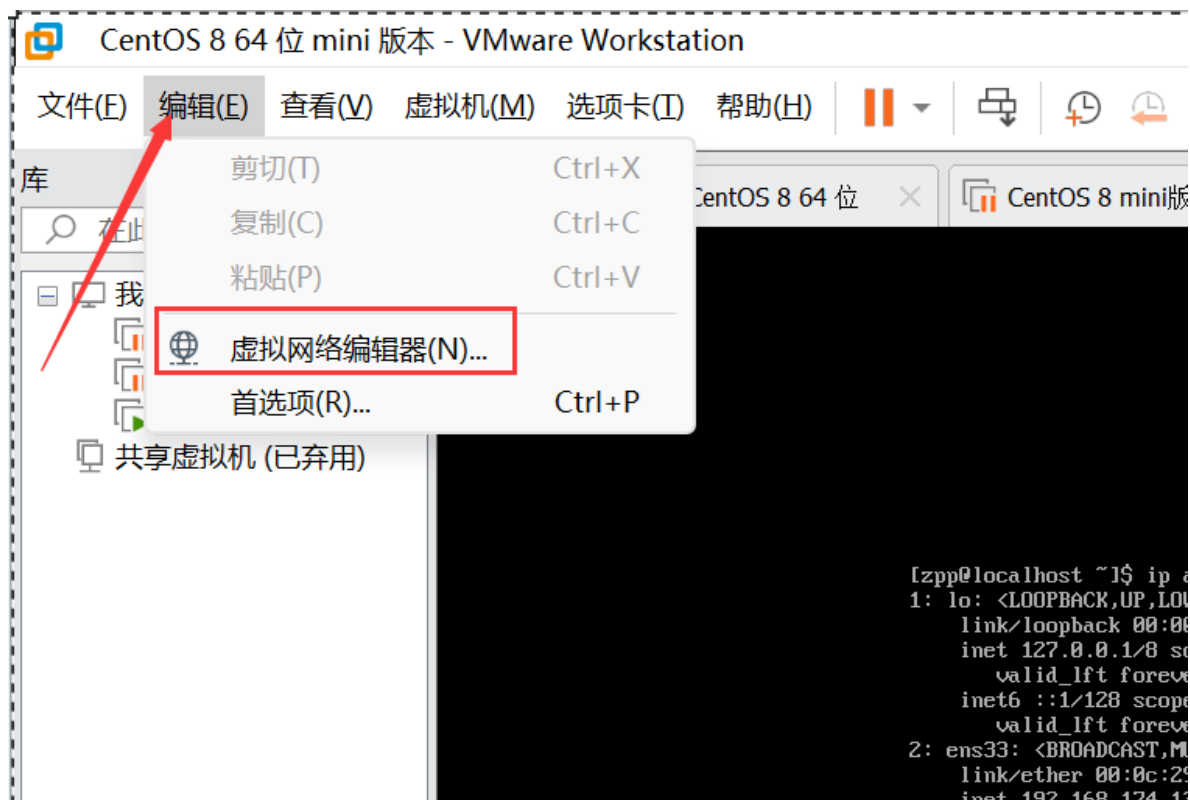
tazpp@localhost ~1$ ip addr | less
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:14:b0:fa brd ff:ff:ff:ff:ff:ff
    inet 192.168.174.131/24 brd 192.168.174.255 scope global dynamic noprefixroute ens33
        valid_lft 1056sec preferred_lft 1056sec
    inet6 2408:8444:8788:31f:3cec:f0b:a82c:8872/64 scope global noprefixroute
        valid_lft forever preferred_lft forever
    inet6 2408:8445:a9a3:6af:1f2f:f32:8d9a:ef11/64 scope global noprefixroute
        valid_lft forever preferred_lft forever
    inet6 fe80::b537:b67:50bd:80e4/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(END)

```

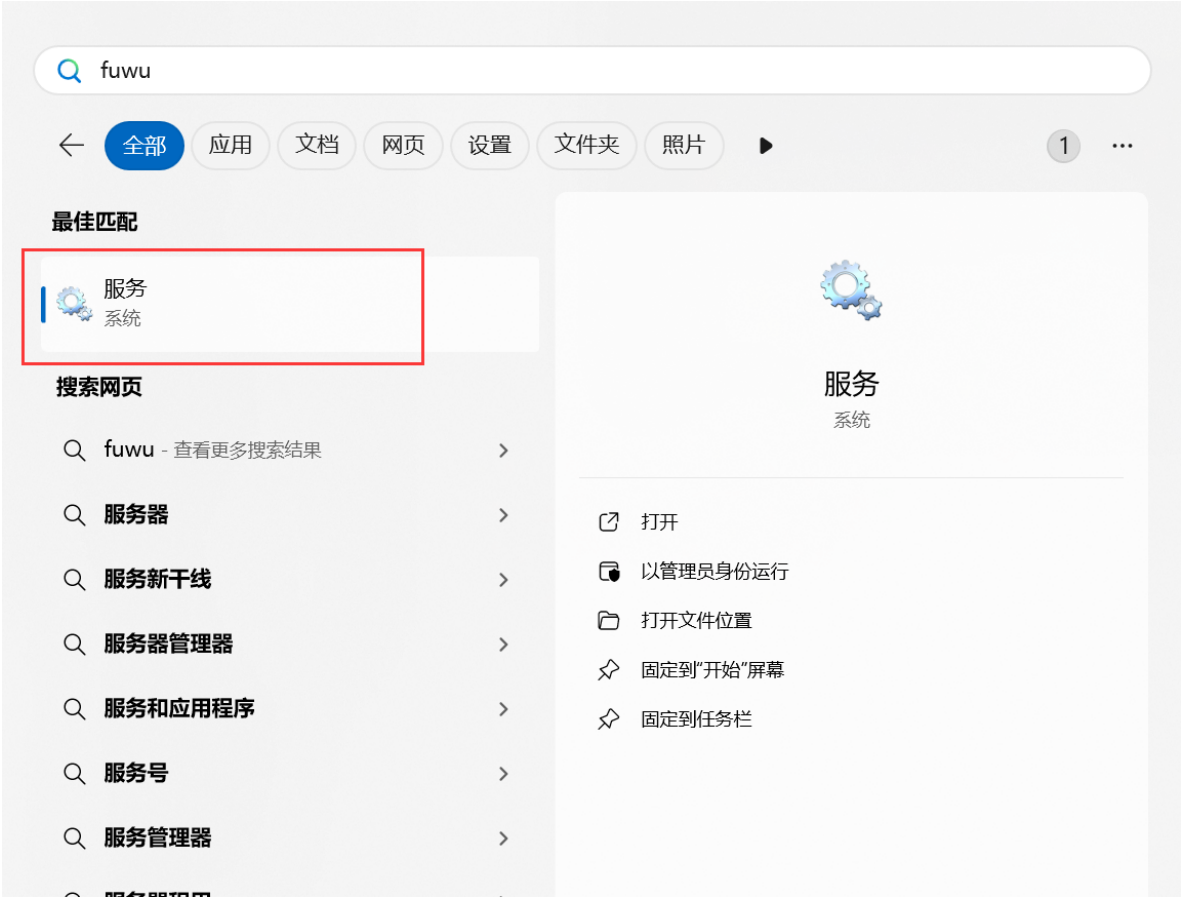
检查IP地址是否存在

不存在的解决方案：

看一下虚拟机的网络编辑器：



看一下虚拟机的服务是否开启：如果没有开启，需要开启一下



看一下网络适配器：

← WLAN



张鹏鹏的iPhone
已连接, 安全



断开连接



Redmi K70 Pro



欢迎来玩咸鱼之王



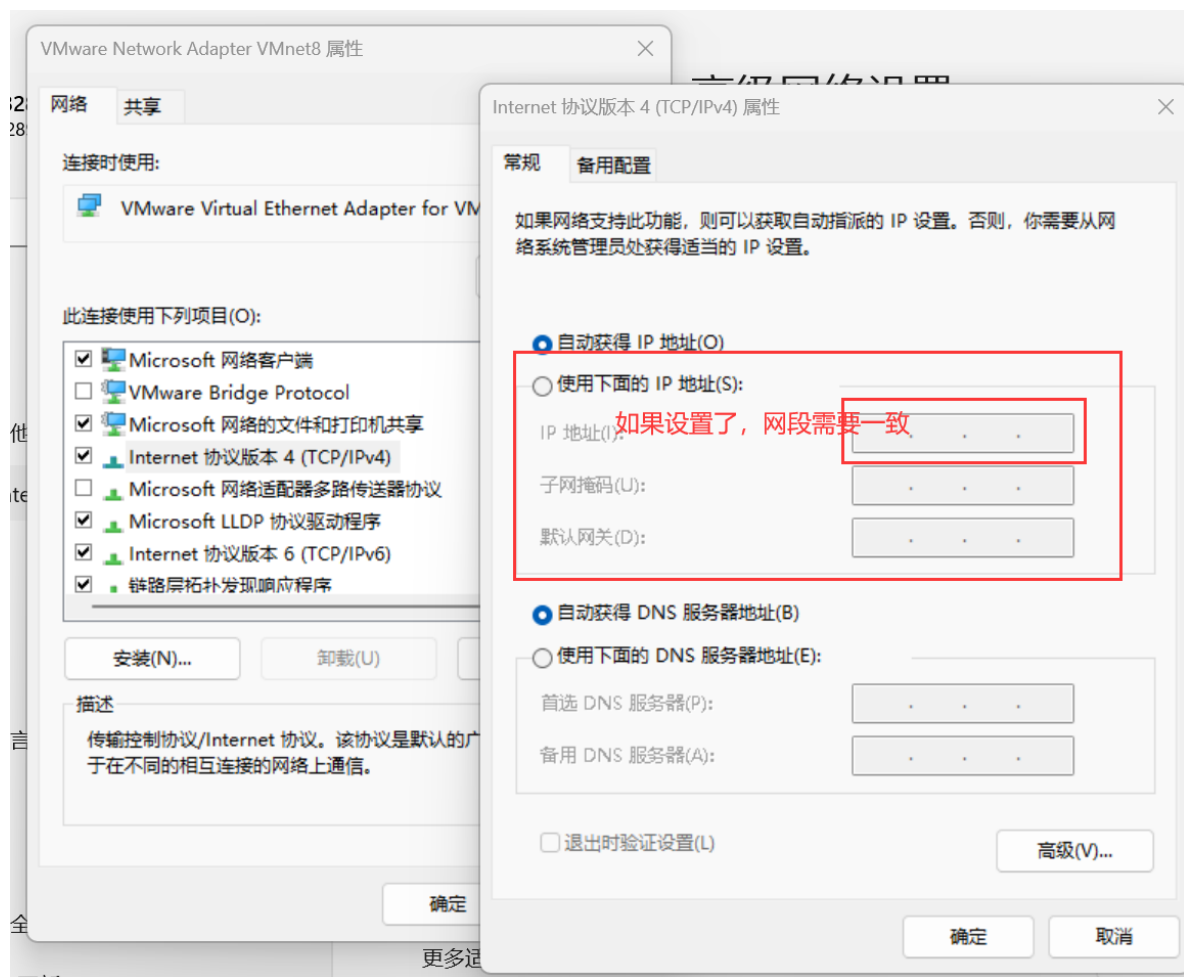
玛卡巴卡



更多 Wi-Fi 设置

更多 Wi-Fi 设置





5、如果上面的操作依然没有解决你的问题：重置一下虚拟机的网络

名称	类型	外部连接	主机连接	DHCP	子网地址
VMnet1	仅主机...	-	已连接	已启用	192.168.203.0
VMnet8	NAT 模式	NAT 模式	已连接	已启用	192.168.174.0

添加网络(E)...

移除网络(O)

重命名网络(W)...

VMnet 信息

☐ 桥接模式(将虚拟机直接连接到外部网络)(B)

已桥接至(G):



自动设置(U)...

☐ NAT 模式(与虚拟机共享主机的 IP 地址)(N)

NAT 设置(S)...

☒ 仅主机模式(在专用网络内连接虚拟机)(H)☒ 将主机虚拟适配器连接到此网络(V)

主机虚拟适配器名称: VMware 网络适配器 VMnet1

☒ 使用本地 DHCP 服务将 IP 地址分配给虚拟机(D)

DHCP 设置(P)...

子网 IP (I): 192 . 168 . 203 . 0

子网掩码(M): 255 . 255 . 255 . 0

⚠ 需要具备管理员特权才能修改网络配置。

更改设置(C)

还原默认设置(R)

导入(T)...

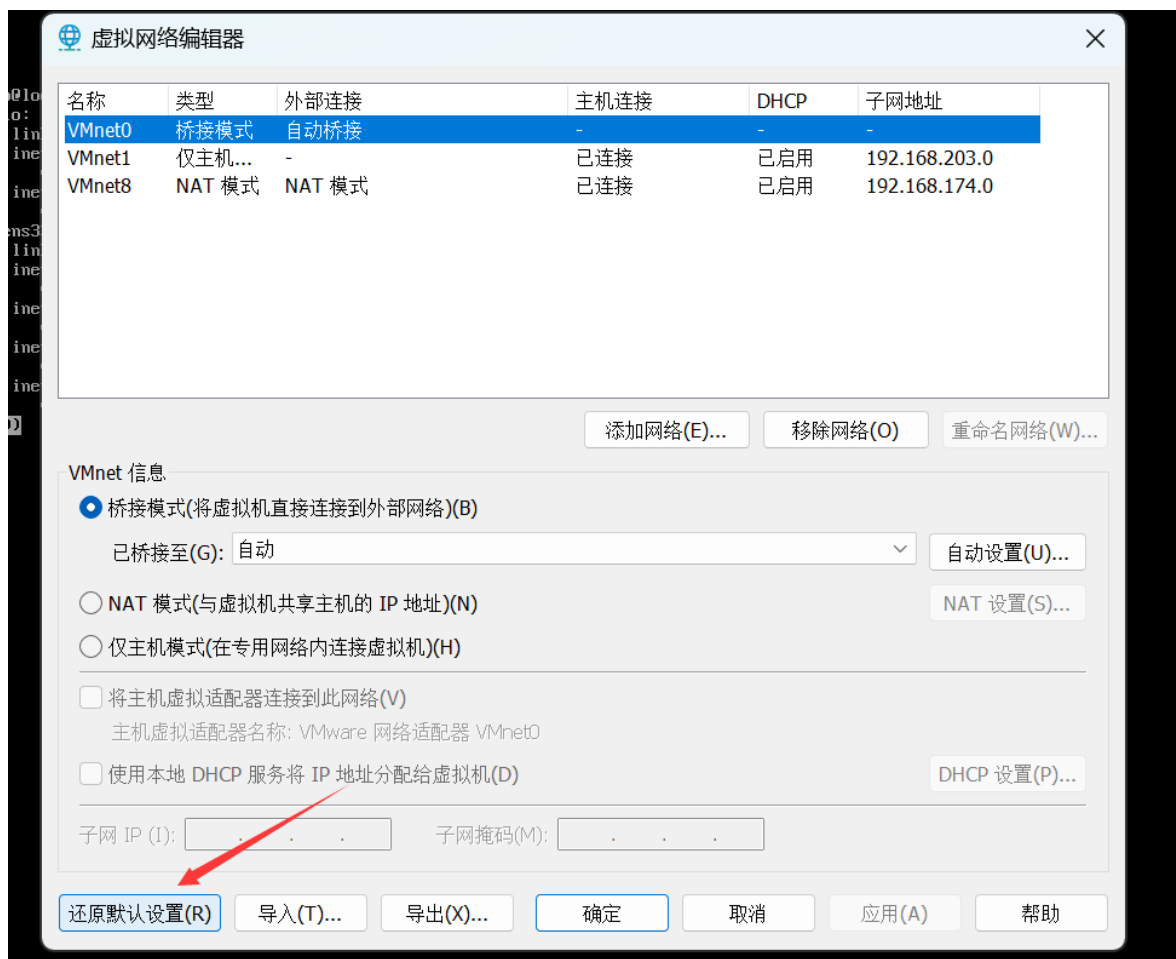
导出(X)...

确定

取消

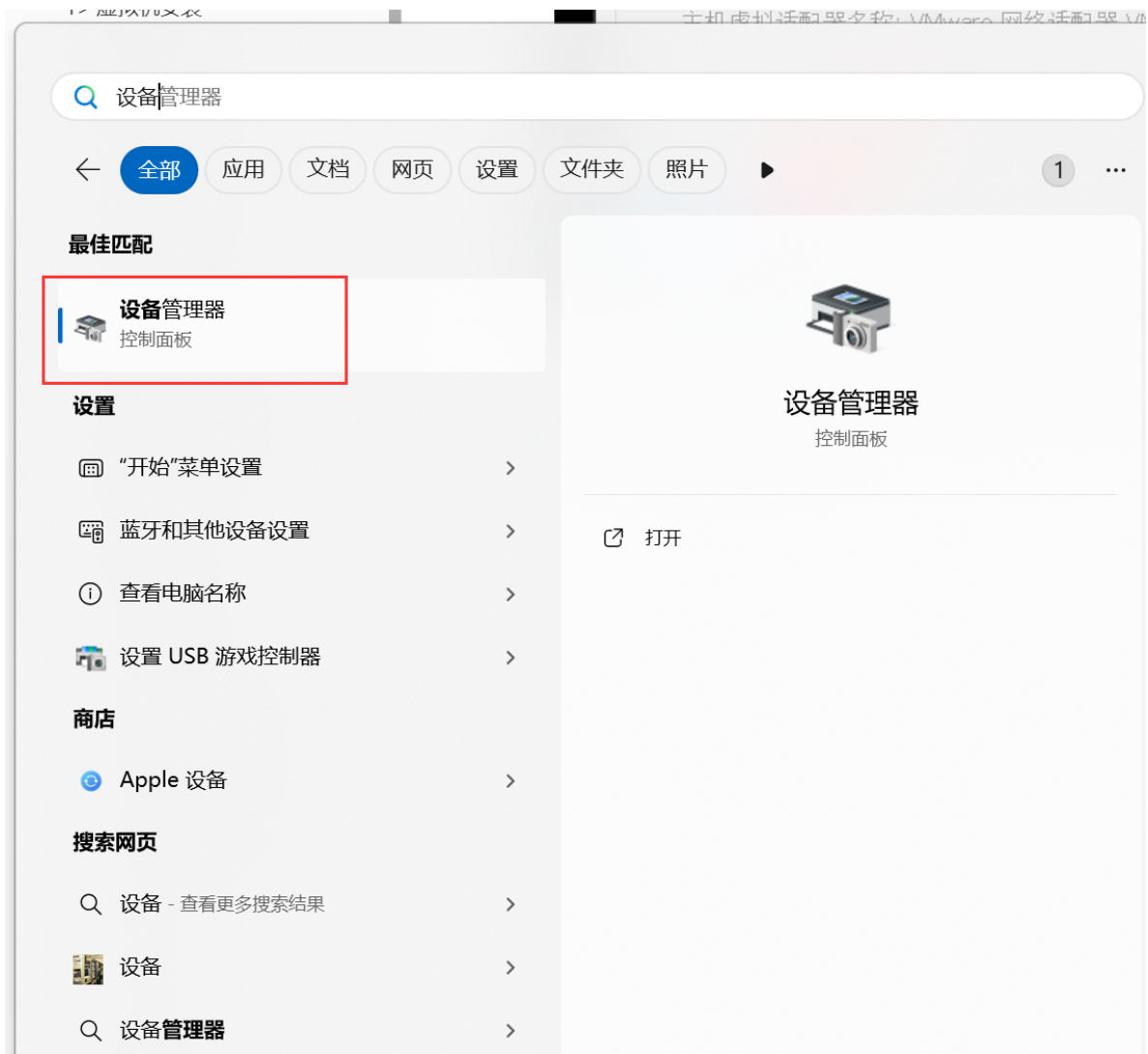
应用(A)

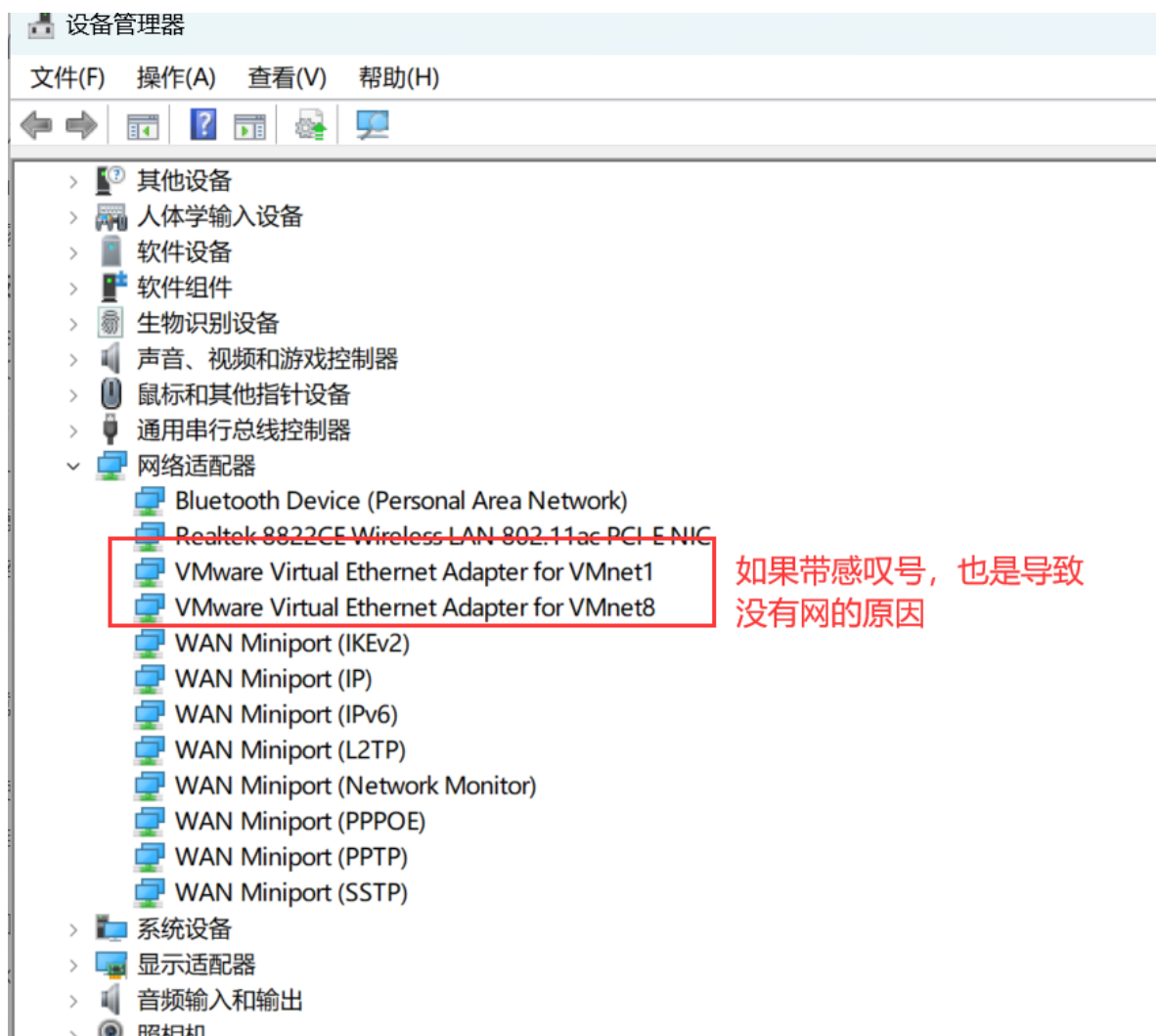
帮助



6、查看windows是否为家庭版，如果为家庭版，建议升级成专业版

查看设备管理器中的网络连接



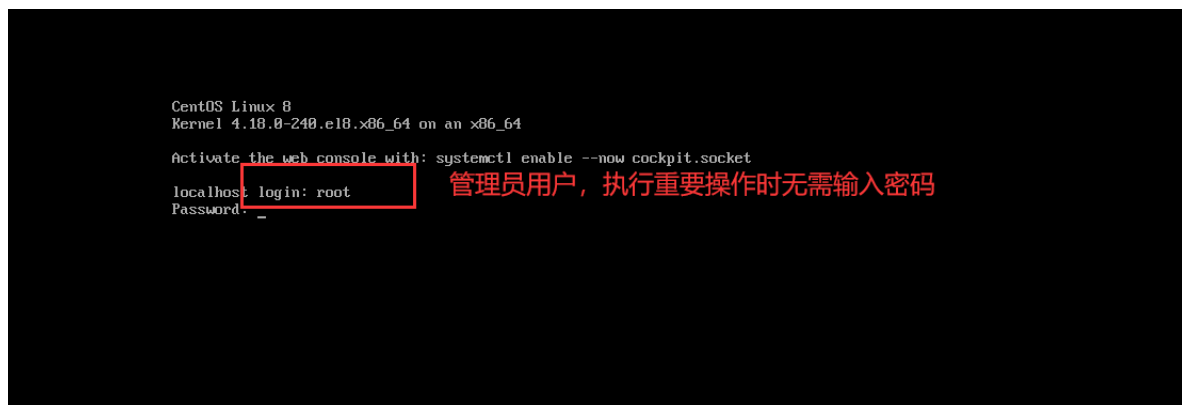


总结一下：

- 1、虚拟机无法获取IP地址：网络管理冲突（一般发生在CentOS7中）、网络模式（NAT）、VM服务不正常
- 2、主机和虚拟机网络不互通：网络适配器不正常
- 3、虚拟机无法连接外网：配置了静态IP不正确，检查三个IP段是否一致

7> 有关用户介绍

Linux是支持多用户开发的，有一个管理员用户课多个普通用户



普通用户：正常用于开发合作的用户


```
CentOS Linux 8  
Kernel 4.18.0-240.el8.x86_64 on an x86_64  
  
Activate the web console with: systemctl enable --now cockpit.socket  
  
localhost login: zpp 普通用户  
Password:
```

普通用户进入管理员用户

`su` 管理员用户名

```
表示普通用户  
[zpp@localhost ~]$ su root 从普通用户进入管理员用户  
Password:  
[root@localhost zpp]#  
表示管理员用户
```

退出超级用户:

```
[zpp@localhost ~]$ su root  
Password:  
[root@localhost zpp]# exit 退出管理员用户  
exit  
[zpp@localhost ~]$
```

8> linux下常用指令介绍

```
tzpp@localhost ~$ cd /
tzpp@localhost /$ ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
tzpp@localhost /$ ls -l
total 20
drwxrwxrwx. 1 root root 7 Nov 3 2020 bin -> usr/bin
dr-xr-xr-x. 5 root root 4096 Nov 7 22:36 boot
drwxr-xr-x. 20 root root 3120 Nov 21 23:17 dev
drwxr-xr-x. 85 root root 8192 Nov 21 23:17 etc
drwxr-xr-x. 3 root root 17 Nov 7 22:34 home
drwxrwxrwx. 1 root root 7 Nov 3 2020 lib -> usr/lib
drwxrwxrwx. 1 root root 9 Nov 3 2020 lib64 -> usr/lib64
drwxr-xr-x. 2 root root 6 Nov 3 2020 media
drwxr-xr-x. 2 root root 6 Nov 3 2020 mnt
drwxr-xr-x. 2 root root 6 Nov 3 2020 opt
dr-xr-xr-x. 143 root root 0 Nov 21 23:17 proc
dr-xr-xr-x. 2 root root 135 Nov 7 22:55 root
drwxr-xr-x. 27 root root 768 Nov 21 23:17 run
drwxrwxrwx. 1 root root 8 Nov 3 2020 sbin -> usr/sbin
drwxr-xr-x. 2 root root 6 Nov 3 2020 srv
dr-xr-xr-x. 13 root root 0 Nov 21 23:17 sys
drwxrwxrwt. 9 root root 227 Nov 21 23:21 tmp
drwxr-xr-x. 12 root root 144 Nov 7 22:30 usr
drwxr-xr-x. 21 root root 4096 Nov 7 22:36 var
tzpp@localhost /$ _
```

进入根目录
列举出目录中的所有文件

类型 权限 大小 创建时间 文件名

3> Xshell安装与远程连接CentOS

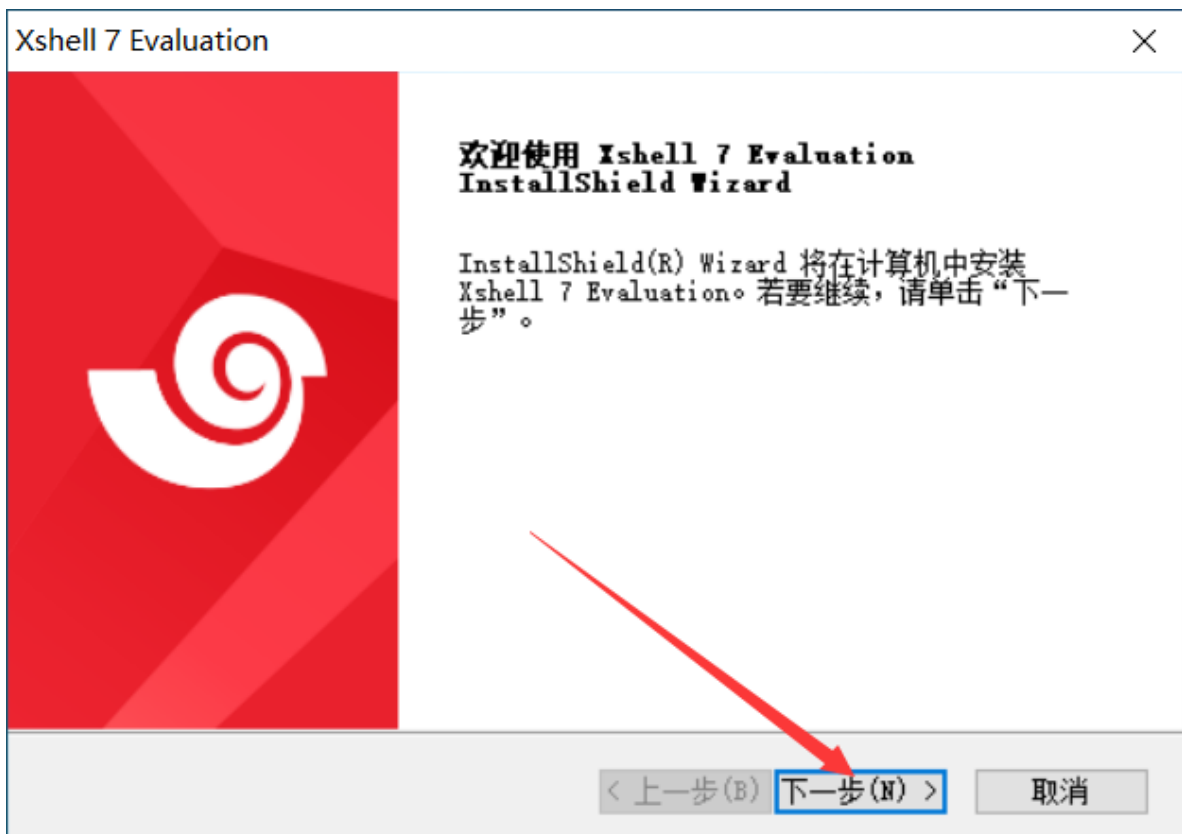
1、Xshell是什么

Xshell是一款Windows平台上的SSH(secure Shell)客户端软件，能够完成远程连接和管理linux服务，支持SSH1和SSH2协议，提供了安全的加密连接，能够防止数据被窃取和篡改。

2、Xshell下载地址：<https://www.xshell.com/zh/free-for-home-school/>

链接: https://pan.baidu.com/s/1zUUQbv0m2yJtr_a445bWVA?pwd=gbau 提取码: gbau

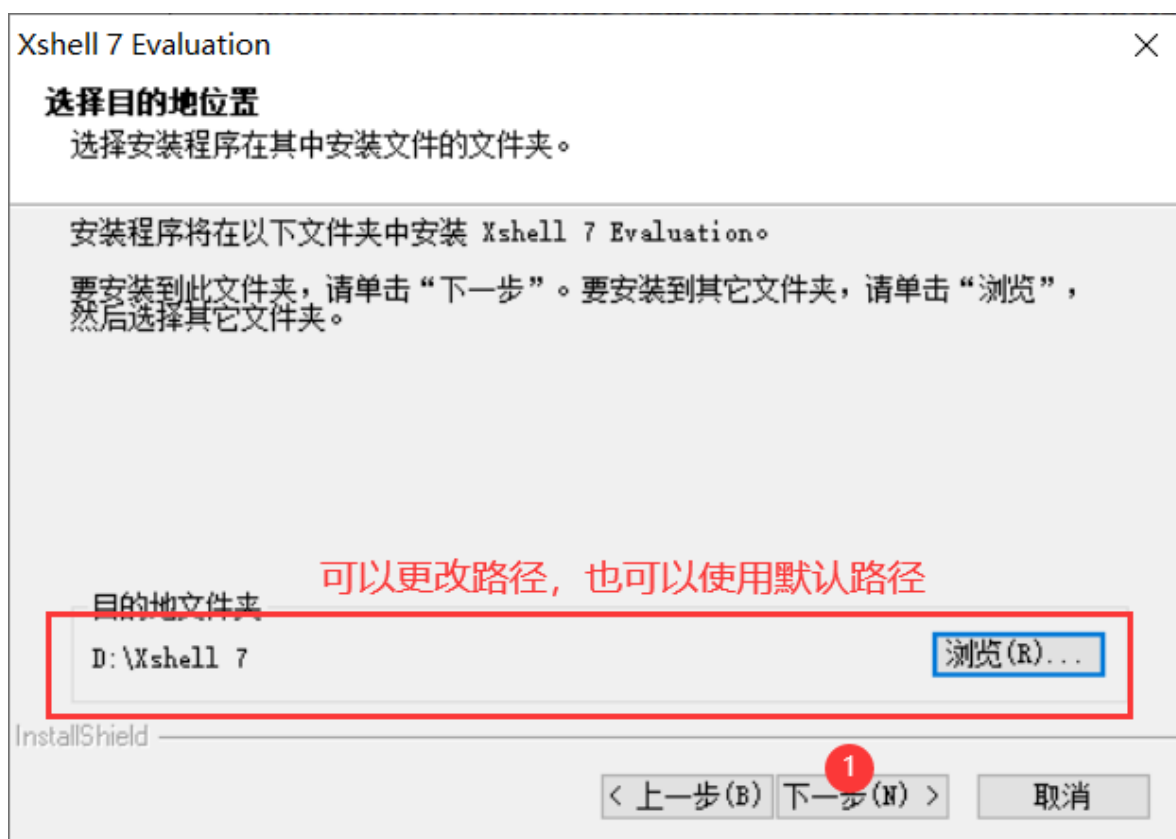
3、双击安装包

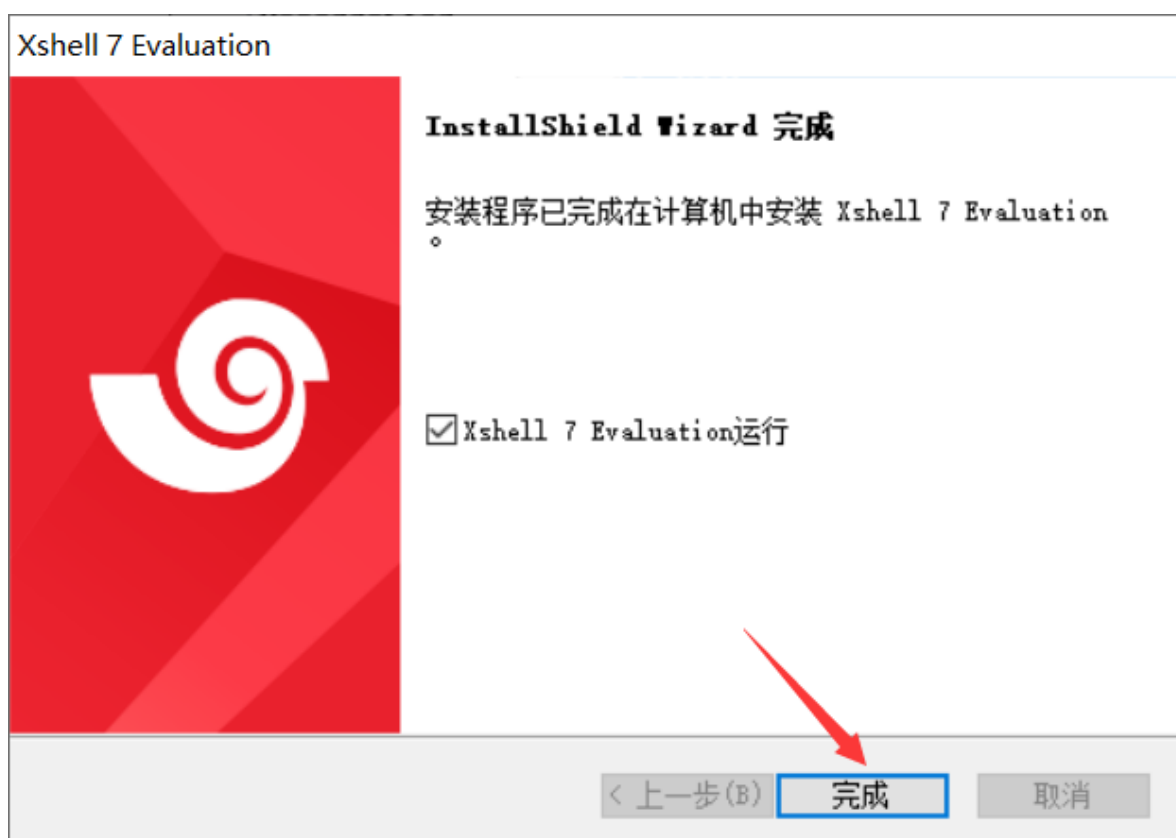
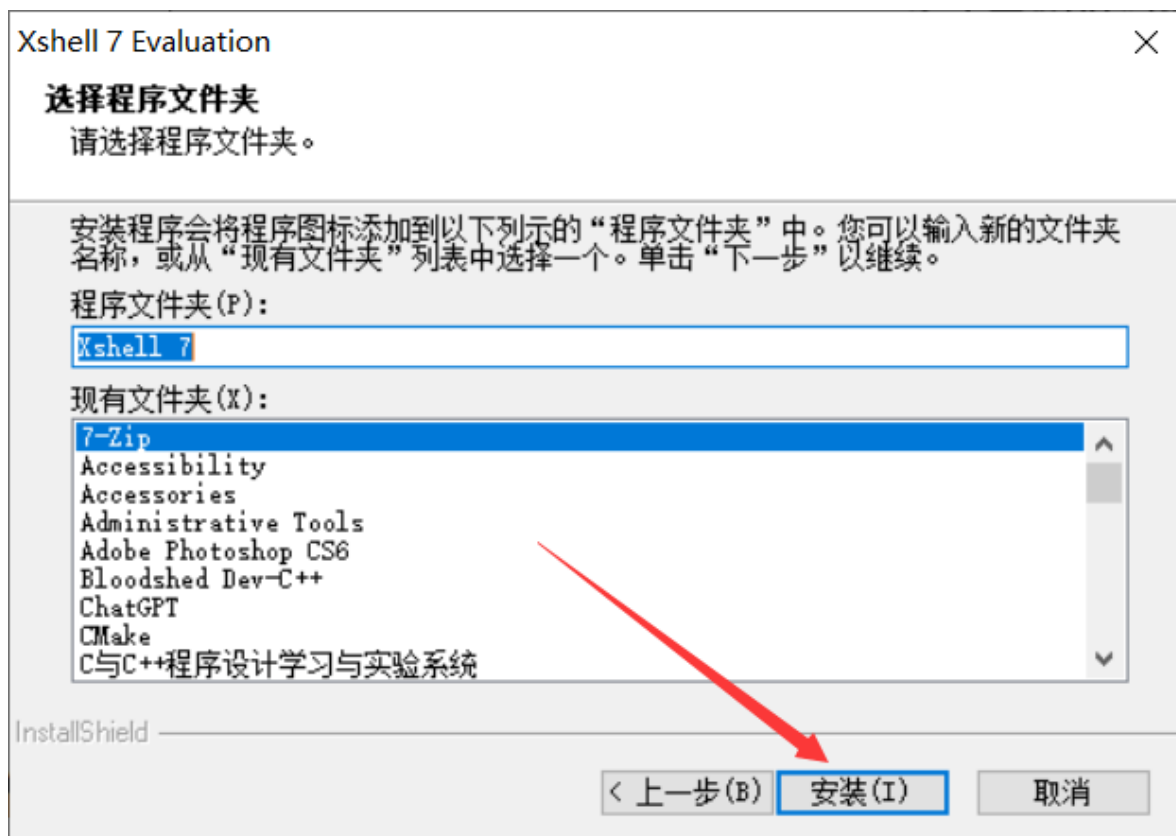


4、

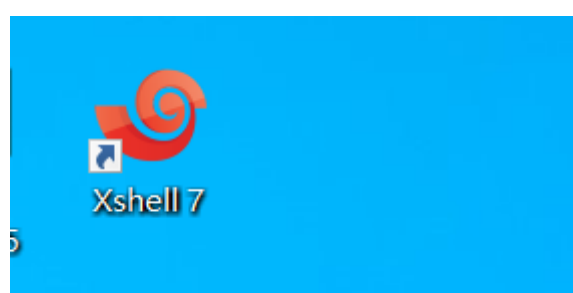


5、

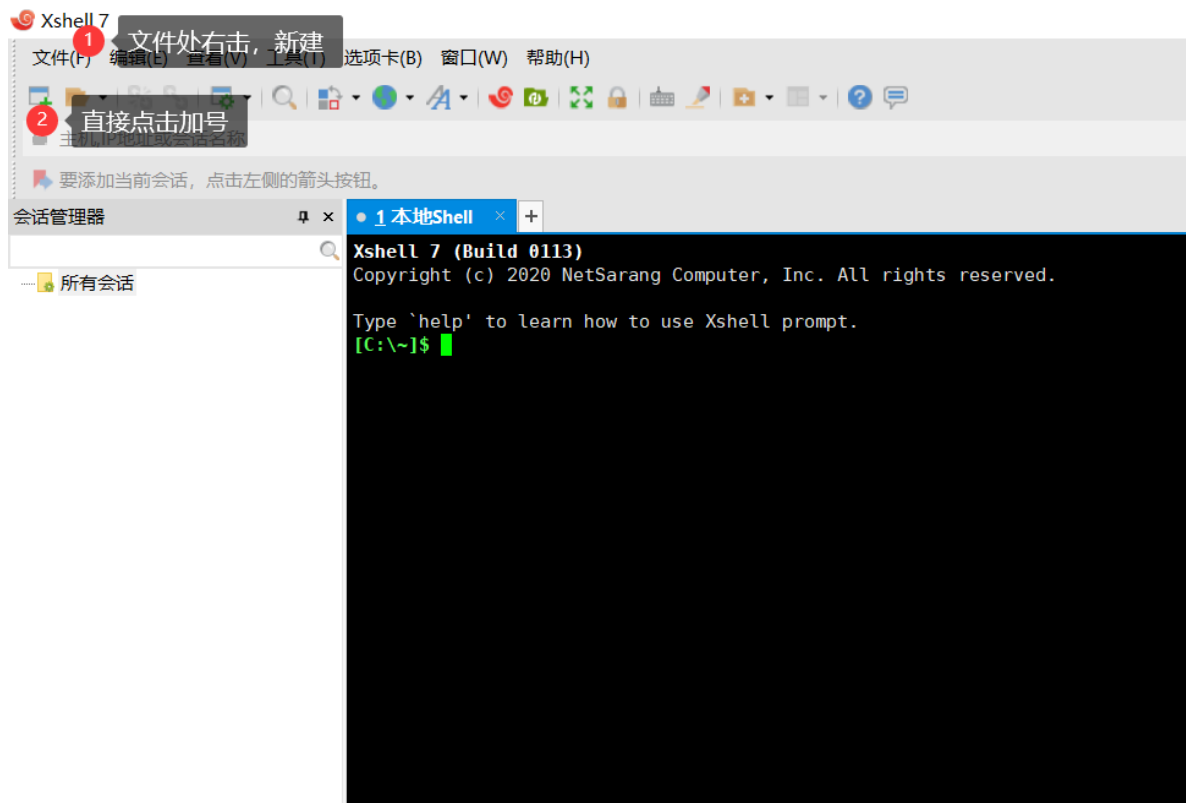




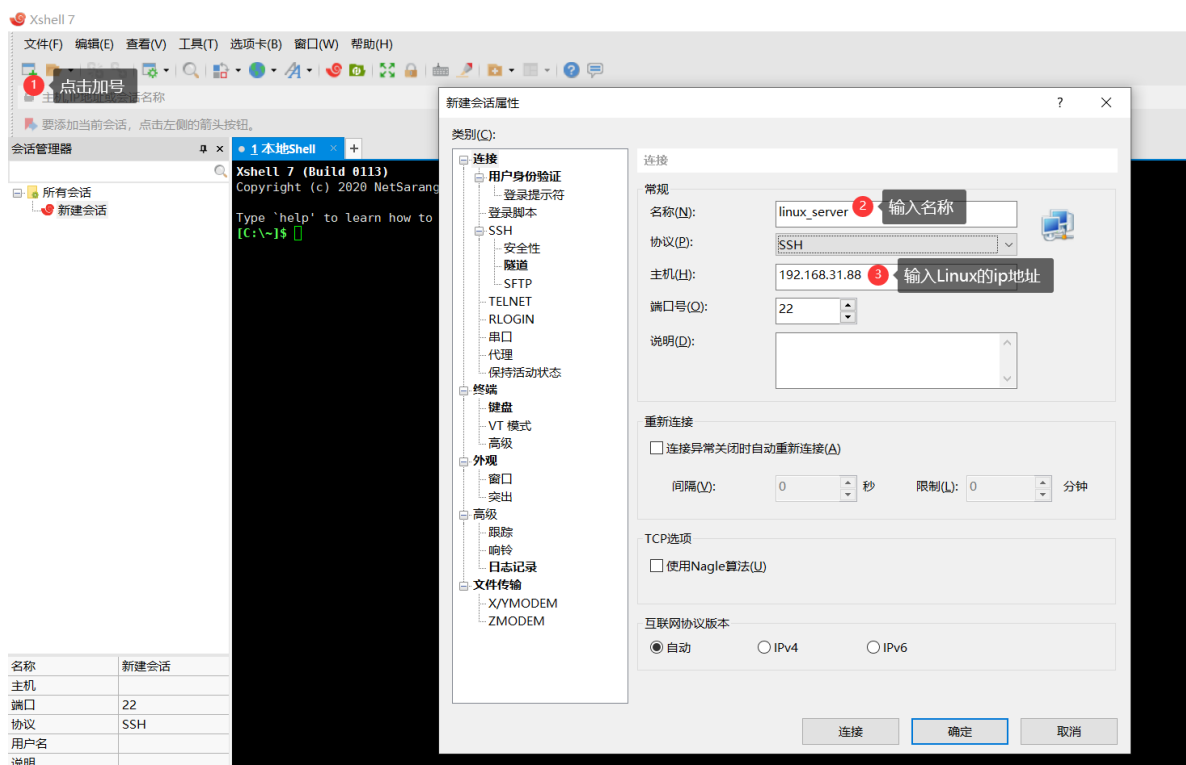
6、安装完成，图标如下



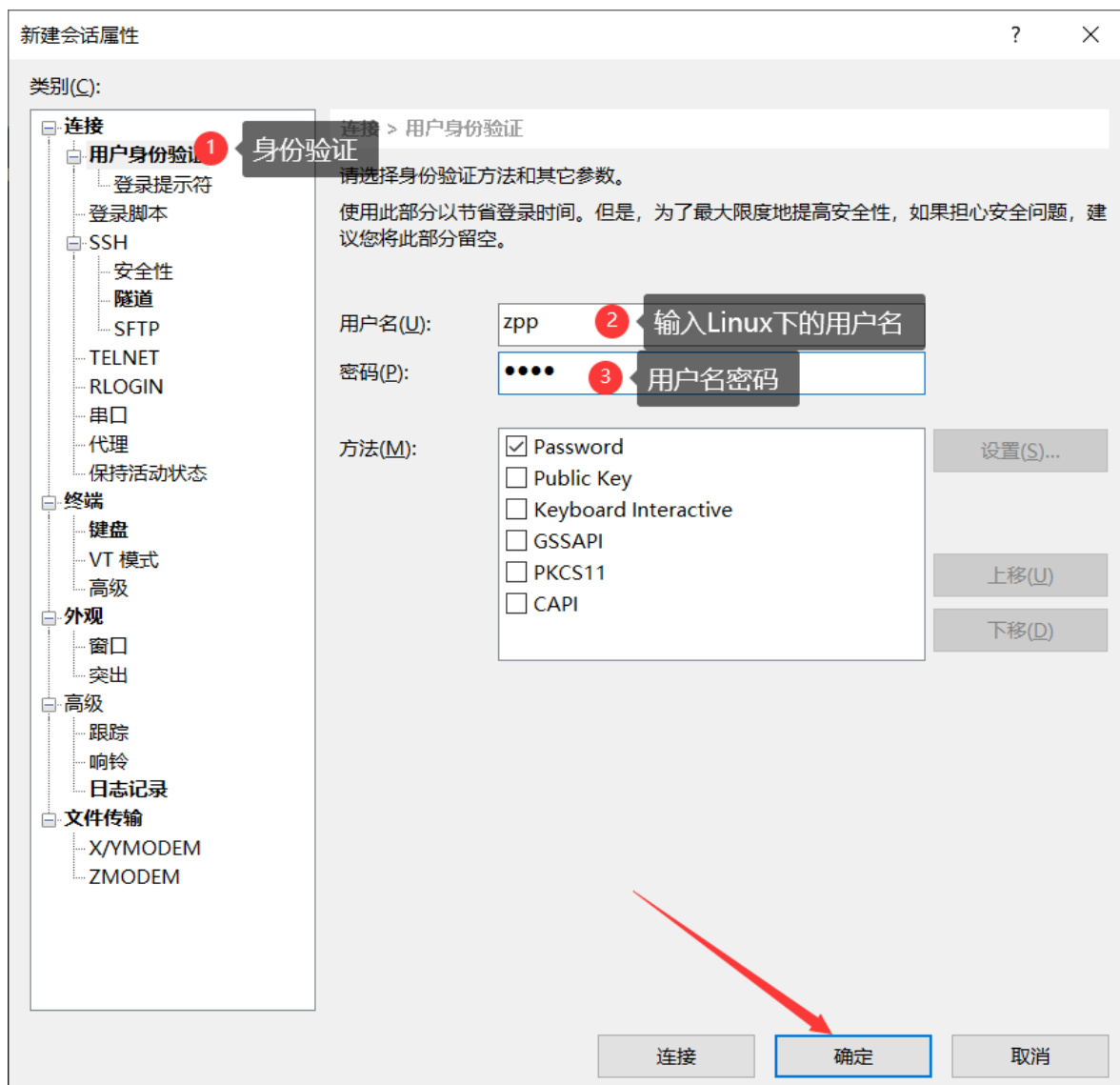
7、打开客户端



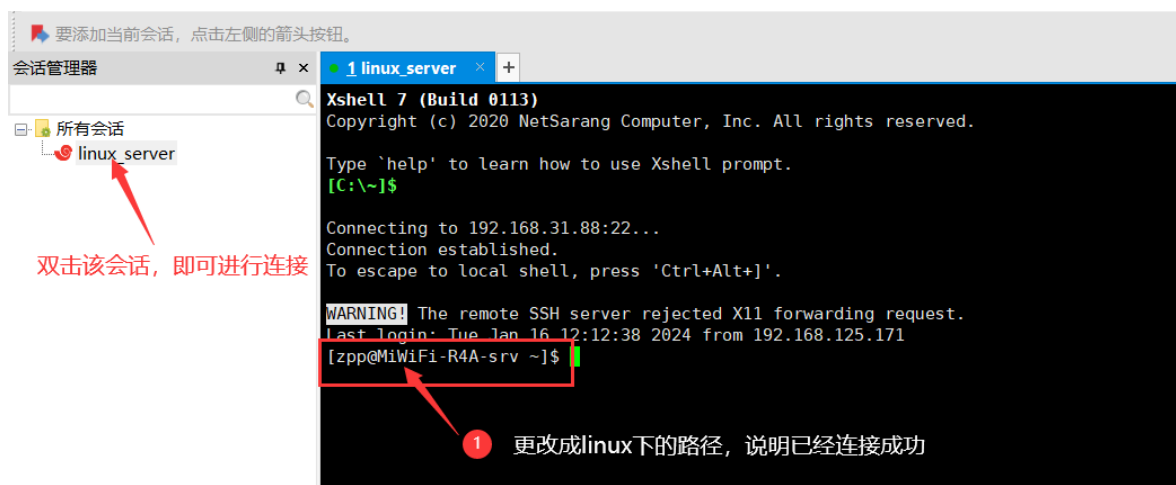
8、创建一个连接到linux服务器



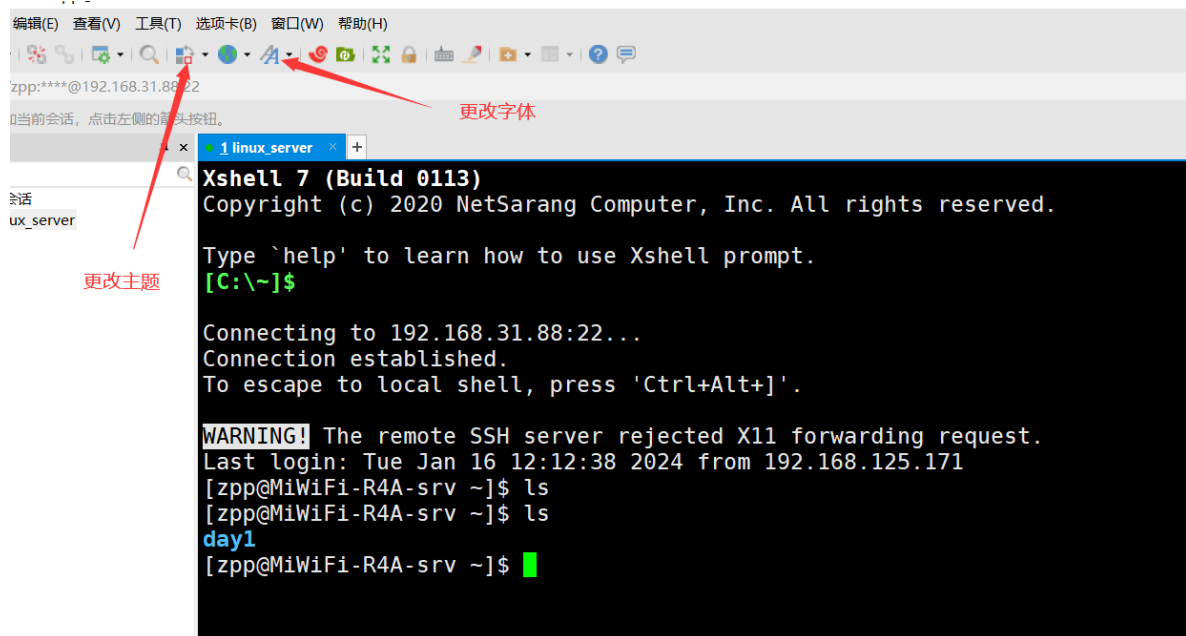
9、用户身份验证



10、远程连接Centos系统



11、可视化配置



4> Vim使用点回顾

1、vim是什么

vim是linux系统下的文本编辑器，可以对不同文本进行编辑，功能是对文件内容的编辑、保存、更改等操作

2、vim有三种模式：命令行模式、插入模式、底行模式

3、如何打开vim编辑器

- 1) 先touch一个文件，然后使用指令：**vi/vim + 文件名**即可，这种方式打开文件，无论是否编辑文件，都会在文件系统中产生一个普通文件
- 2) 直接使用**vi/vim + 不存在的文件名**，如果对文本内容进行更改，则保存退出后，会在文件系统上产生一个普通文件，如果没有更改数据，保存后退出，则不会在文件系统上创建该文件

4、命令行模式

- 1) 使用**vi + 文件名**，刚进入编辑器的模式就是命令行模式
- 2) 命令行模式的作用：主要完成对文本内容整体的复制、粘贴、剪切、删除、光标移动等操作
- 3) 如何进入命令行模式：
刚进入编辑器的模式就是命令行模式
其他模式进入命令行模式只需要按：**ESC**键即可
- 4) 命令行模式主要操作
 - yy**: 复制当前行的文本内容
 - nyy**: 复制从广播所在行及以后的n行文本内容
 - p**: 将剪切板中的内容进行粘贴
 - dd**: 删除光标所在的一行文本内容（剪切）
 - ndd**: 删除光标所在行及以后的n行数据（剪切）
 - u**: 撤销上一步操作
 - ctrl+r**: 反撤销
 - gg**: 将光标直接跳到首行位置
 - nG**: 表示跳到地n行
 - O**: 表示光标移动到当前行的行首位置
 - \$**: 表示光标移动到当前行的行尾位置

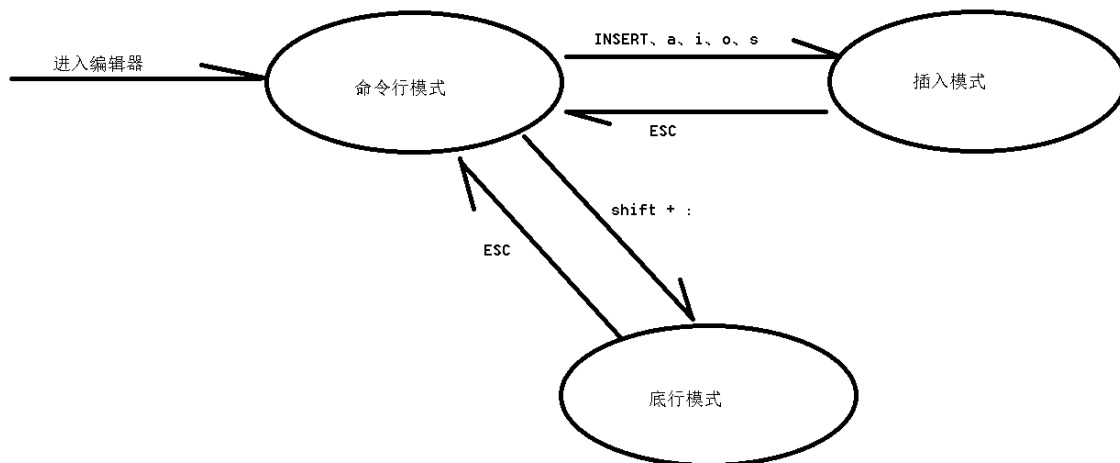
5、插入模式

- 1) 功能：用于文件内容的编辑
- 2) 如何进入插入模式，只能从命令行模式进入插入模式，不能直接从底行模式进入插入模式
键盘上特殊的键能从命令行模式进入插入模式
INSERT键：从光标所在的字符前进入插入模式
i键：从光标所在的字符前进入插入模式
I键：从光标所在的行行首进入插入模式
a键：从光标所在的字符后面进入插入模式
A键：从光标所在行的行尾进入插入模式
o键：从光标所在行的下一行进入插入模式
O键：从光标所在行的上一行进入插入模式
s键：删除光标所在的字符后，从光标位置进入插入模式
S键：删除光标所在的一行文本后，从当前行进入插入模式

6、底行模式

- 1) 功能：完成对文本内容的保存、退出操作或者替换、查找工作
- 2) 如何进入底行模式：从命令行模式中键入 **shift + 冒号**，就是输入一个冒号
- 3) 主要操作
 - :w** 保存文本内容
 - :q** 退出编辑器
 - :q!** 强制退出
 - :wq** 保存后退出
 - :x** 保存退出
 - :set number** 在编辑器左侧显示行号
 - :set nonumber** 不显示行号
 - :/string** 查找字符串string，并将光标定位在包含该字符串的行首
 - :%s/string1/string2/g** 表示将所有的string1更换成string2
 - :m,ns/string1/string2/g** 表示[m,n]行闭区间内的所有string1更换成string2

7、总结



5> C++环境的搭建

- 1、安装g++编译环境、gcc编译环境


```

sudo yum install gcc           //安装gcc编译器，用于编译c语言程序
sudo yum install gcc-c++       //安装g++编译器，用于编译C++程序

验证是否安装成功
gcc --version                  //如果显示版本号，则表示安装成功
g++ --version                  //如果显示版本号，则表示安装成功

sudo yum remove gcc           //卸载gcc编译器，用于编译c语言程序
sudo yum remove gcc-c++       //卸载g++编译器，用于编译C++程序

```

2、安装gdb调试工具

```

sudo yum install gdb           //安装gdb调试工具
gdb --version                  //验证是否安装成功

```

3、CMake工具

```

sudo yum install cmake         //安装CMake脚本工具
cmake --version                //查看是否安装成功

```

6> 第一个Linux下C++程序

1> C++程序框架的介绍

```

#include<iostream>
//第一条指令表示预处理指令
//#是预处理指令的标识
//include: 表示文件包含指令，将指定的文件在该处进行展开
//iostream: i(input)、o(output)、stream(流)，本质上也是一个类，在C++中，一般头文件名就是类名
#include<string>           //例如string类所在的头文件就是string

using namespace std;
//使用标准的命名空间，头文件中定义的，系统指定的同一的名称
//using: 使用命名空间的关键字。有三种用途：命名空间使用、类似于c语言的typedef用于类型重定义、子类中重新定义父类继承的成员的类型
//namespace: 命名空间的关键字
//std: 系统提供的标准命名空间，系统提供的C++的头文件中，定义的类型名、函数名、宏名、类型名，都会同一定义到该命名空间中，每个头文件中几乎都会定义一个名为std的命名空间，所以，同一作用域下，可以定义多个同名的命名空间，但是要求命名空间中的名字不能重复

int main(int argc, const char *argv[])
//主函数：程序的入口函数，每个程序有且只能有一个名为main的函数
//这个函数，可以使用有参的，也可以是无参的，是有返回值的
//参数argc: 外部传参的个数
//参数argv: 是一个字符指针数组，每一个元素都是一个字符串，表示外部传参的值
{
//花括号表示函数的作用域

    cout << "hello world" << endl; //输出语句，cout是ostream的一个类对象

    return 0;
}

```

2> 如何将程序编译生成可执行程序

方法一：

一步到位的编译

- 1、`g++ ***.cpp` //此时会默认生成一个 `a.out` 的可执行程序
- 2、`g++ ***.cpp -o 可执行程序名称` //此时会生成可执行程序

方法二：

分步编译：（重要） `ESC` `--->` `iso`

1、预处理（Pre_processing）

功能：将源程序头文件展开、删除注释、宏替换

语法格式：`g++ -E ***.cpp -o ***.i`

2、编译(Compiling)

功能：将程序编译生成汇编语言

语法格式：`g++ -S ***.i -o ***.s`

3、汇编(ASsembling)

功能：将汇编语言编译生成二进制文件

语法格式：`g++ -c ***.s -o ***.o`

4、链接（Linking）

功能：链接相关库文件，生成可执行程序

语法格式：`g++ ***.o -o 可执行程序`

```
[zpp@MiWi-Fi-R4A-srv day1]$ vi 02demo.cpp
[zpp@MiWi-Fi-R4A-srv day1]$ g++ -E 02demo.cpp -o test.i
[zpp@MiWi-Fi-R4A-srv day1]$ ls
01demo.cpp 02demo.cpp a.out demo test.i
[zpp@MiWi-Fi-R4A-srv day1]$ vi test.i
[zpp@MiWi-Fi-R4A-srv day1]$ g++ -S test.i -o test.s
[zpp@MiWi-Fi-R4A-srv day1]$ ls
01demo.cpp 02demo.cpp a.out demo test.i test.s
[zpp@MiWi-Fi-R4A-srv day1]$ vi test.s
[zpp@MiWi-Fi-R4A-srv day1]$ g++ -c test.s -o test.o
[zpp@MiWi-Fi-R4A-srv day1]$ ls
01demo.cpp 02demo.cpp a.out demo test.i test.o test.s
[zpp@MiWi-Fi-R4A-srv day1]$ vi test.o
[zpp@MiWi-Fi-R4A-srv day1]$ g++ test.o -o test
[zpp@MiWi-Fi-R4A-srv day1]$ ls
01demo.cpp 02demo.cpp a.out demo test test.i test.o test.s
[zpp@MiWi-Fi-R4A-srv day1]$ ./test
hello world
[zpp@MiWi-Fi-R4A-srv day1]$
```

3> 如果执行可执行程序

语法格式：`./可执行程序名称`

1.2 sys库的使用

1> man手册的解析以及使用方法

1、什么是man手册

linux系统提供的有关函数或指令介绍的相关帮助手册，可以在该手册中也查看函数、指令的功能，说白了就是相关操作的使用说明书，一共有七章内容，主要使用前三章，第一章是shell指令相关说明，第二章是系统调用函数相关说明（重点），第三章是库函数（重要）

2、man手册使用

只需要键入 man + 相关函数/指令即可

退出手册：键入 q

3、查询指令

```
LS(1) 指明要查询的内容在第几章 User Commands LS(1)
NAME
ls - list directory contents 一句话介绍查询内容的功能
SYNOPSIS
ls [OPTION]... [FILE]... 使用格式
DESCRIPTION 详细描述
List information about the FILES (the current directory by default). Sort entries alphabetically if none
of -cftuvSUX nor --sort is specified.
Mandatory arguments to long options are mandatory for short options too.
-a, --all
do not ignore entries starting with .
-A, --almost-all
do not list implied . and ..
--author
with -l, print the author of each file
-b, --escape
print C-style escapes for nongraphic characters
每一个选项的功能
```

4、如果想要查看C语言库函数相关功能，需要安装相关库

sudo yum install man-pages man-pages-devel

5、c/c++库函数的使用

```
PRINTF(3) 函数所在章节 Linux Programmer's Manual PRINTF(3)
NAME
printf, fprintf, sprintf, snprintf, vprintf, vfprintf, vsprintf, vsnprintf - formatted output conversion
SYNOPSIS 相关函数名以及相关功能
#include <stdio.h> 函数所在头文件
int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);
int snprintf(char *str, size_t size, const char *format, ...);
#include <stdarg.h>
int vprintf(const char *format, va_list ap);
int vfprintf(FILE *stream, const char *format, va_list ap);
int vsprintf(char *str, const char *format, va_list ap);
int vsnprintf(char *str, size_t size, const char *format, va_list ap);
Feature Test Macro Requirements for glibc (see feature_test_macros(7)):
snprintf(), vsnprintf():
_BSD_SOURCE || _XOPEN_SOURCE >= 500 || _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L;
or cc -std=c99
DESCRIPTION 详细, 描述
The functions in the printf() family produce output according to a format as described below. The func-
```

2> 常用的内核提供的函数库 sys

1、文件的操作：open () 、read () 、write()、close()、lseek()等等

2、进程控制函数：fork () 、exit()、wait () 、execl () 等等

3、信号操作：kill () 、signal () 等等

4、网络通信：socket()、bind()、listen()等等

```

#include<iostream>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<string.h>
#include<stdio.h>
#include<error.h>
#include<unistd.h>
using namespace std;
int main()
{

    //1、定义文件描述符，并打开文件
    int fd = -1;
    if((fd = open("./test.txt", O_WRONLY|O_CREAT|O_TRUNC, 0664)) == -1)
    {
        perror("open error");
        return -1;
    }

    //2、读写文件
    write(fd, "hello", strlen("hello"));

    //3、关闭文件
    close(fd);

    return 0;
}

```

3> 使用GDB调试程序

1、在linux系统下的警告和错误，当程序出现bug时，linux终端会给大家两种不同的信息

警告 (warning)：有时的警告是不影响可执行程序的产生

```

[zpp@localhost day2]$ g++ 01test.cpp
01test.cpp: 在函数‘int main(int, const char*)’中:
01test.cpp:7:24: 错误: ‘printf’在此作用域中尚未声明
    printf("hello world\n");    //该函数是C语言的相关函数，表示向终端输出数据
                           ^
[zpp@localhost day2]$ ls    此时说明错误是影响可执行程序的产生
01test.cpp  02test.c
[zpp@localhost day2]$ gcc 02test.c
02test.c: 在函数‘main’中: 警告所在的函数
02test.c:5:2: 警告: 隐式声明与内建函数‘printf’不兼容 [默认启用] 警告信息
    printf("hello world\n");    //该函数是C语言的相关函数，表示向终端输出数据
    ^                          产生警告的代码行
[zpp@localhost day2]$ ls
01test.cpp 02test.c a.out    此时说明警告是不影响可执行程序的产生
[zpp@localhost day2]$ ./a.out
hello world
[zpp@localhost day2]$

```

错误 (error) :错误如果不改正，是不能生产可执行程序的

```

[zpp@localhost day2]$ g++ 01test.cpp
01test.cpp: 在函数‘int main(int, const char*)’中: 错误所在函数
01test.cpp:7:24: 错误: ‘printf’在此作用域中尚未声明 错误的原因
    printf("hello world\n");    //该函数是C语言的相关函数，表示向终端输出数据
    ^                          错误代码行

```

总结：相比于gcc编译器，g++编译器要求更加严格

警告可以被忽略，继续产生可执行程序，但是错误必须更改后才能产生可执行程序

2、什么是gdb: <https://www.sourceware.org/gdb/>

GDB, GNU项目调试器, 允许您查看一个程序执行时“内部”发生了什么, 或者一个程序崩溃时正在做什么。

3、gdb是干啥的

gdb只要做四件事:

启动程序, 指定可能影响其行为的任何内容。

使程序在指定条件下停止。

检查程序停止时发生了什么。

更改程序中的内容, 这样您就可以尝试纠正一个错误的影响, 并继续了解另一个错误。

gdb可以调试指定的当前程序, 向程序中传递参数

gdb还可以调试出错的文件, 查看错误原因

gdb还可以调试正在运行的进程

4、gdb如何使用

1)准备C++程序

```
#include<iostream>
#include<stdio.h>           //属于C语言的头文件, 包含了输入输出函数
using namespace std;

//自定义函数
void print()
{
    cout<<"hello world"<<endl;

    cout<<"这是我想要说给大家听的!!!"<<endl;
}

int main(int argc, const char *argv[])
{

    int arr[5] = {1,2,3,4,5};    //定义一个整形数组
    int i = 0;                  //循环变量

    for(i=0; i<5; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    //调用自定义函数
    print();

    return 0;
```

```
}
```

2) 编译程序，编译选项中需要加上 -g

```
g++ -g ***.cpp -o ***
```

3)启动gdb调试

```
gdb ./***
```

```
[zpp@localhost day2]$ g++ -g 03gdb.cpp 编译可执行程序
[zpp@localhost day2]$ ls
01test.cpp 02test.c 03gdb.cpp a.out
[zpp@localhost day2]$ gdb a.out 进入gdb调试
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/zpp/day2/a.out...done.
(gdb) 
```

表示正在进行gdb调试

4) gdb常用指令

quit (q) : 表示退出gdb调试

run (r):表示执行可执行程序，如果没有设置断点，则将整个程序从头到尾执行一遍

list(l): 展示可执行程序的相关行信息，默认展示10行

list m,n :表示展示从m行到n行的信息

list func:表示展示func函数旁边的相关程序

```
(gdb) list 直接使用，默认向后展示10行信息
1      #include<iostream>
2      #include<stdio.h>                //属于C语言的头文件，包含了输入输出函数
3      using namespace std;
4      int main(int argc, const char *argv[])
5      {
6
7          int arr[5] = {1,2,3,4,5};    //定义一个整形数组
8          int i = 0;                  //循环变量
9
10         for(i=0; i<5; i++)
(gdb) l
11         {
12             cout << arr[i] << " ";
13         }
14         cout << endl;
15
16
17         return 0;
18     }
(gdb) l
```

```

(gdb) l 8,17 表示展示指定区间内的程序代码
8      int i = 0;                                //循环变量
9
10     for(i=0; i<5; i++)
11     {
12         cout << arr[i] << " ";
13     }
14     cout << endl;
15
16
17     return 0;
(gdb) l main 表示展示给定函数旁边的相关代码
1  #include<iostream>
2  #include<stdio.h>                                //属于C语言的头文件，包含了输入输出函数
3  using namespace std;
4  int main(int argc, const char *argv[])
5  {
6
7      int arr[5] = {1,2,3,4,5};                    //定义一个整形数组
8      int i = 0;                                    //循环变量
9
10     for(i=0; i<5; i++)
(gdb) █

```

break(b): 表示设置断点，当调试器将程序运行到断点所在位置后，会暂停于此

break 行号: 表示在某行设置断点

break func:表示在指定的函数处设置断点

info break:查看所有断点的信息

delete breakpoint 断点编号: 表示删除指定的断点

```

(gdb) b 21 在指定行处设置断点
Breakpoint 1 at 0x400894: file 03gdb.cpp, line 21.
(gdb) b print 在指定函数处设置断点
Breakpoint 2 at 0x400821: file 03gdb.cpp, line 8.
(gdb) info break 查看所有断点信息
Num Type Disp Enb Address What
1 breakpoint keep y 0x0000000000400894 in main(int, char const**) at 03gdb.cpp:21
2 breakpoint keep y 0x0000000000400821 in print() at 03gdb.cpp:8
(gdb) delete breakpoint 1 删除指定的断点
(gdb) info break
Num Type Disp Enb Address What
2 breakpoint keep y 0x0000000000400821 in print() at 03gdb.cpp:8
(gdb) █

```

next(n):表示执行下一条语句

continue (c) :表示从断点处继续向后执行，直到遇到下一个断点或者程序结束

```

25     print();
(gdb) b 18 在程序指定行处设置断点
Breakpoint 1 at 0x400878: file 03gdb.cpp, line 18.
(gdb) b print 在程序的指定函数处设置断点
Breakpoint 2 at 0x400821: file 03gdb.cpp, line 8.
(gdb) r 运行程序
Starting program: /home/zpp/day2/a.out

Breakpoint 1, main (argc=1, argv=0x7fffffff598) at 03gdb.cpp:18
18     for(i=0; i<5; i++)
Missing separate debuginfos, use: debuginfo-install glibc-2.17-326.el7_9.x86_64 libgcc-4.8.5-44.el7.x86_64
(gdb) n 执行下一条程序
20     cout << arr[i] << " ";
(gdb) c 继续执行后续程序
Continuing.
1 2 3 4 5

Breakpoint 2, print () at 03gdb.cpp:8
8     cout<<"hello world"<<endl;
(gdb) c 继续执行后续程序
Continuing.
hello world
[Inferior 1 (process 26699) exited normally]
(gdb) █

```

step(s):能够跳入到指定函数中，查看相关函数内部代码

```

Breakpoint 2, print () at 03gdb.cpp:8
8          cout<<"hello world"<<endl;
Missing separate debuginfos, use: debuginfo-install glibc-2.17-326.el7_9.
+-4.8.5-44.el7.x86_64
(gdb) step 进入断点所在的函数内部继续执行
hello world
10          cout<<"这是我想要说给大家听的!!!"<<endl;
(gdb) n
这是我想要说给大家听的!!!
12      }
(gdb) n
main (argc=1, argv=0x7fffffff598) at 03gdb.cpp:31
31          return 0;
(gdb) █

```

print (p) 变量名/地址 :表示打印指定变量或地址信息

```

(gdb) print i 查看指定变量的内容
$1 = 0
(gdb) n
21          for(i=0; i<5; i++)
(gdb) n
23          cout << arr[i] << " ";
(gdb) n
21          for(i=0; i<5; i++)
(gdb) print i
$2 = 0
(gdb) n
23          cout << arr[i] << " ";
(gdb) print i
$3 = 1
(gdb) print &i 查看地址
$4 = (int *) 0x7fffffff598
(gdb) █

```

set variable 变量名=值: 表示给某个变量设置相关的值

```

No symbol variable in current context.
(gdb) set variable i=5 给相关变量设置值
(gdb) print i
$5 = 5
(gdb) n
21          for(i=0; i<5; i++)
(gdb) n
25          cout << endl;
(gdb) █

```

5) gdb使用小技巧

shell : 后面可以跟终端指令, 表示执行终端相关操作

set logging on:设置开启日志功能, 会在当前目录中生成一个gdb.txt文件记录接下来的调试内容

watchpoint : 观察点, 如果设置的观察点的值发生改变, 则会将该值的旧值和新值全部展示出来


```

(gdb) print &i 查看变量地址
$1 = (int *) 0x7fffffffef4ac
(gdb) watch *0x7fffffffef4ac 设置观察点
Hardware watchpoint 2: *0x7fffffffef4ac
(gdb) n
21         for(i=0; i<5; i++)
(gdb) n
23         cout << arr[i] << " ";
(gdb) n
21         for(i=0; i<5; i++)
(gdb) n
Hardware watchpoint 2: *0x7fffffffef4ac

Old value = 0
New value = 1
0x00000000004008c3 in main (argc=1, argv=0x7fffffffef598) at 03gdb.cpp:21
21         for(i=0; i<5; i++)
(gdb) █

```

当观察点的内容发生变化时，会将该观察点的旧值和新值一起展示

6) gdb调试出错的文件

当一个可执行程序出现错误时，会产生一个core文件，用于查看相关错误信息

linux系统默认是不产生core文件，需要进行相关设置后才能产生

通过ulimit -a 查看所有linux的限制内容

```

[zpp@localhost day2]$ ulimit -a 查看限制
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 3795
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 3795
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited

```

0表示不能产生core文件

通过 ulimit -c unlimited来设置core文件的个数

```

[zpp@localhost day2]$ ulimit -c unlimited 更改对核心文件的限制
[zpp@localhost day2]$ ulimit -a
core file size          (blocks, -c) unlimited 更改成功
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 3795
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 3795
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
[zpp@localhost day2]$

```

查看错误原因

```

[zpp@localhost day2]$ ./a.out 执行可执行程序, 如果程序有问题, 会生成core文件
段错误(吐核)
[zpp@localhost day2]$ ls
01test.cpp 02test.c 03gdb.cpp 04test.cpp a.out core.26778 gdb.txt
[zpp@localhost day2]$ gdb core.26778 查看出错信息
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
[New LWP 26778]
Missing separate debuginfo for the main executable file
Try: yum --enablerepo='*debug*' install /usr/lib/debug/.build-id/79/allabb6acc19fff15f9bb3b0a
Core was generated by './a.out'.
Program terminated with signal 11, Segmentation fault. 错误原因
#0 0x000000000400664 in ?? ()
"/home/zpp/day2/core.26778" is a core file.
Please specify an executable to debug.
(gdb)

```

7) gdb调试其他正在运行的进程

./可执行程序 & : 表示将可执行程序后台运行, 不占用当前终端

```

[zpp@localhost day2]$ g++ -g 05test.cpp
[zpp@localhost day2]$ ./a.out & 将程序后台运行
[1] 26825
[zpp@localhost day2]$

```

作业号 进程号

```

[zpp@localhost day2]$ g++ -g 05test.cpp //编译程序
[zpp@localhost day2]$ ./a.out & //后台运行程序
[1] 26837 //显示当前后台运行程序的作业号和
进程号
[zpp@localhost day2]$ gdb -p 26837 //调试指定的运行的进程
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7

```

```

Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Attaching to process 26837
Reading symbols from /home/zpp/day2/a.out...done.
Reading symbols from /lib64/libstdc++.so.6...(no debugging symbols
found)...done.
Loaded symbols for /lib64/libstdc++.so.6
Reading symbols from /lib64/libm.so.6...(no debugging symbols found)...done.
Loaded symbols for /lib64/libm.so.6
Reading symbols from /lib64/libgcc_s.so.1...(no debugging symbols found)...done.
Loaded symbols for /lib64/libgcc_s.so.1
Reading symbols from /lib64/libc.so.6...(no debugging symbols found)...done.
Loaded symbols for /lib64/libc.so.6
Reading symbols from /lib64/ld-linux-x86-64.so.2...(no debugging symbols
found)...done.
Loaded symbols for /lib64/ld-linux-x86-64.so.2
main (argc=1, argv=0x7ffd31897908) at 05test.cpp:17
17  int main(int argc, const char *argv[])           //进程运行在某处
Missing separate debuginfos, use: debuginfo-install glibc-2.17-326.el7_9.x86_64
libgcc-4.8.5-44.el7.x86_64 libstdc++-4.8.5-44.el7.x86_64
(gdb) n
22      test();
(gdb) n
23      test1();
(gdb) step
test1 () at 05test.cpp:12
12      int num = 520;
(gdb) n
13      num++;
(gdb) c
Continuing.

```

4> 库的制作（静态库与动态库）

1、为什么引入库

```

[zpp@localhost static]$ ls
add.cpp  add.h  main.cpp
[zpp@localhost static]$ g++ add.cpp main.cpp -o exec
[zpp@localhost static]$ ls
add.cpp  add.h  exec  main.cpp
[zpp@localhost static]$ ./exec
11
[zpp@localhost static]$

```

准备头文件、源文件和主程序文件

将源程序进行联合编译生成可执行程序

执行可执行程序

在上述案例中，主程序要是有的源程序代码，在add.cpp中，如果项目结束后，到了交付阶段，由于主程序的生成需要其他程序联合编译，那么就要将源程序打包一起发给老板，这样该程序的开发者自身的价值就不大了，该项目的知识产权就很容易被窃取。为了保护我们的知识产权，我们引入了库的概念。

2、什么是库

库在linux中是一个二进制文件，它是由 .cpp文件（不包含main函数）编译而来，其他程序如果想要使用该源文件中的函数时，只需在编译生成可执行程序时，链接上该源文件生成的库文件即可。库中存储的是二进制文件，不容易被窃取知识产权，做到了保护作用。

库在linux系统中分为两类，分别是静态库和动态库

windows:

***.lib: 静态库

***.dll: 动态库

linux:

***.a: 静态库

***.so: 动态库

3、静态库及其制作

概念：将一个***.cpp的文件编译生成一个 lib***.a的二进制文件，当你需要使用该源文件中的函数时，只需要链接该库即可，后期可以直接调用

静态体现在：在使用g++编译程序时，会将你的文件和库最终生成一个可执行程序（把静态库也放入到可执行程序中），每个可执行程序单独拥有一个静态库，体积较大，但是，执行效率较高

3.1 准备程序

add.h

```
#ifndef _ADD_H_
#define _ADD_H_

int add(int m, int n);           //函数声明

#endif
```

add.cpp

```
int add(int m, int n)
{
    return m+n;
}
```

main.cpp

```
#include<iostream>
#include<stdio.h>
#include "add.h"
using namespace std;

int main(int argc, const char *argv[])
{
    cout << add(3,8) << endl;    //调用外部文件中的相关函数

    return 0;
}
```

3.2 编译生成静态库

```
gcc -c *.cpp -o *.o    //只编译不链接生成二进制文件
ar -crs lib*.a *.o    //编译生成静态库
```

如果有多个.o文件共同编译生成静态库 : `ar -crs lib*.a *.o xxx.o ...`

ar:用于生成静态库的指令
c:用于创建静态库
r:将文件插入或者替换静态库中同名文件
s:重置金泰克索引

3.3 使用静态库

```
gcc main.cpp -L 库的路径 -l库名 -I 头文件的名字
```

```
[zpp@localhost static]$ ls
add.cpp add.h main.cpp 准备源文件
[zpp@localhost static]$ g++ -c add.cpp -o add.o 只编译不链接生成二进制文件
[zpp@localhost static]$ ls
add.cpp add.h add.o main.cpp
[zpp@localhost static]$ ar -crs libadd.a add.o 通过二进制文件编译生成静态库
[zpp@localhost static]$ ls
add.cpp add.h add.o libadd.a main.cpp
[zpp@localhost static]$ vi libadd.a
[zpp@localhost static]$ g++ main.cpp -L . -ladd -I . 使用静态库
[zpp@localhost static]$ ls
add.cpp add.h add.o a.out libadd.a main.cpp
[zpp@localhost static]$ ./a.out
11
[zpp@localhost static]$
```

库名

库所在的路径

头文件所在的路径

```

[zpp@localhost static]$ ls
header lib src
[zpp@localhost static]$ cd src
[zpp@localhost src]$ ls
add.cpp main.cpp
[zpp@localhost src]$ g++ add.cpp main.cpp
main.cpp:3:17: 致命错误: add.h: 没有那个文件或目录
#include "add.h"
          ^
编译中断。
[zpp@localhost src]$ g++ main.cpp -L ../lib/ -ladd -I ../header/
[zpp@localhost src]$ ls
add.cpp a.out main.cpp
[zpp@localhost src]$ ./a.out
11
[zpp@localhost src]$

```

多路径编译

库所在的路径 头文件所在下的路径

4、动态库及其制作

概念：将一个***.cpp的文件编译生成一个 lib***.so的二进制文件，当你需要使用该源文件中的函数时，只需要链接该库即可，后期可以直接调用

动态体现在：在使用g++编译程序时，会将你的文件和库中的相关函数的索引表一起生成一个可执行程序，每个可执行程序只拥有函数的索引表，当程序执行到对应函数时，会根据索引表，动态寻找相关库所在位置进行调用，体积较小，执行效率较低，但是可以多个程序共享同一个动态库，所以，动态库也叫共享库

4.1 准备程序

add.h

```

#ifndef _ADD_H_
#define _ADD_H_

int add(int m, int n);    //函数声明

#endif

```

add.cpp

```

int add(int m, int n)
{
    return m+n;
}

```

main.cpp

```
#include<iostream>
#include<stdio.h>
#include "add.h"
using namespace std;

int main(int argc, const char *argv[])
{
    cout << add(3,8) << endl;    //调用外部文件中的相关函数

    return 0;
}
```

4.2 编译生成动态库

```
g++ -fPIC -c ***.cpp -o ***.o    //编译生成二进制文件
g++ -shared ***.o -o lib***.so    //依赖于二进制文件生成一个动态库
```

上述两个指令可以合成一个指令

```
g++ -fPIC -shared ***.cpp -o lib***.so
```

4.3 使用动态库

```
gcc main.cpp -L 库的路径 -l库名 -I 头文件的名字
```

```
[zpp@localhost src]$ ls
add.cpp  main.cpp
[zpp@localhost src]$ g++ -fPIC -shared add.cpp -o libadd.so    编译生成动态库
[zpp@localhost src]$ ls
add.cpp  libadd.so  main.cpp
[zpp@localhost src]$ g++ main.cpp -L . -ladd -I ../header/    使用动态库
[zpp@localhost src]$ ls
add.cpp  a.out  libadd.so  main.cpp
[zpp@localhost src]$ ./a.out
./a.out: error while loading shared libraries: libadd.so: cannot open shared object file: No such file or directory
[zpp@localhost src]$
```

4.4 以上错误的解决方式

方式1: 更改路径的宏

```
export LD_LIBRARY_PATH=库的路径
```

```
[zpp@localhost src]$ ls
add.cpp  a.out  libadd.so  main.cpp
[zpp@localhost src]$ ./a.out
./a.out: error while loading shared libraries: libadd.so: cannot open shared object file: No such file or directory
[zpp@localhost src]$ export LD_LIBRARY_PATH=.    更改路径宏值
[zpp@localhost src]$ ./a.out
11
[zpp@localhost src]$
```

方法2: 将自己的动态库放入到系统的库函数目录中 (/lib64 /usr/lib64)

```
[zpp@localhost src]$ ./a.out
11
[zpp@localhost src]$ ls /lib64/
audit                                libgobject-2.0.so.0.5600.1
bind9-export                         libgomp.so.1
cmake                                libgomp.so.1.0.0
cracklib_dict.hwm                   libgpg-error.so.0
cracklib_dict.pwd                   libgpg-error.so.0.10.0
cracklib_dict.pwi                   libgpgme-pthread.so.11
crt1.o                              libgpgme-pthread.so.11.8.1
crti.o                              libgpgme.so.11
crtn.o                              libgpgme.so.11.8.1
dbus-1                              libgssapi_krb5.so.2
device-mapper                       libgssapi_krb5.so.2.2
ebtables                            libgssrpc.so.4
elfutils                            libgssrpc.so.4.2
fipscheck                           libgthread-2.0.so.0
games                               libgthread-2.0.so.0.5600.1
gconv                               libguestlib.so.0
gcrt1.o                             libguestlib.so.0.0.0
gettext                             libhandle.so.1
gio                                  libhandle.so.1.0.3
girepository-1.0                   libhdfs.so.0
krb5                                 libhdfs.so.0.0.0
ld-2.17.so                         libhistory.so.6
ld-linux-x86-64.so.2               libhistory.so.6.2
libc7.so.1                         libidn.so.11
libc7.so.1.1.0                    libidn.so.11.6.11
libadd.so                          libieee.a
```

5、动态库和静态库编译生成可执行程序的大小比较

```
[zpp@localhost src]$ ls
add.cpp  a.out  b.out  main.cpp
[zpp@localhost src]$ ls -l
总用量 32
-rw-rw-r--. 1 zpp zpp 40 1月 31 14:34 add.cpp
-rwxrwxr-x. 1 zpp zpp 8960 1月 31 14:37 a.out
-rwxrwxr-x. 1 zpp zpp 8984 1月 31 14:47 b.out
-rw-rw-r--. 1 zpp zpp 205 1月 31 14:34 main.cpp
[zpp@localhost src]$
```

链接动态库生成的可执行程序

链接静态库生成的可执行程序

5> 如何使用第三方库

1、C/C++语言默认是支持标准输入输出库的，但是其他的相关函数库需要第三方引入，并且，编译程序时，需要链接上对应的库

2、使用数学库：#include<math.h>


```

[zpp@localhost day3]$ man math.h
[zpp@localhost day3]$ vi 01math.cpp
[zpp@localhost day3]$ g++ 01math.cpp
[zpp@localhost day3]$ ./a.out
8
[zpp@localhost day3]$ g++ 01math.cpp -lm -lc
[zpp@localhost day3]$ ./a.out
8
[zpp@localhost day3]$ █

```

使用数学库，需要连接上数学库

链接标准的c库

3、线程支持类库：#include<pthread.h>

如果不能支持线程库：sudo yum install manpages-posix manpages-posix-dev

```

[zpp@localhost day3]$ g++ 02pthread.cpp
/tmp/cciRH7Ug.o: 在函数'main'中:
02pthread.cpp:(.text+0x5a): 对'pthread_create'未定义的引用
collect2: 错误: ld 返回 1
[zpp@localhost day3]$ g++ 02pthread.cpp -lpthread
[zpp@localhost day3]$ ./a.out
hello world
hello world
hello world
hello world
hello world
hello world
hello world
^C
[zpp@localhost day3]$ █

```

对于线程支持库的相关函数，必须链接对于的

```

#include<iostream>
#include<math.h>           //数学相关函数所在的头文件
#include<pthread.h>        //线程支持库的相关头文件
#include<unistd.h>
using namespace std;

//定义分支线程的线程体函数
void *task(void *arg)
{
    while(1)
    {
        cout << "hello world"<<endl;
        sleep(1);           //休眠函数
    }
}

int main(int argc, const char *argv[])
{
    pthread_t tid;           //定义一个线程号变量，用于存储创建出来的线程的线程号

    //创建线程
    if(pthread_create(&tid, NULL, task, NULL) != 0)
    {
        cout << "pthread_create error" << endl;
    }
}

```

```
}

//主线程
while(1);

return 0;
}
```

1.3 Makefile文件

1> 什么是Makefile

用于工程项目管理的一个文本文件，文件名为**Makefile**的文本文件
Makefile可以大写，也可以小写，一般**Makefile**首字母使用大写
如果**Makefile**和**makefile**两个文件都存在，系统会默认使用小写的

```
[zpp@localhost 01make]$ ls
makefile  Makefile
[zpp@localhost 01make]$ make → 默认执行小写的makefile
makefile success
[zpp@localhost 01make]$ █
```

2> 什么是make

make是一个执行**Makefile**的工具，是一个解释器
用来对**Makefile**中的命令进行解析并执行一个**shell**指令

make这个指令在 `/usr/bin` 中，默认**linux**系统中都已经安装
如果没有安装 **make**，安装指令如下
`sudo yum install make`
查看是否安装成功：`make --version`

3> Makefile的用途

描述了整个工程的编译、链接规则
软件项目的自动化编译，相当于给软件编译写一份脚本文件

4> 学习Makefile的必要性

Linux/Unix环境下开发的必备技能
系统架构师、项目经理的核心技能
研究开源项目、**Linux**内核原码的必需品
加深对底层软件构造系统及过程的理解

5> 如何学习Makefile

1、理论基础

软件的构造过程、程序的编译和链接：预处理 --> 编译 --> 汇编-->链接
面向依赖的思想：

依赖一个.ccpp文件的程序

可执行程序 <--依赖于-- .o文件 <--依赖于-- .s文件 <--依赖于-- .i文件 <--依赖于-- .cpp文件

依赖多个.ccpp文件的程序

可执行程序 <--依赖于-- .o文件 <--依赖于-- .s文件 <--依赖于-- .i文件 <--依赖于-- .cpp文件

| -- .o文件 <--依赖于-- .s文件 <--依赖于-- .i文件 <--依赖于-- .cpp文件

| -- .o文件 <--依赖于-- .s文件 <--依赖于-- .i文件 <--依赖于-- .cpp文件

| -- .o文件 <--依赖于-- .s文件 <--依赖于-- .i文件 <--依赖于-- .cpp文件

2、项目编程基础

C++程序语言基础

C语言基础

多文件原码管理、头文件包含、函数声明与定义

6> Makefile的工作过程

Makefile本身是面向依赖进行编写的

源文件 ---> 编译 ----> 目标文件 ---> 链接 ---->可执行文件

hello.cpp ---->hello.o --->hell

本质下一步就可以生成可执行程序，但是，由于在生成可执行程序时，可能会有多个文件进行参与，后期其他文件可能要进行更改，更改后，会影响到可执行程序的执行，其他没有更改的文件也要参与编译，浪费时间

7> 第一个Makefile

1、编写一个hello.cpp文件

```
#include<iostream>
#include<stdio.h>
using namespace std;

int main(int argc, const char *argv[])
{
    cout << "hello world" << endl;

    return 0;
}
```

2、通过终端编译程序

```

[zpp@localhost 02make]$ g++ -E hello.cpp -o hello.i 预处理
[zpp@localhost 02make]$ g++ -S hello.i -o hello.s 编译
[zpp@localhost 02make]$ g++ -c hello.s -o hello.o 汇编
[zpp@localhost 02make]$ g++ hello.o -o hello 链接
[zpp@localhost 02make]$ ls
a.out hello hello.cpp hello.i hello.o hello.s
[zpp@localhost 02make]$ ./hello 执行可执行程序
hello world

```

3、使用Makefile管理工程

编写Makefile文件

```

# Makefile中的注释是以#开头
# 语法格式:
# 目标: 依赖
# 通过依赖生成目标的指令
# 注意: 指令前面必须使用同一个tab键隔开, 不能使用多个空格顶过来

hello:hello.o
    g++ hello.o -o hello

hello.o:hello.s
    g++ -c hello.s -o hello.o

hello.s:hello.i
    g++ -S hello.i -o hello.s

hello.i:hello.cpp
    g++ -E hello.cpp -o hello.i

```

简化的Makefile文件

```

# Makefile中的注释是以#开头
# 语法格式:
# 目标: 依赖
# 通过依赖生成目标的指令
# 注意: 指令前面必须使用同一个tab键隔开, 不能使用多个空格顶过来

hello:hello.o
    g++ hello.o -o hello

hello.o:hello.cpp
    g++ -c hello.cpp -o hello.o

```

4、执行Makefile文件

```

make    --->默认找到Makefile中的第一个目标开始进行执行
make 目标 --->    找到对应的目标进行执行

```

```
[zpp@localhost 02make]$ make 默认找第一个目标进行执行
g++ -c hello.cpp -o hello.o
g++ hello.o -o hello
[zpp@localhost 02make]$ ls
hello hello.cpp hello.o Makefile
[zpp@localhost 02make]$ make clean 直接找到对应的目标进行执行
rm hello.o hello
[zpp@localhost 02make]$ ls
hello.cpp Makefile
[zpp@localhost 02make]$
```

8> Makefile的语法规则

1、规则

构成Makefile的基本单元，构成依赖关系的核心部件

其他内容可以看做为规则的服务

2、变量

类似于C++中的宏，使用变量：\$(变量名) 或者 \${变量名}

作用：使得Makefile更加灵活

3、条件执行

根据某一变量的值来控制make执行或者忽略Makefile的某一部分

4、函数

文本处理函数：字符串的替换、查找、过滤等等

文件名的处理：读取文件/目录名、前后缀等等

5、注释

Makefile中的注释，是以#号开头

9> 规则

1、规则的构成：目标、目标依赖、命令

2、语法格式

目标：目标依赖

命令

注意事项：

命令必须使用tab键开头，一般是shell指令

一个规则中，可以无目标依赖，仅仅是实现某种操作

一个规则中可以没有命令，仅仅描述依赖关系

一个规则中，必须要有一个目标

```

#仅仅只有目标和依赖，没有命令
all:hello

# Makefile中的注释是以#开头
# 语法格式：
# 目标：依赖
# 通过依赖生成目标的指令
# 注意：指令前面必须使用同一个tab键隔开，不能使用多个空格顶过来

hello:hello.o
    g++ hello.o -o hello

hello.o:hello.cpp
    g++ -c hello.cpp -o hello.o

# 此处一个规则是没有依赖，只有目标和指令
clean:
    rm hello.o hello

```

只有目标和目标依赖，没有命令

三者都有

只有目标和命令，没有目标依赖

3、目标详解

1) 默认目标

一个Makefile里面可以有多个目标，一般会选择第一个当做默认目标
也就是make默认执行的目标

2) 多目标

一个规则中可以有多个目标，多个目标具有相同的生成命令和依赖文件

```

clean distclean:
    rm hello.[^cpp] hello

```

```

# 此处一个规则是没有依赖，只有目标和指令
clean distclean:
    rm hello.o hello

```

多目标规则

3)多规则目标

多个规则可以是同一目标

```

all:test1
all:test2
test1:
    echo "hello"
test2:
    echo "world"

```

```
[zpp@localhost 03make]$ make
hello
world
[zpp@localhost 03make]$ cat Makefile
all:test1
all:test2

test1:
    @echo "hello"
test2:
    @echo "world"
```

多规则目标

4) 伪目标

并不是一个真正的文件名，可以看做是一个标签
无依赖，相比一般文件，不会重新生成、执行
伪目标：可以无条件执行，相当于对应的指令

```
eg:
    .PHONY:clean          #设置伪目标
    clean:
        rm hello.[^cpp] hello
```

```
[zpp@localhost 02make]$ make clean
rm hello.o hello
[zpp@localhost 02make]$ cat Makefile

#仅仅只有目标和依赖，没有命令
all:hello

# Makefile中的注释是以#开头
# 语法格式：
# 目标： 依赖
#         通过依赖生成目标的指令
# 注意： 指令前面必须使用同一个tab键隔开，不能使用多个空格顶过来

hello:hello.o
    g++ hello.o -o hello

hello.o:hello.cpp
    g++ -c hello.cpp -o hello.o

# 此处一个规则是没有依赖，只有目标和指令
.PHONY:clean
clean distclean:
    rm hello.o hello
```

伪目标

4、目标依赖

1) 文件时间戳

根据时间戳来判断目标依赖是否要进行更新
所有文件都更改过，则对所有文件进行编译，生成可执行程序
在上次make之后修改过的cpp文件，会被重新编译
在上次make只写修改过的头文件，依赖该头文件的目标依赖也会重新编译

2) 模式匹配

% ---->通配符匹配
\$@ ---->目标
\$^ ---->依赖
\$< ---->第一个依赖
* ---->普通通配符

注意：%是Makefile中的规则通配符，*是普通通配符

头文件

```
#ifndef _OPERATOR_H
#define _OPERATOR_H

int add(int m, int n);

#endif
```

源文件

```
int add(int m, int n)
{
    return m+n;
}
```

主程序：

```
#include<iostream>
#include"operator.h"

using namespace std;
int main(int argc, const char *argv[])
{

    cout << add(3,5) << endl;

    return 0;
}
```

#第一个依赖关系

all:operator

operator:main.o operator.o

```
#   g++ main.o operator.o -o operator
g++ $^ -o $@
```

#main.o:main.cpp

```
#   g++ -c $< -o $@
```

#operator.o:operator.cpp

```
#   g++ -c $^ -o $@
```

#将上述两个规则统一成一个规则

#此时的%表示所有的Makefile内容，不能换成*

```
%.o:%.cpp
```

```
g++ -c $< -o $@
```

```
clean:
```

```
rm -f *.o operator
```

#此处的*是普通通配符

5、命令

1) 命令的组成

由shell命令组成，以tab键开头

2) 命令的执行

每条命令，make会开一个进程

每条命令执行完，make会检测这个命令的返回码

如果返回成功，则继续执行后面的命令

如果返回失败，则make会终止当前执行的规则并退出

3) 并发执行命令

```
make -j4      ----->表示开辟4个线程执行
```

```
time make     ----->执行make时，显示当前时间
```

10> 变量

1、变量基础

1) 变量定义：变量名 = 变量值

2) 变量的赋值

追加赋值：+= ----->在原有的基础上追加相关内容

条件赋值：?= ----->如果之前没有值，则为变量赋值，如果之前有值，则不进行赋值

3) 变量的使用： \$(变量名)或者\${变量名}

2、变量的分类

1) 立即展开变量

使用:=操作符进行赋值

在解析阶段直接赋值常量字符串

2) 延迟展开变量

使用=操作符进行赋值

将最后一次赋值的结果给变量名使用

3) 注意事项

一般在目标、目标依赖中使用立即展开赋值

在命令中一般使用延迟展开赋值变量

3、变量的外部传递

可以通过命令行给变量进行赋值操作

```
make ARCH=g++
```

```

var1 = main.o          #定义一个变量并赋值
var1 += operator.o    #给变量追加值

CC:=g++

#第一个依赖关系
all:operator

operator:$(var1)
#   g++ main.o operator.o -o operator
    $(CC) $^ -o $@

#main.o:main.cpp
#   g++ -c $< -o $@
#operator.o:operator.cpp
#   g++ -c $^ -o $@
#将上述两个规则统一成一个规则

#此时的%表示所有的Makefile内容，不能换成*
%.o:%.cpp
    $(CC) -c $< -o $@

clean:
    rm -f *.o operator    #此处的*是普通通配符

```

11> 条件执行

1、关键字

```

ifeq 、else 、endif
ifneq 、else 、endif

```

2、使用

```

ifeq (要判断的量, 判断的值)
    Makefile语句
else
    Makefile语句
endif

```

注意:

条件语句从ifeq开始执行，括号与关键字自减使用空格隔开
括号里面挨着括号处，不允许加空格

```

#条件选择，如果变量COMPILE的值是g++那么CC的值就是g++，否则是gcc
ifeq ($(COMPILE), g++)
    CC:=g++
else
    CC:=gcc
endif

```

12> 总结

1、执行过程

进入编译目录、执行**make**命令
依赖关系解析阶段
命令执行阶段

2、依赖解析阶段

解析**Makefile**，建立依赖关系树
控制解析过程，引入**Makefile**，变量展开、条件执行
生成依赖关系树

3、命令执行阶段

把解析生成的依赖关系树加载到内存
按依赖关系，按顺序生成这些文件
再次编译**make**会检测文件的时间戳，判断是否过期
 如果无过期，在不在编译
 如果文件有更新，则依赖该文件的所有依赖关系上的目标都会重新编译

4、make执行结果

make的退出码
 0：表示成功执行
 1：表示允许错误，退出

13> 参考原码

1、头文件

```
#ifndef _OPERATOR_H
#define _OPERATOR_H

int add(int m, int n);

#endif
```

2、源文件

```
int add(int m, int n)
{
    return m+n;
}
```

3、主程序

```
#include<iostream>
#include"operator.h"

using namespace std;
int main(int argc, const char *argv[])
{

    cout << add(3,5) << endl;

    return 0;
}
```

4、Makefile

```
var1 = main.o          #定义一个变量并赋值
var1 += operator.o #给变量追加值

#条件选择，如果变量COMPILE的值是g++那么CC的值就是g++，否则是gcc
ifeq ($(COMPILE), g++)
    CC:=g++
else
    CC:=gcc
endif

#第一个依赖关系
all:operator

operator:$(var1)
#    g++ main.o operator.o -o operator
    $(CC) $^ -o $@

#main.o:main.cpp
#    g++ -c $< -o $@
#operator.o:operator.cpp
#    g++ -c $^ -o $@
#将上述两个规则统一成一个规则

#此时的%表示所有的Makefile内容，不能换成*
%.o:%.cpp
    $(CC) -c $< -o $@

clean:
    rm -f *.o operator          #此处的*是普通通配符
```

1.4 vscode 的配置

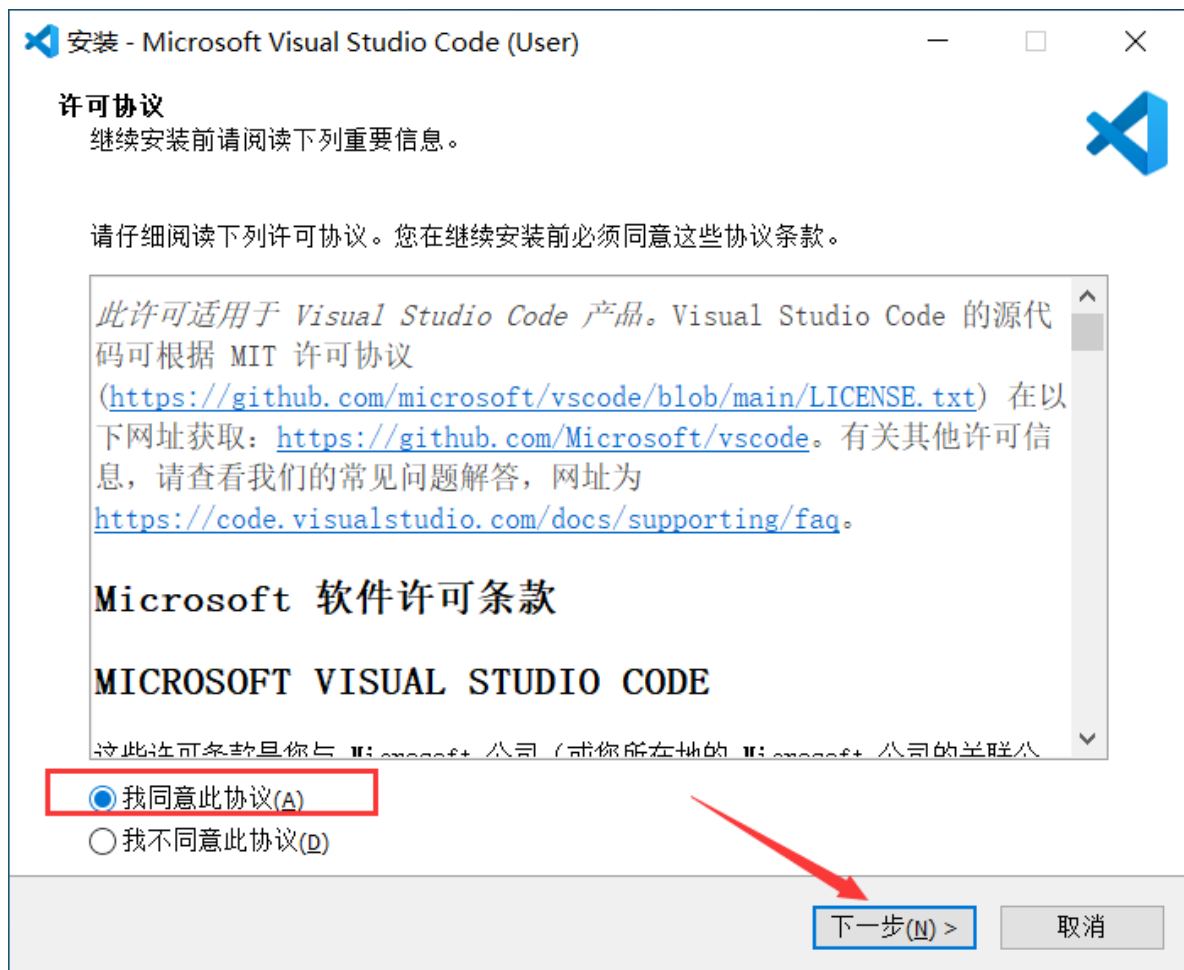
1> vsc安装与c++环境配置

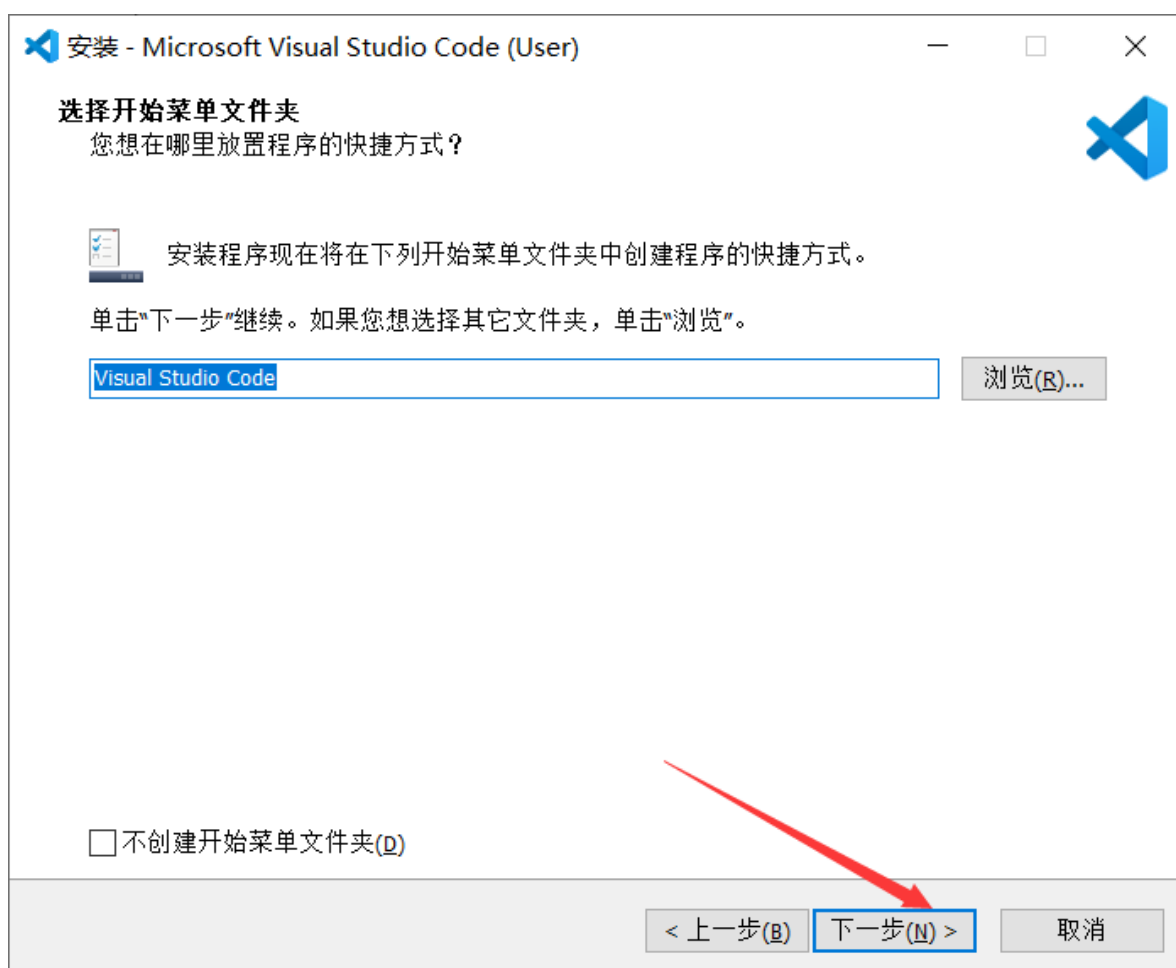
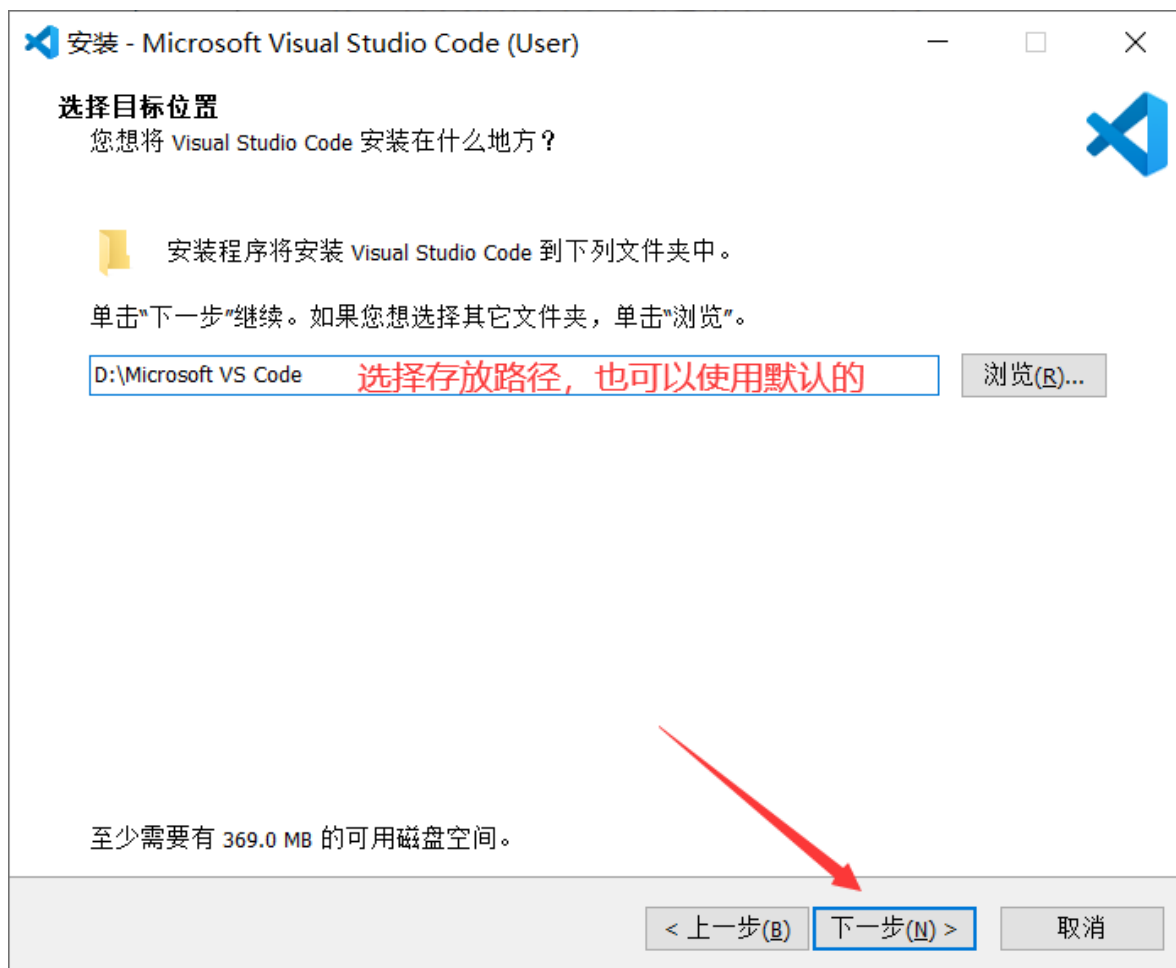
1、安装包下载：

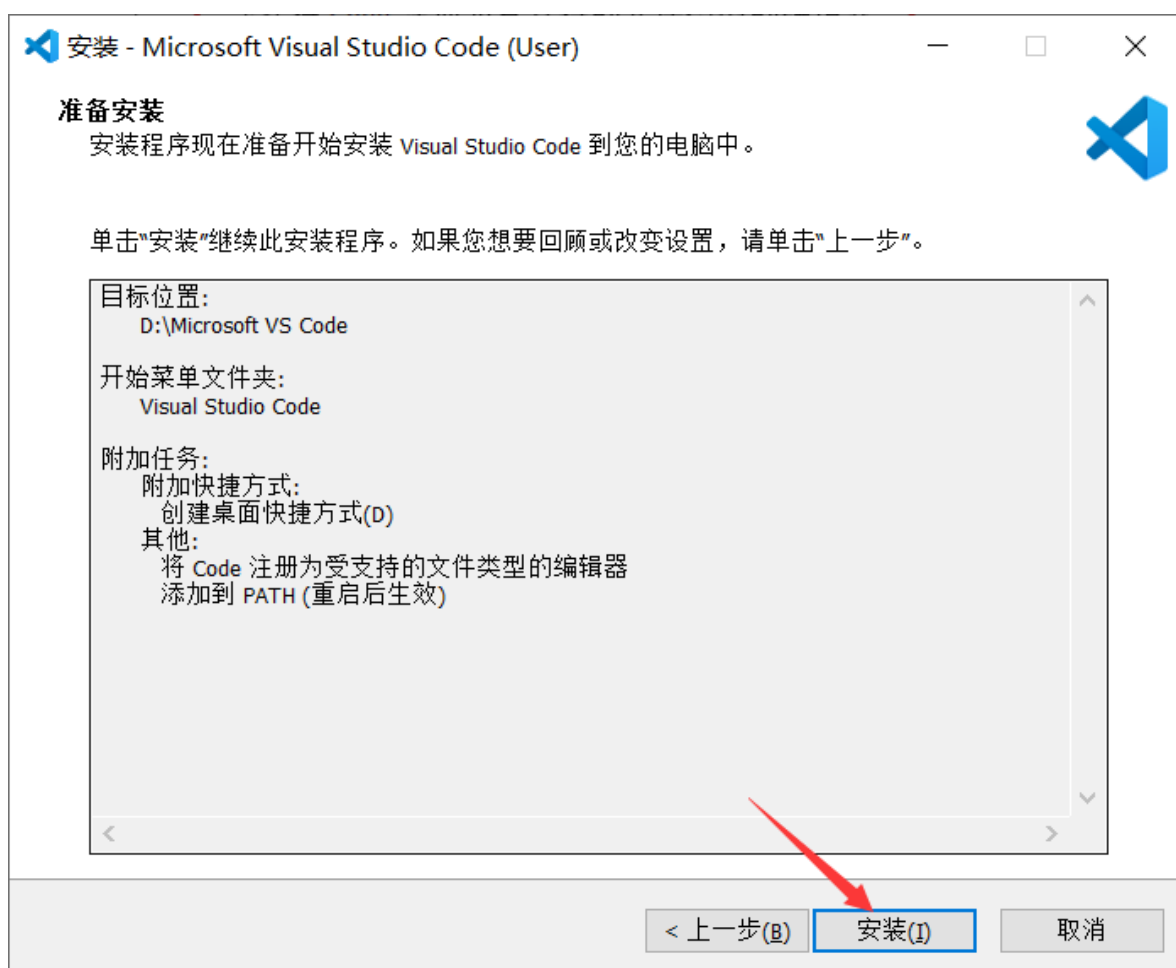
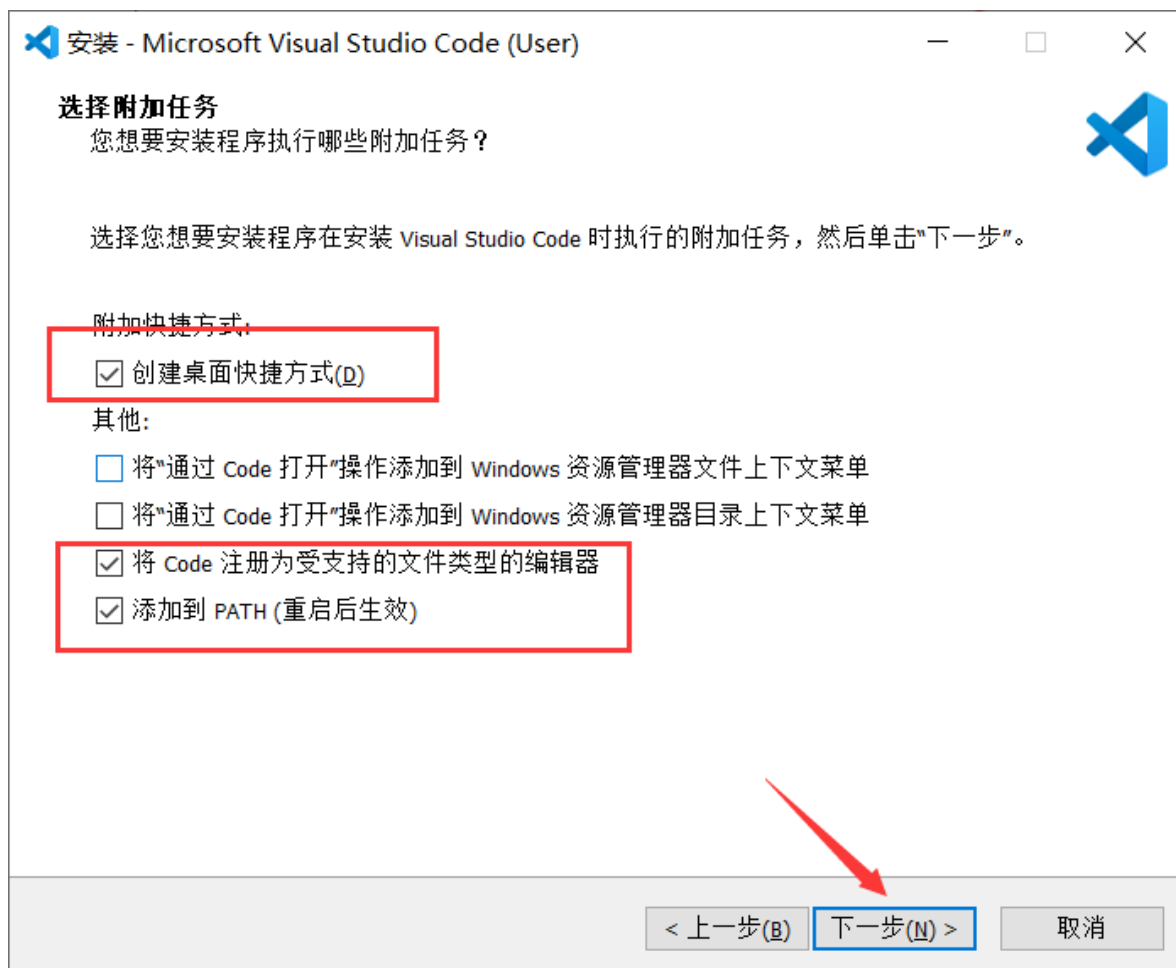
链接: https://pan.baidu.com/s/1zUUQbv0m2yJtr_a445bWVA?pwd=gbau 提取码: gbau

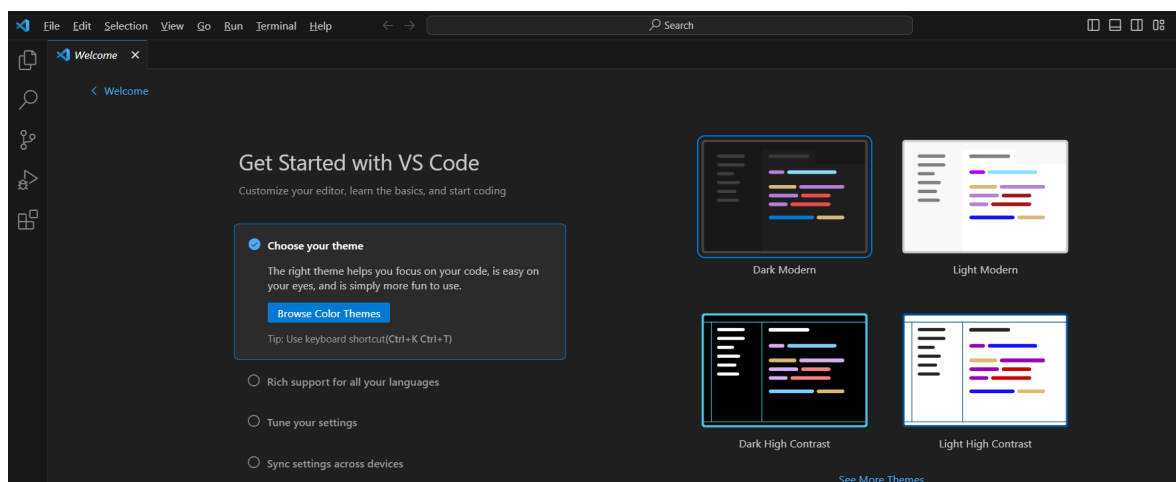
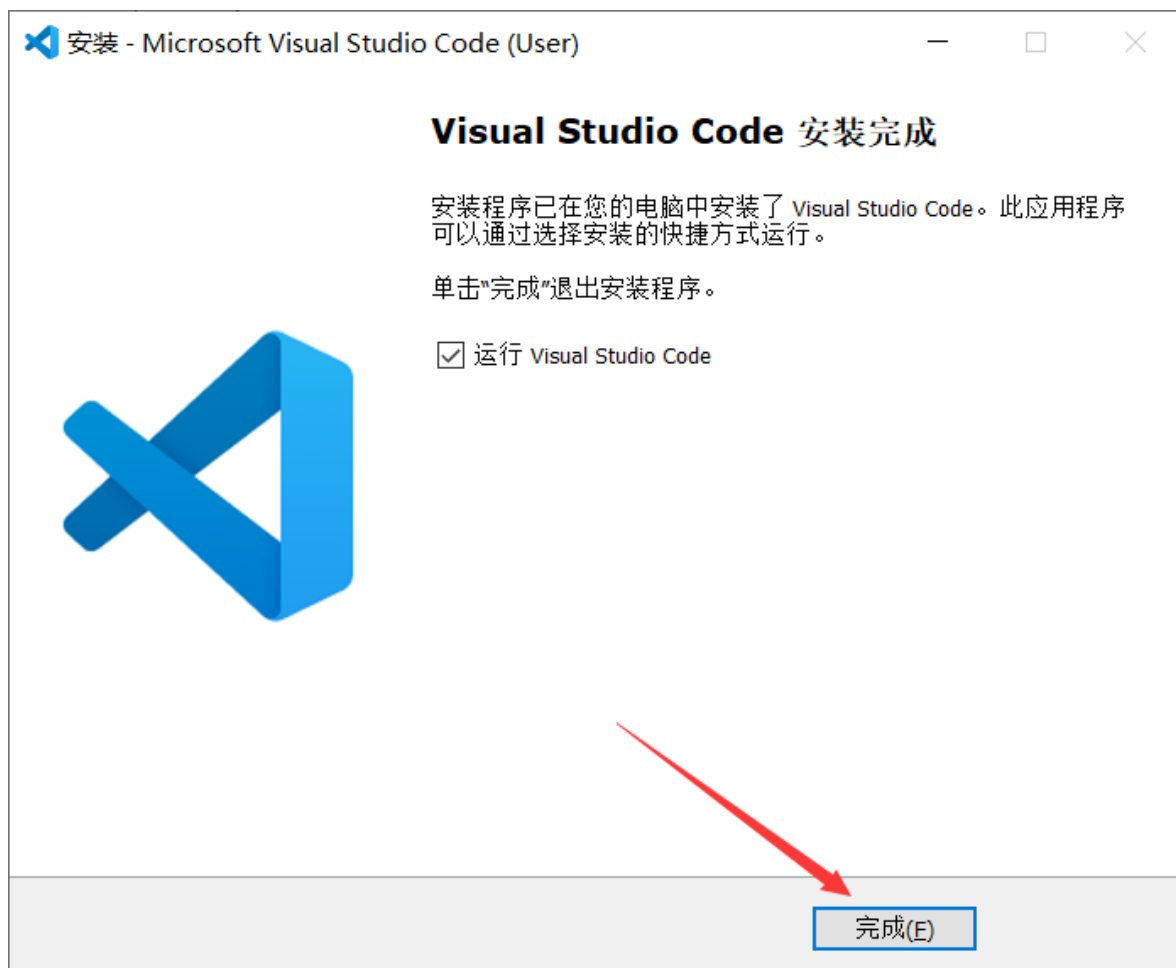
官网: <https://code.visualstudio.com/download>

2、安装vscode

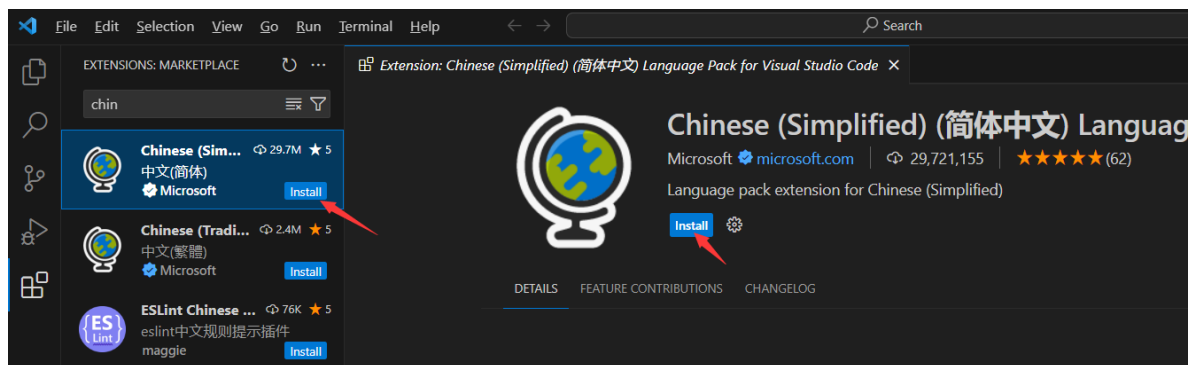
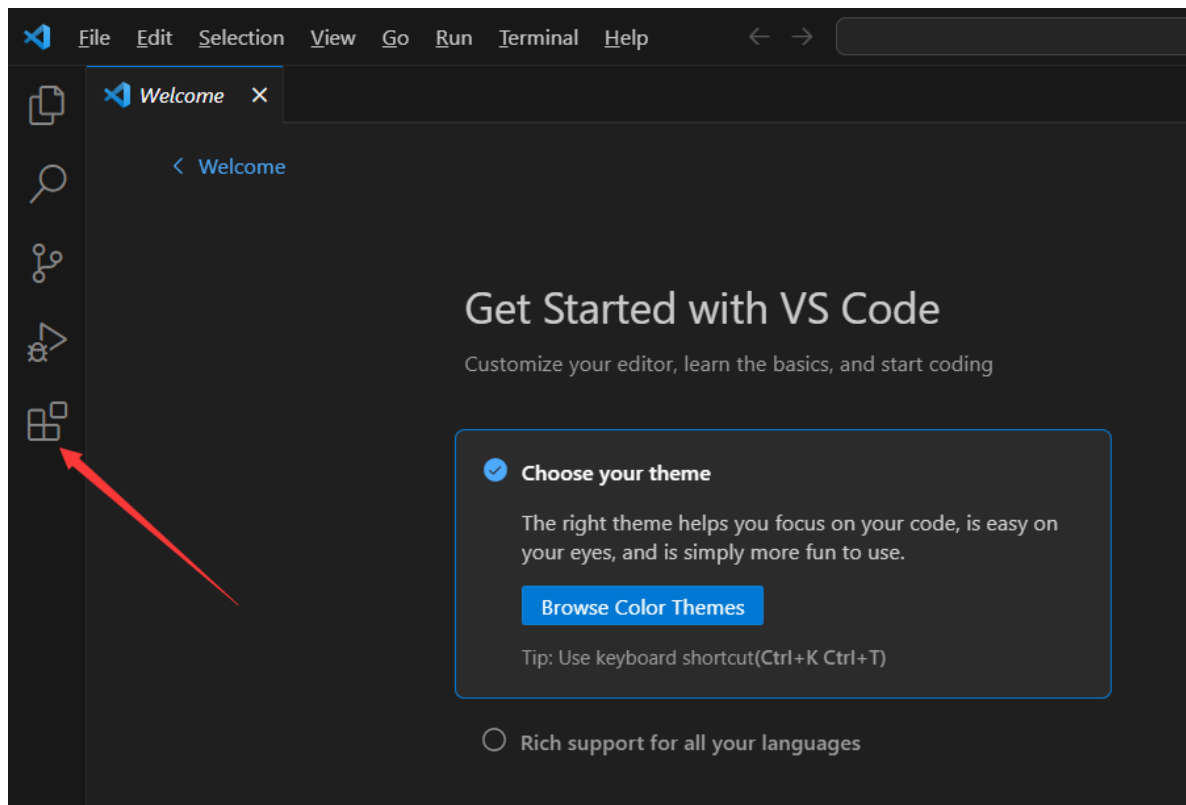








3、安装汉化插件



2> 下载编译工具及配置

1、工具下载官网

<https://www.mingw-w64.org/>

MinGW-w64

Overview

Changelog

Downloads

Contribute

Support

Donate



MinGW-w64

A complete runtime environment for GCC & LLVM
for 32 and 64 bit Windows



MinGW-w64 is an advancement of the original mingw.org project, created to support the GCC compiler on Windows systems. It has forked it in 2007 in order to provide support for 64 bits and new APIs. It has since then gained widespread use and distribution.

The development and community are very active and welcoming with new contributors every month and simple installers.

Headers, Libraries and Runtime

- More than a million lines of headers are provided, not counting generated ones, and regularly expanded to track new Windows APIs.
- Everything needed for linking and running your code on Windows.
- Winthreads, a pthreads library for C++11 threading support and simple integration with existing project.
- Winstorecompat, a work-in-progress convenience library that eases conformance with the Windows Store.
- Better-conforming and faster math support compared to VisualStudio's.

Tools

- gendef: generate Visual Studio .def files from .dll files.
- genidl: generate .idl files from .dll files.
- widl: compile .idl files.

MinGW-w64

Overview

Changelog

Downloads

Contribute

Support

Donate

MSYS2

Installation: [GitHub](#)

Sources

Tarballs for the mingw-w64 sources are hosted on [SourceForge](#).

The latest version from the 11.x series is **11.0.0**.

The latest version from the 10.x series is **10.0.0**.

The latest version from the 9.x series is **9.0.0**.

The latest version from the 8.x series is **8.0.2**.

The latest version from the 7.x series is **7.0.0**.

The latest version from the 6.x series is **6.0.0**.

The latest version from the 5.x series is **5.0.4**.

The old wiki has instructions for building [native](#) and [cross](#) toolchains.

Details on how to get the mingw-w64 code from Git and an Git-web viewer are available on [SourceForge](#).

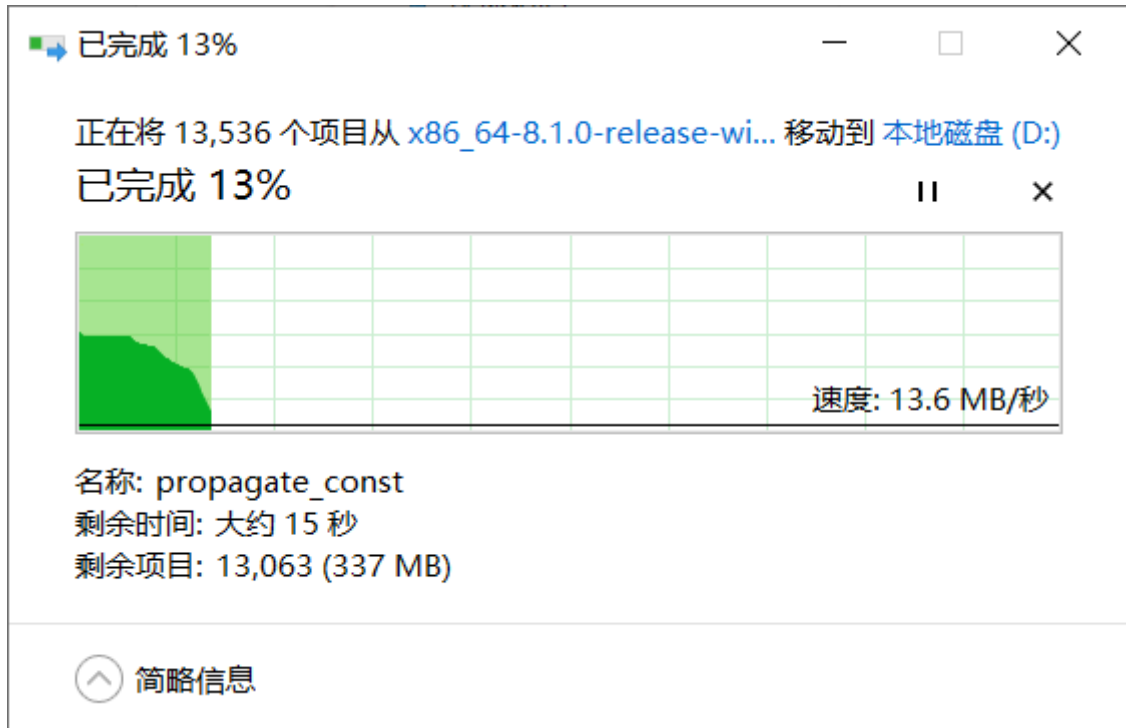
Unsorted complementary list

Totals: 161 Items		750.1 MB	49,071
MinGW-W64 Online Installer • MinGW-W64-install.exe			
MinGW-W64 GCC-8.1.0 • x86_64-posix-sjlj • x86_64-posix-seh • x86_64-win32-sjlj • x86_64-win32-seh • i686-posix-sjlj • i686-posix-dwarf • i686-win32-sjlj • i686-win32-dwarf			
MinGW-W64 GCC-7.3.0			

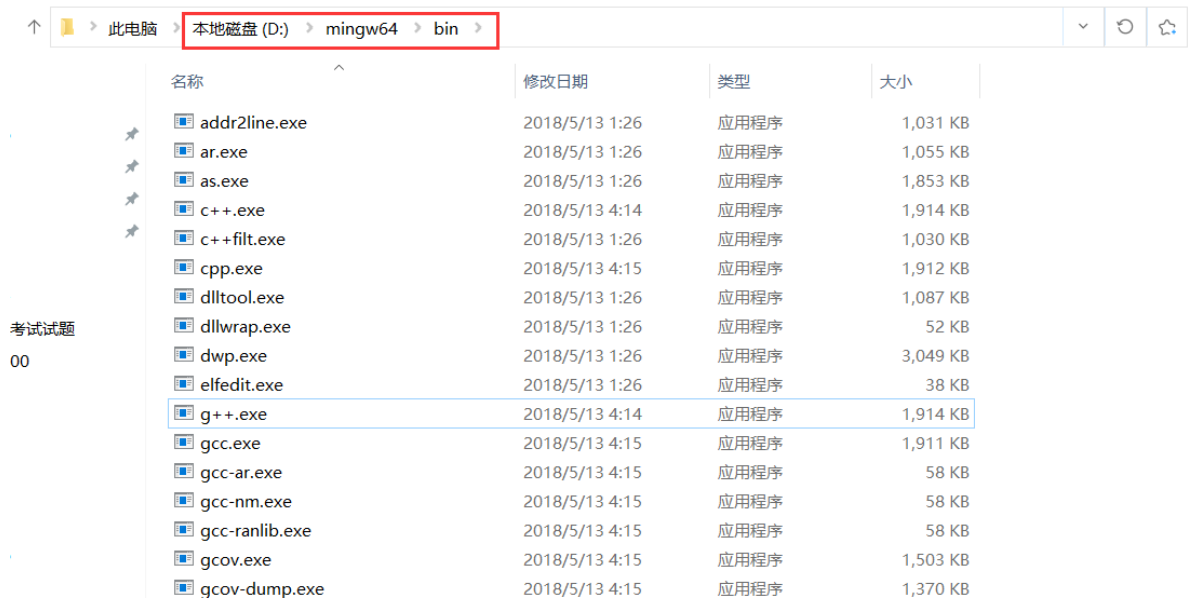
下载编译工具后，进行解压，并将其中的mingw 64文件目录单独取出



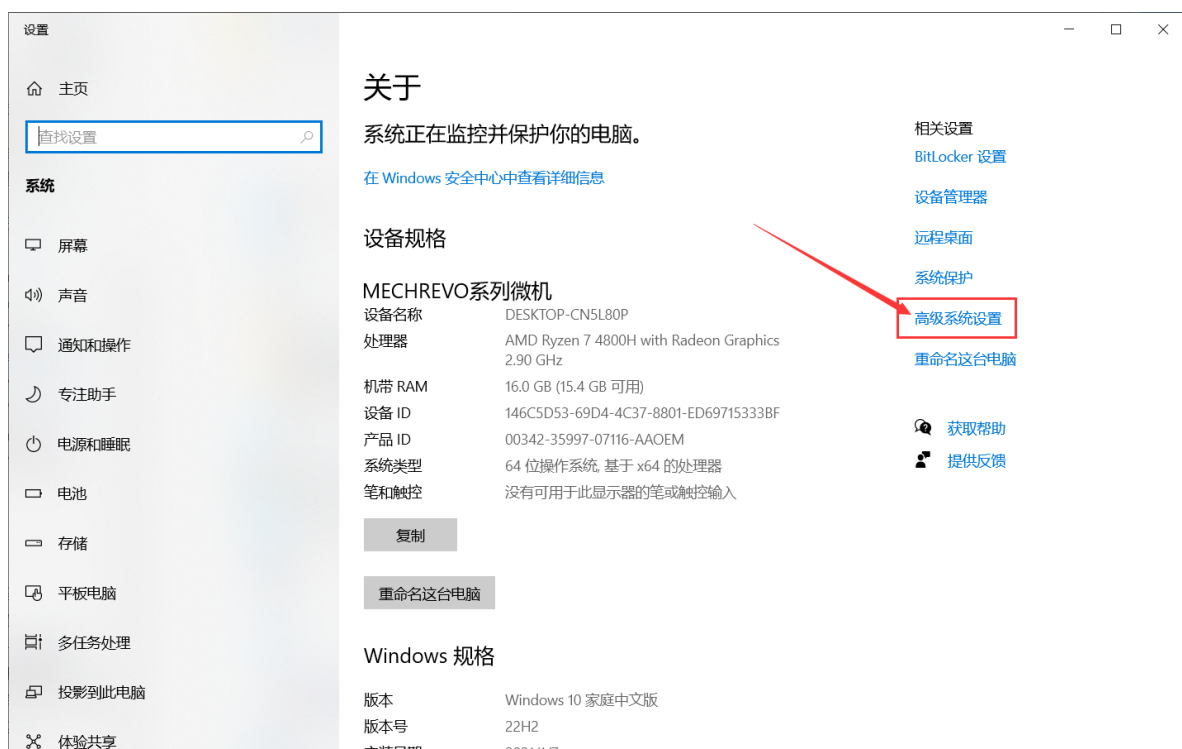
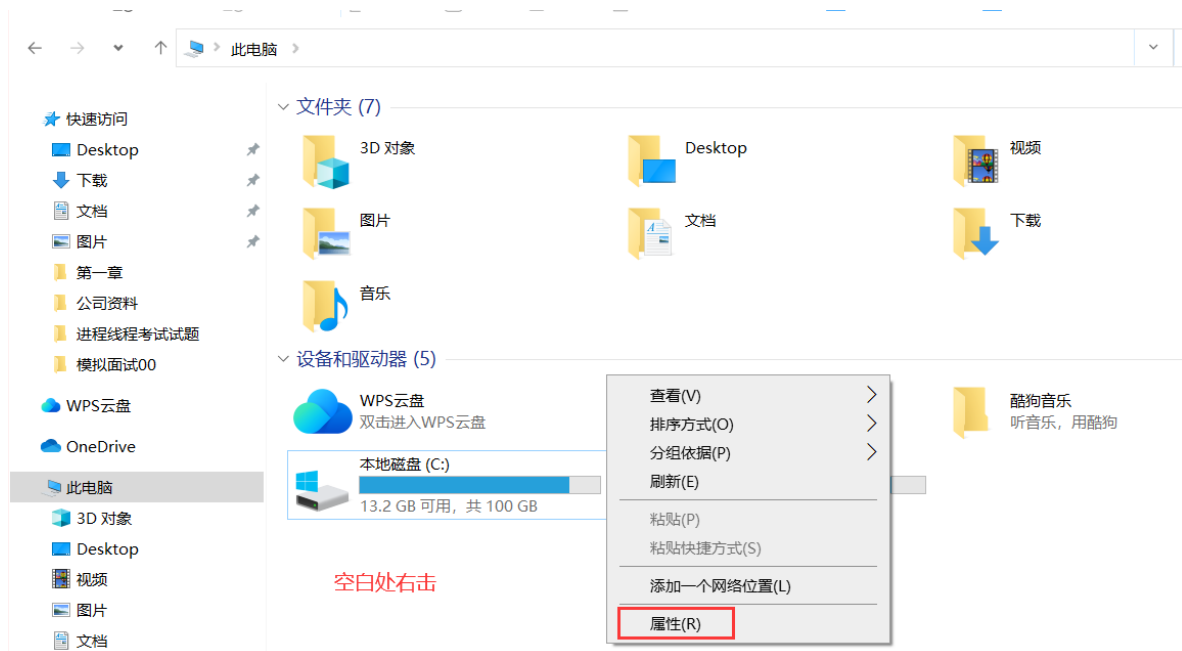
将其剪切到一个不含中文路径的目录下，我这里选择的D盘的根目录



配置环境变量



将路径进行复制--->打开高级系统设置



系统属性



计算机名

硬件

高级

系统保护

远程

要进行大多数更改，你必须作为管理员登录。

性能

视觉效果，处理器计划，内存使用，以及虚拟内存

设置(S)...

用户配置文件

与登录帐户相关的桌面设置

设置(E)...

启动和故障恢复

系统启动、系统故障和调试信息

设置(T)...

环境变量(N)...

确定

取消

应用(A)

鹏程万里 的用户变量(U)

变量	值
OneDrive	C:\Users\鹏程万里\OneDrive
Path	C:\Users\鹏程万里\AppData\Local\Microsoft\WindowsApps;D:\T...
TEMP	C:\Users\鹏程万里\AppData\Local\Temp
TMP	C:\Users\鹏程万里\AppData\Local\Temp

新建(N)...

编辑(E)...

删除(D)

系统变量(S)

变量	值
ComSpec	C:\Windows\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
NUMBER_OF_PROCESSORS	16
OS	Windows NT
Path	C:\Program Files (x86)\VMware\VMware Workstation\bin\D:\de...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE	AMD64
PROCESSOR_IDENTIFIER	AMD64

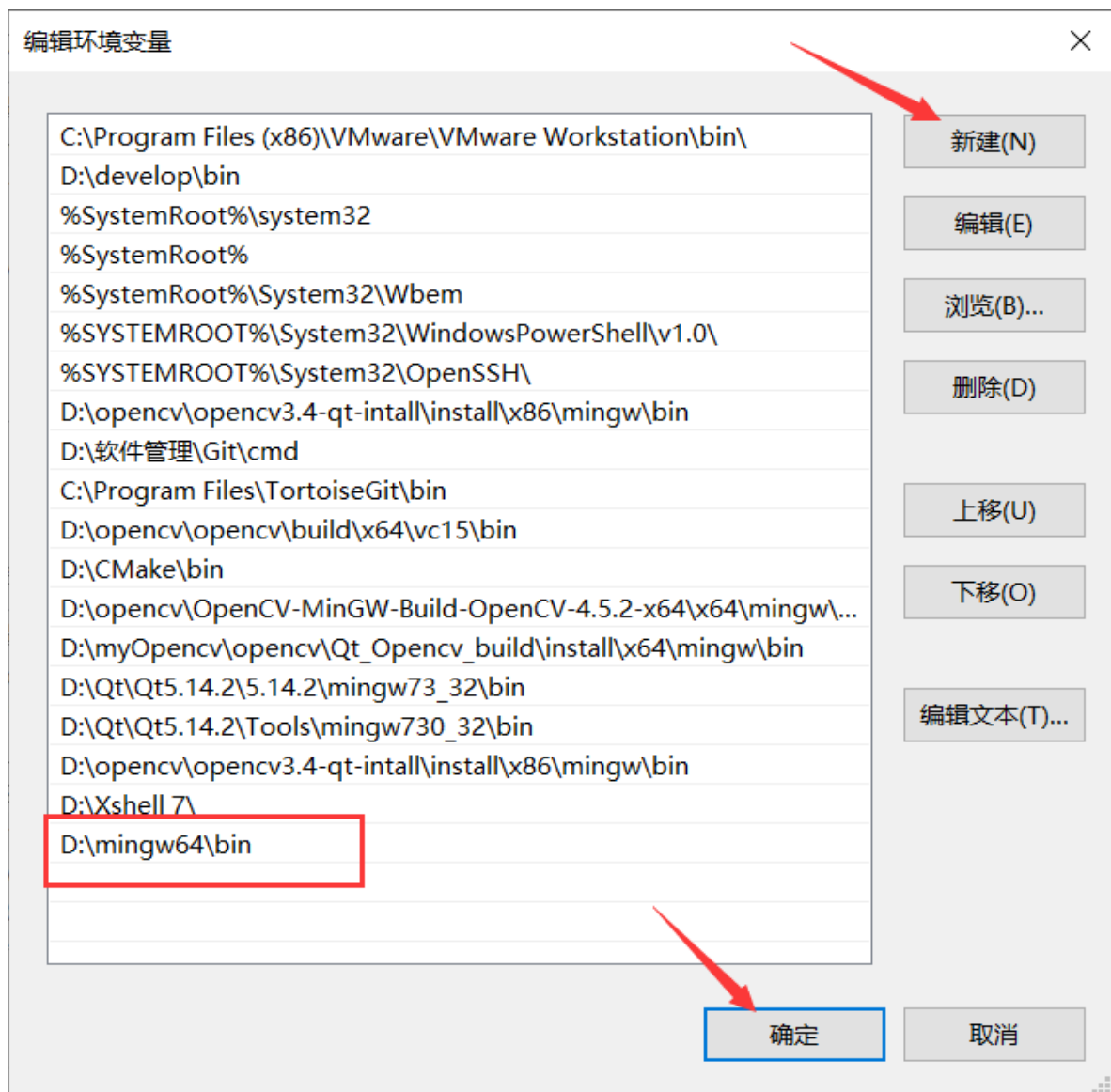
新建(W)...

编辑(I)...

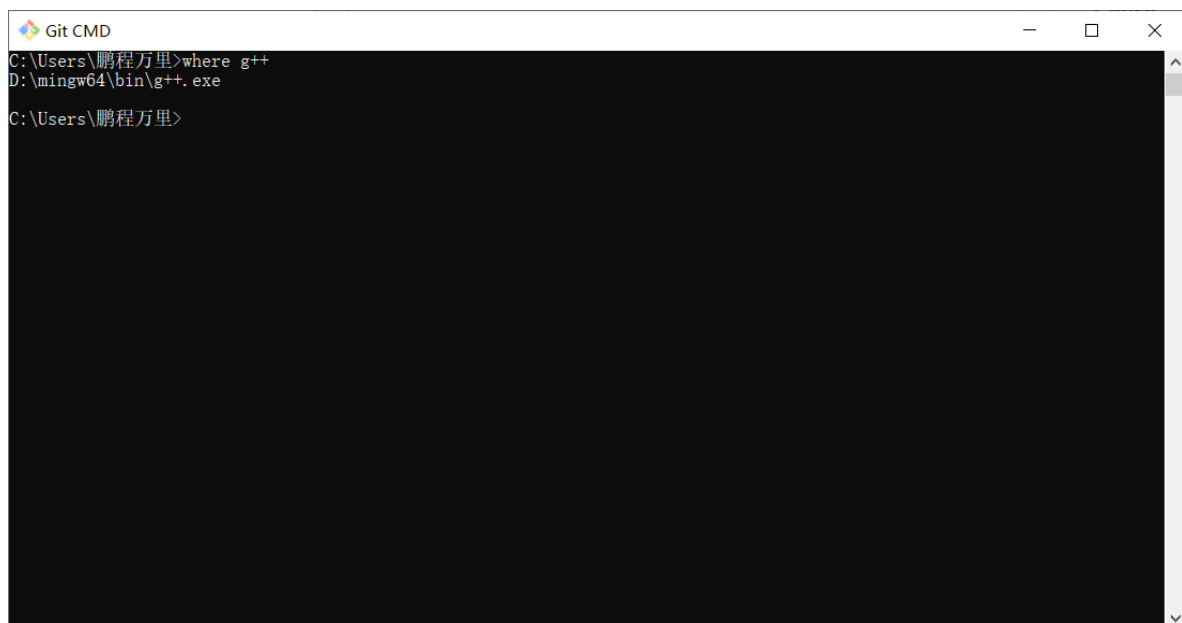
删除(L)

确定

取消

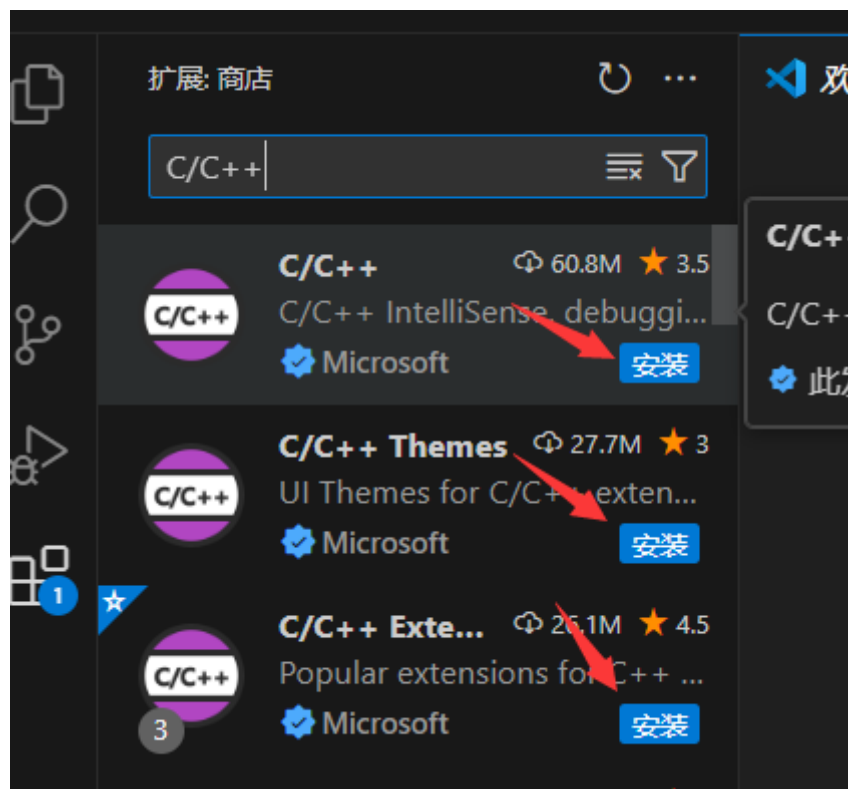


验证是否配置成功 ----> 调取windows下的cmd



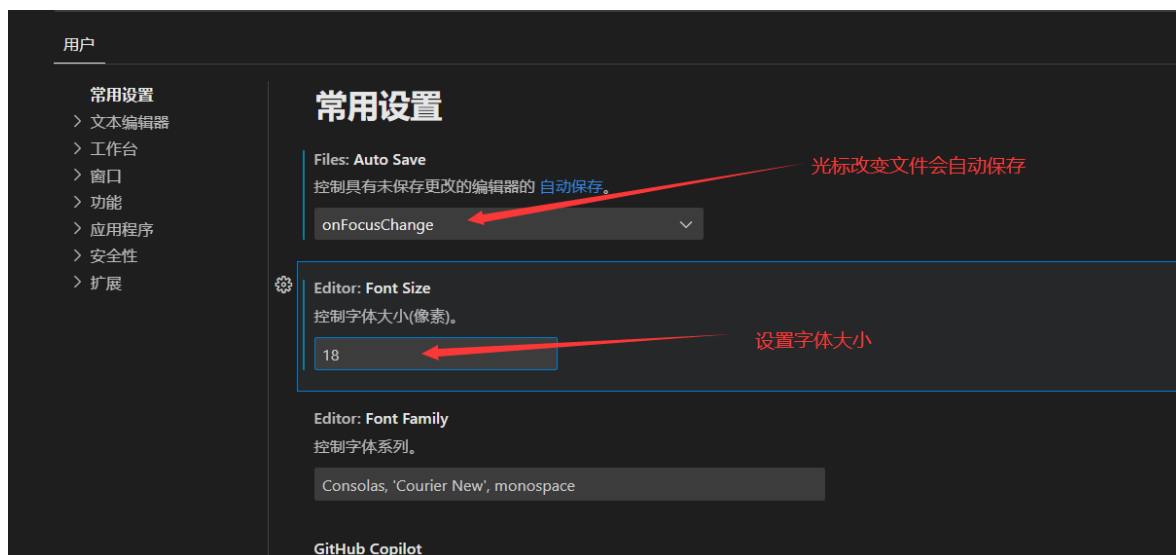
3> vscode中安装C++相关配置

1、安装C++相关的拓展工具



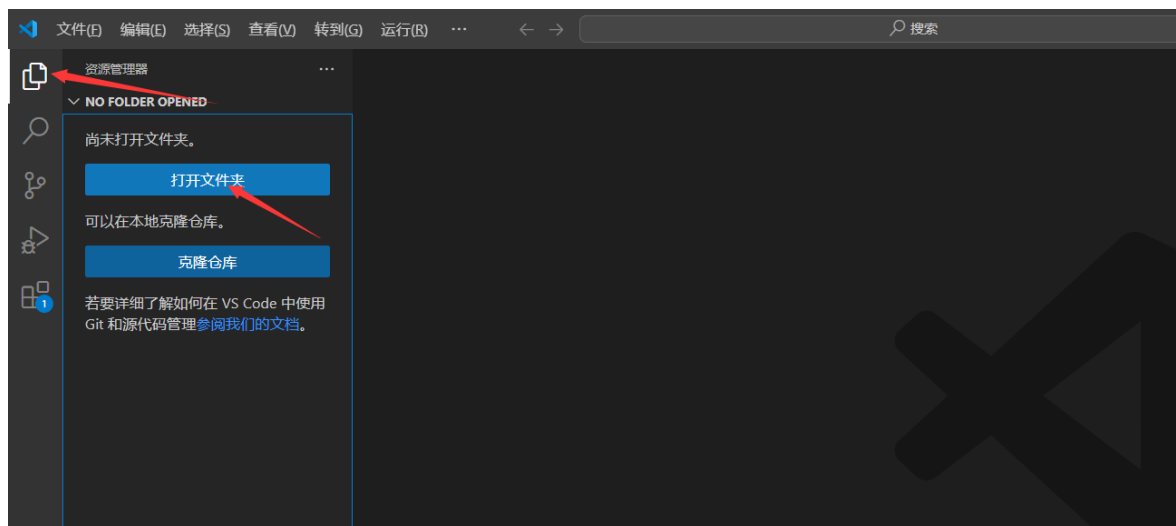
2、配置字体及其他内容

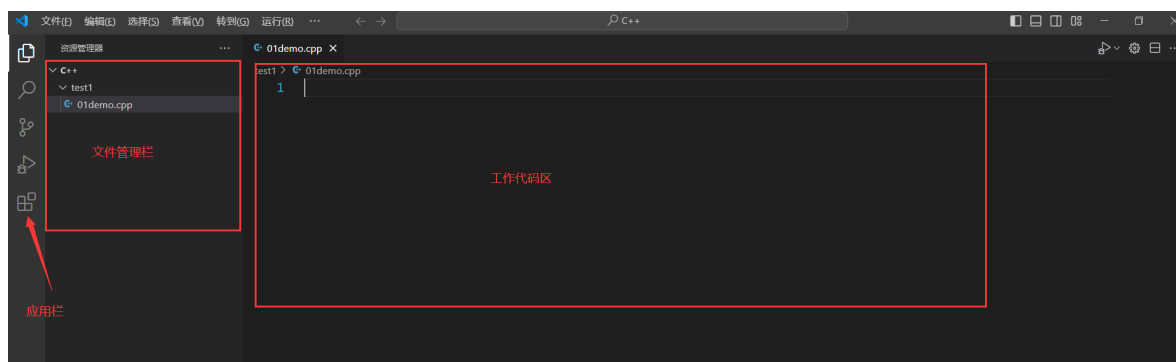
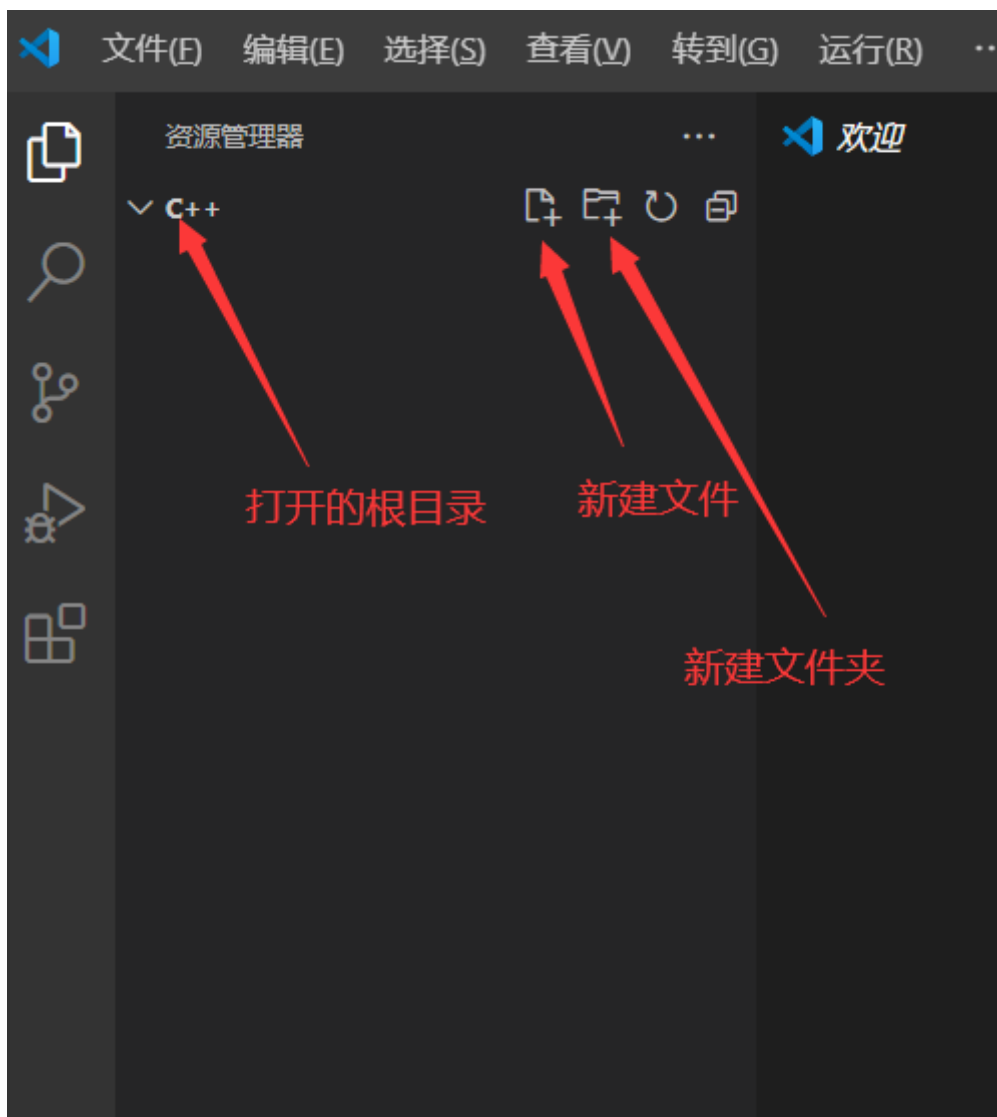




4> 代码编写及调试

1、文件夹的打开





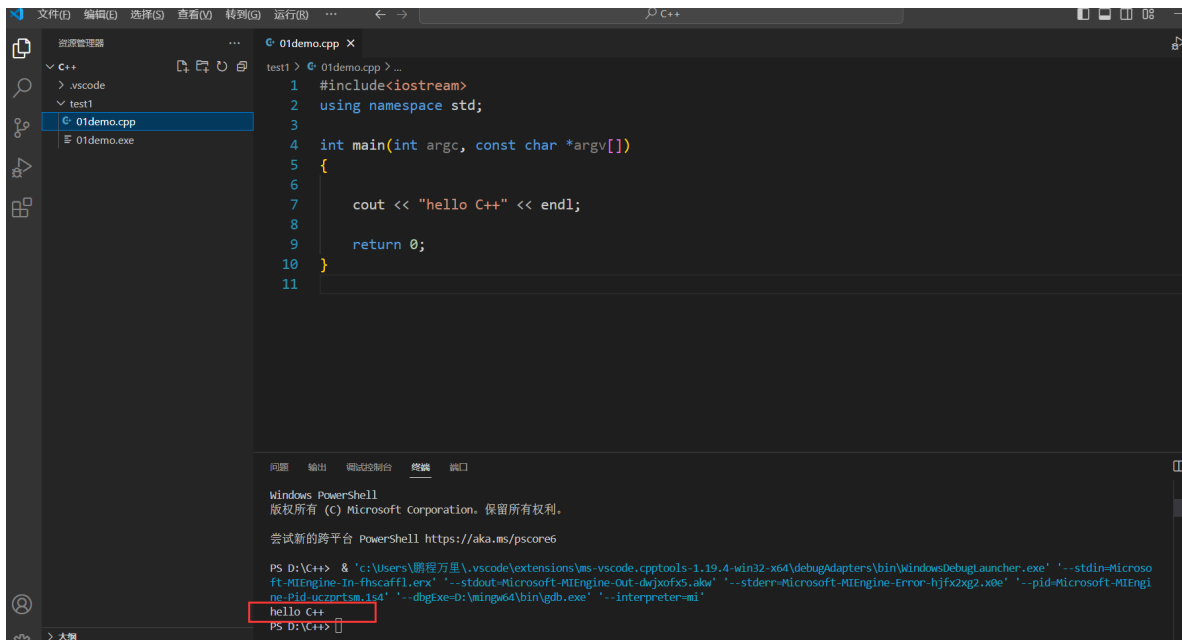
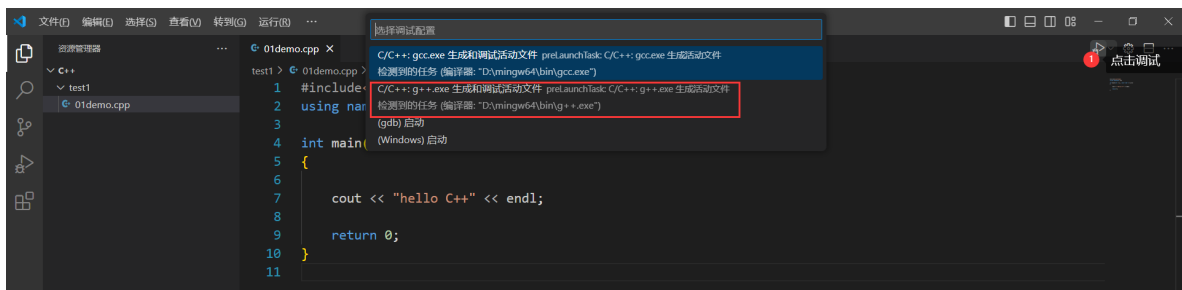
2、编写一个简单的单目录文件，并对其进行调试

```
#include<iostream>
using namespace std;

int main(int argc, const char *argv[])
{
    cout << "hello C++" << endl;

    return 0;
}
```

编译调试当前程序



对 .vscode 文件夹的更改



3、使用vscode完成对多文件编译的操作

operator.h

```

#ifndef _OPERATOR_H_
#define _OPERATOR_H_

//声明两个函数
int my_add(int m, int n);    //声明加法运算

int my_div(int m, int n);    //声明减法运算

#endif

```

operator.cpp

```

//加法运算的实现
int my_add(int m, int n)
{
    int sum = m+n;

    return sum;
}

//减法运算的实现
int my_div(int m, int n)
{
    int res = m-n;

    return m-n;
}

```

main.cpp

```

#include<iostream>

#include"operator.h"    //引入自定义函数的头文件

using namespace std;

int main(int argc, const char *argv[])
{

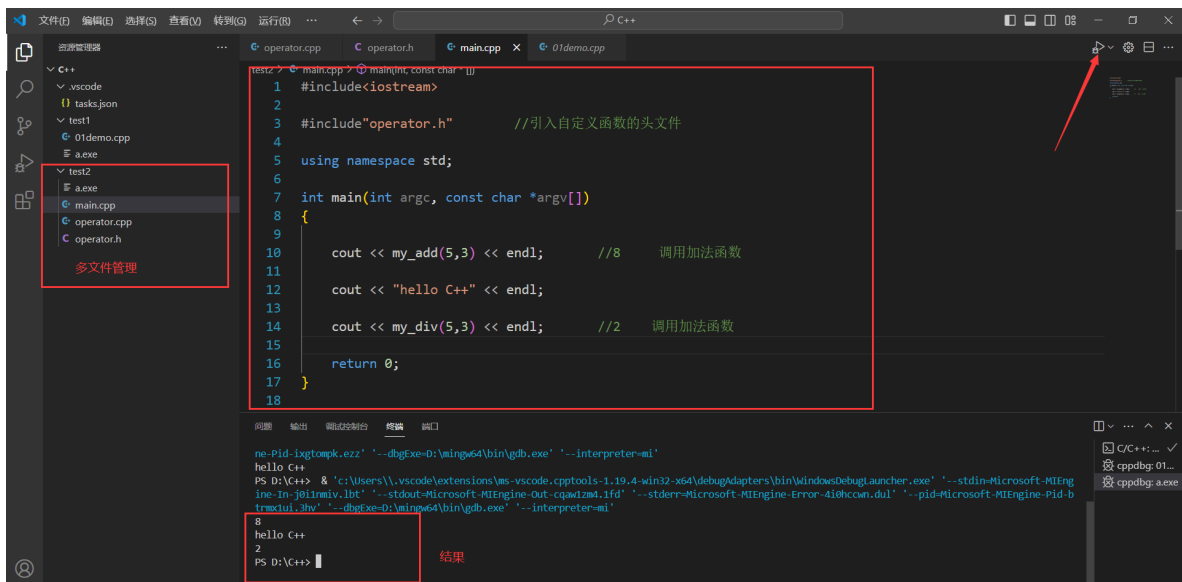
    cout << my_add(5,3) << endl;    //8    调用加法函数

    cout << "hello C++" << endl;

    cout << my_div(5,3) << endl;    //2    调用加法函数

    return 0;
}

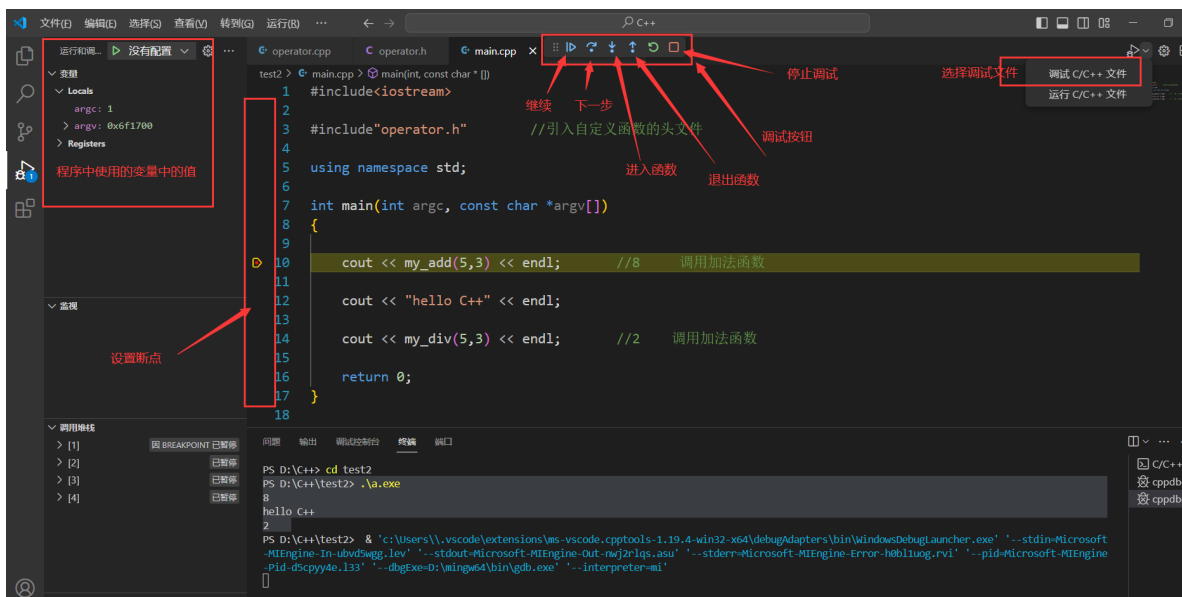
```



5> vsc调试与gdb对比

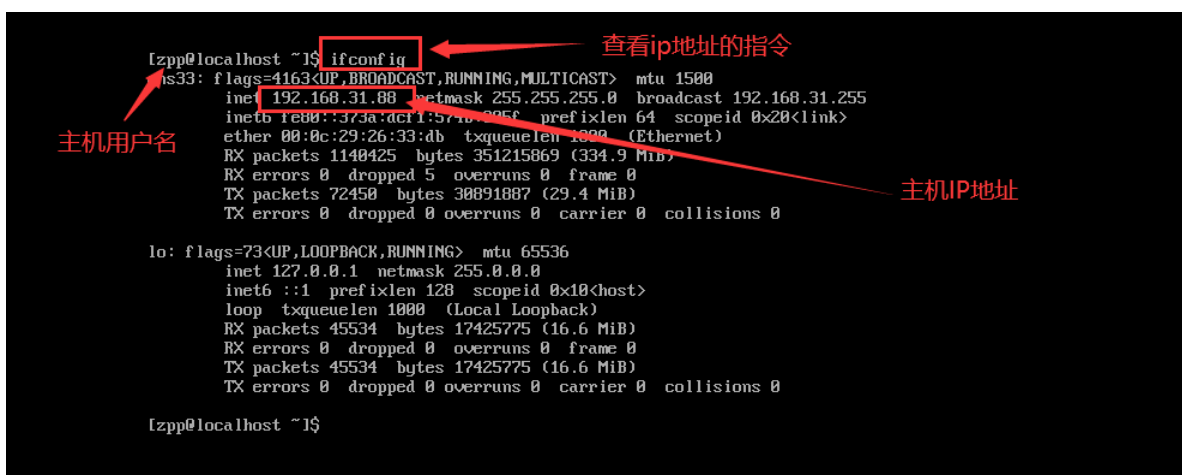
1、linux中可以使用gdb调试相关程序，但是需要使用相关指令来完成，但是在vscode中，可以继续使用gdb完成对项目的调试，也可以通过鼠标进行点击操作来完成对项目的调试

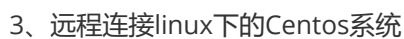
2、vscode中调试工具的使用

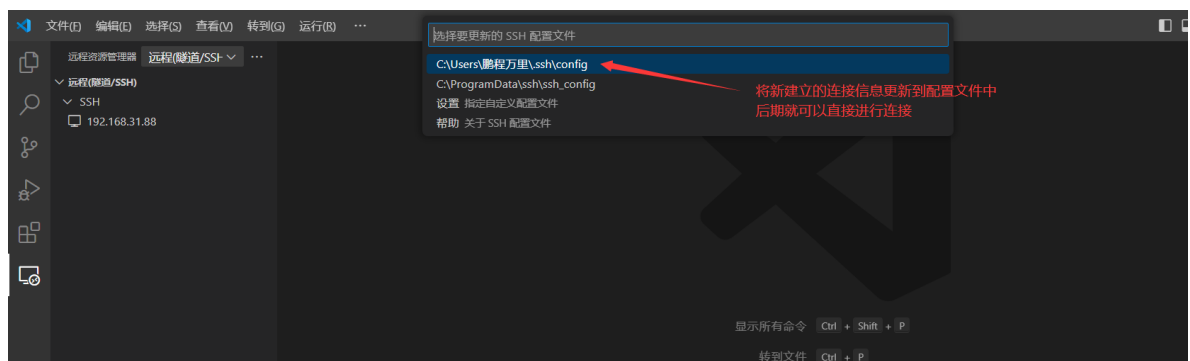
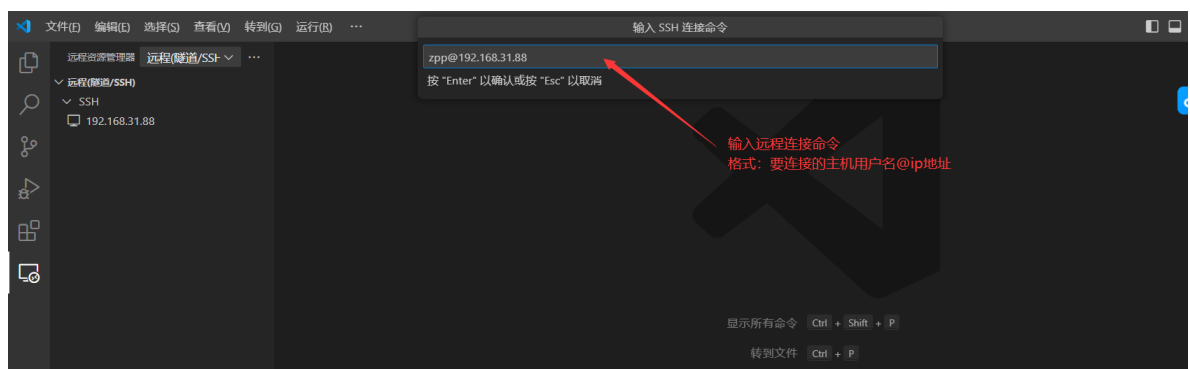
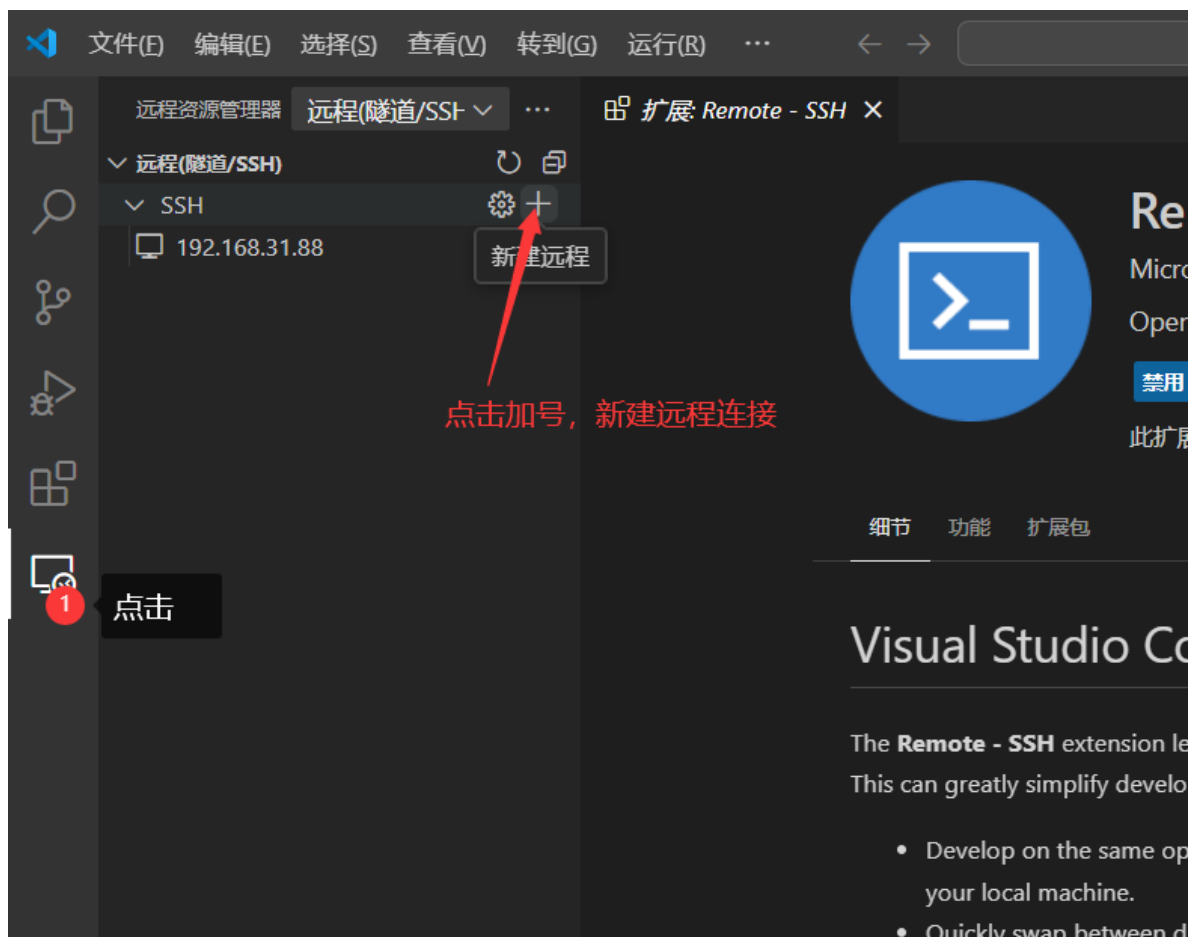


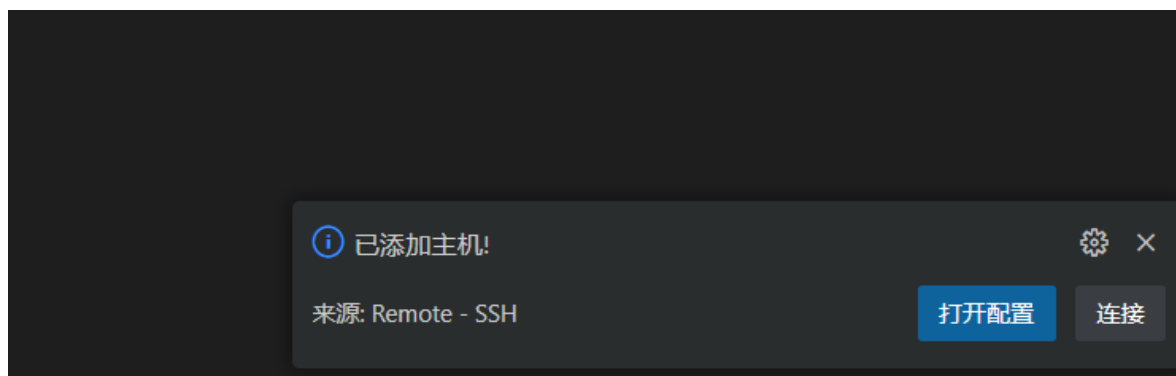
6> vsc远程连接CentOS

1、查看linux系统下的用户名和 ip地址

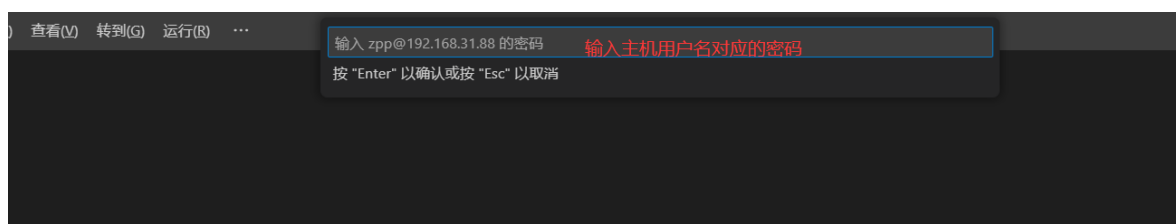
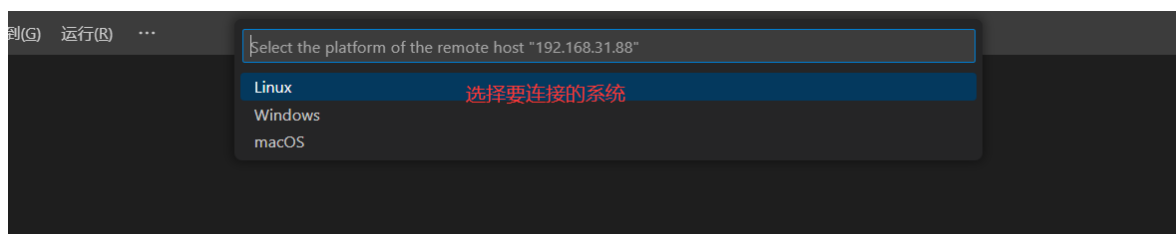
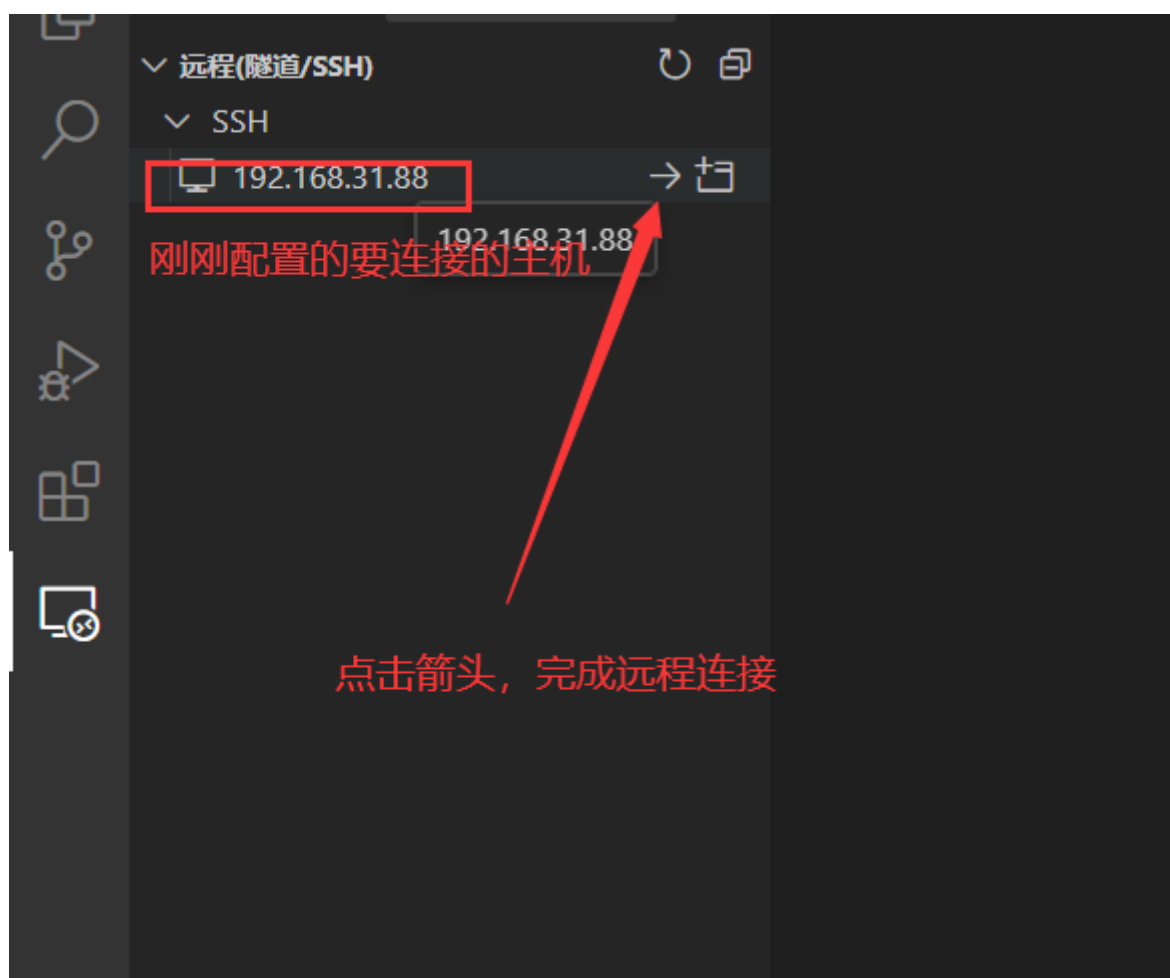




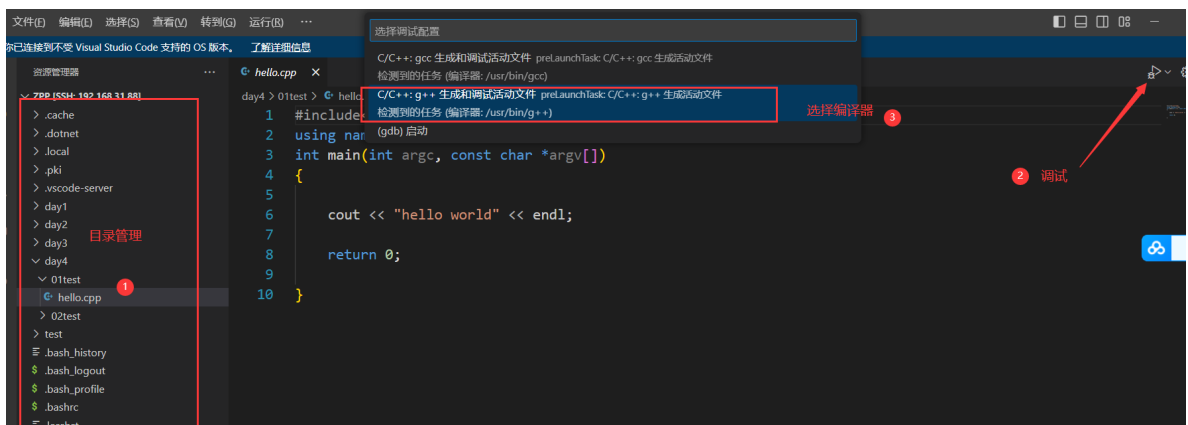
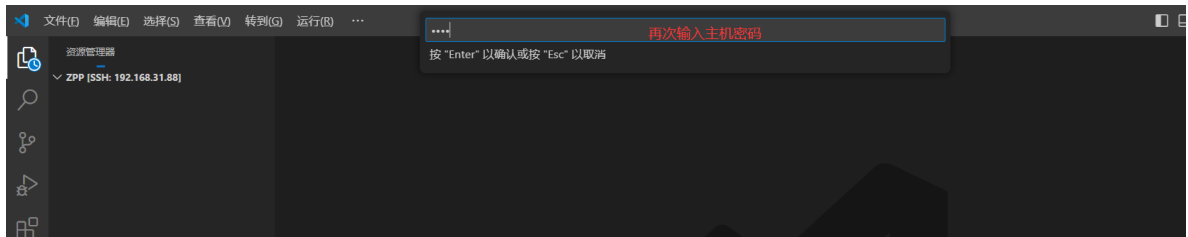
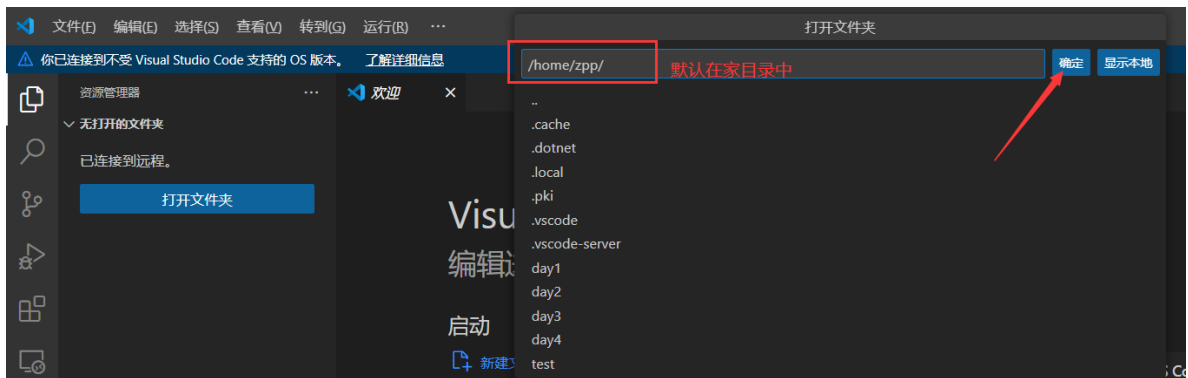
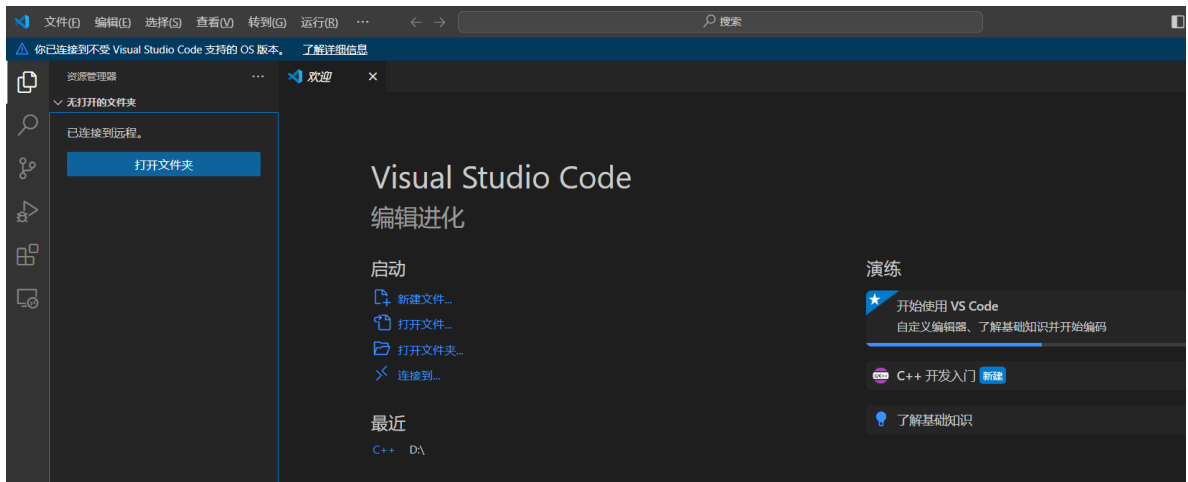




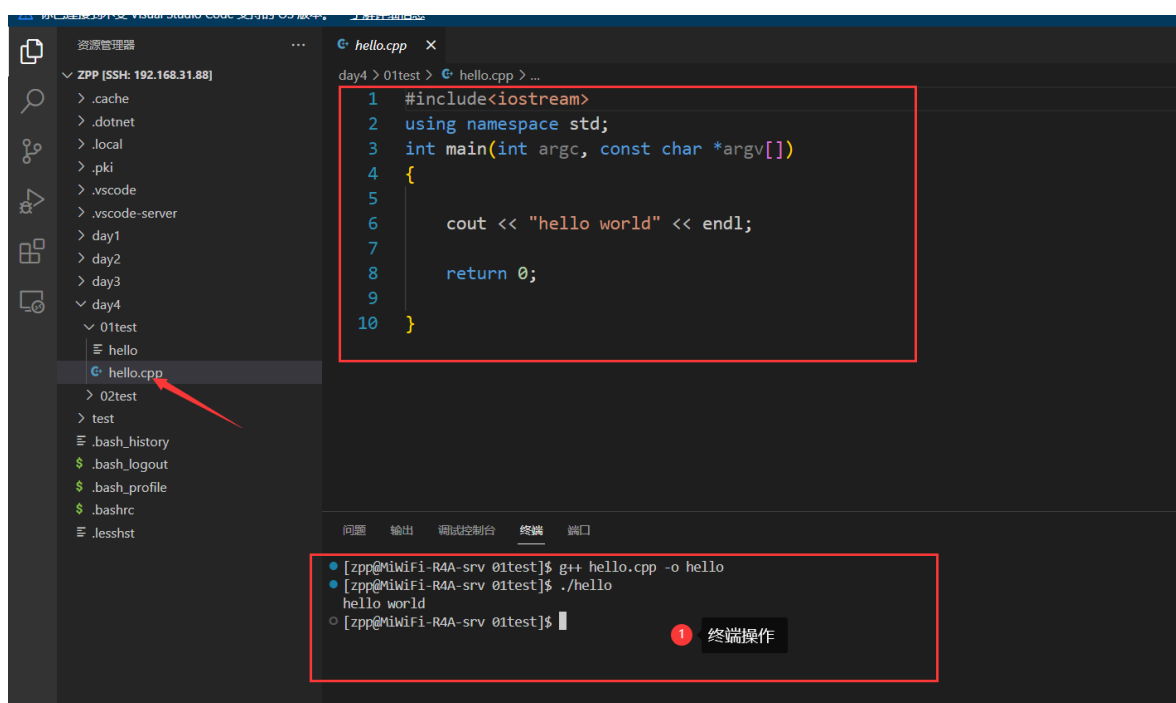
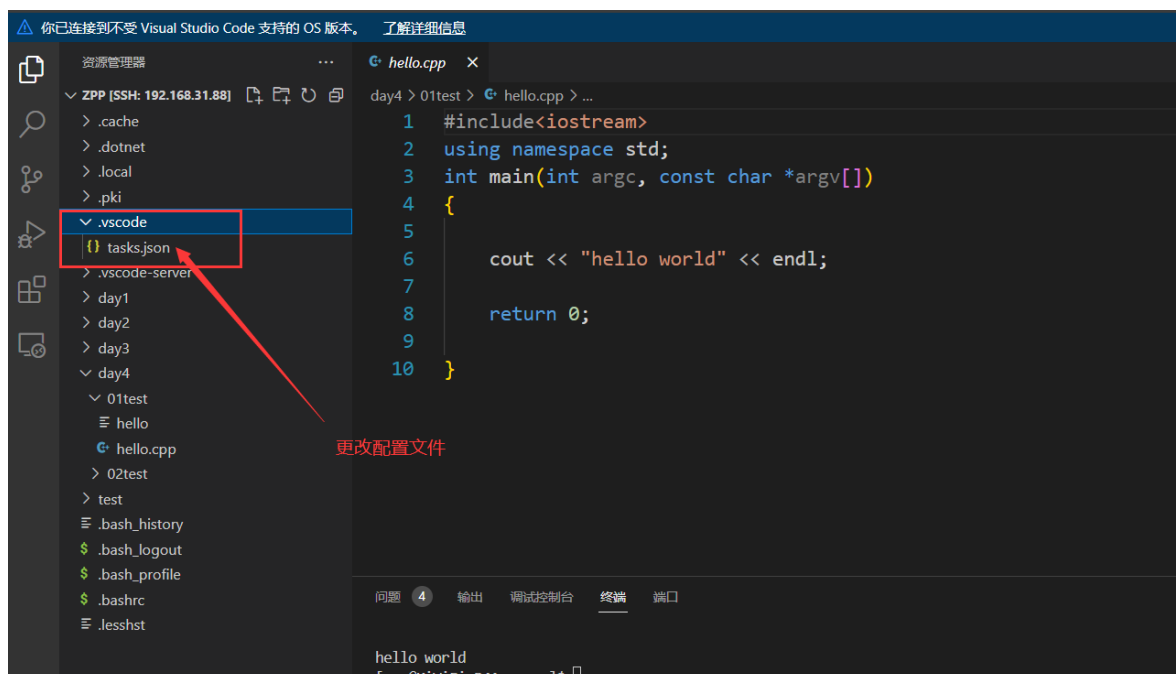
此时说明：已经将添加信息存档了，后面只需正常点击连接即可



连接成功的状态



更改linux下的C++的配置文件



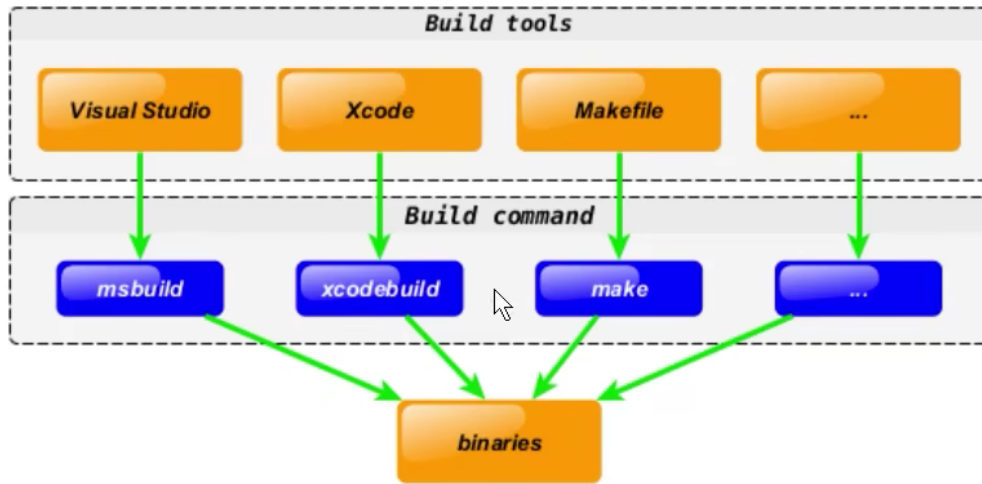
7> vsc中使用cmake

7.1 前言

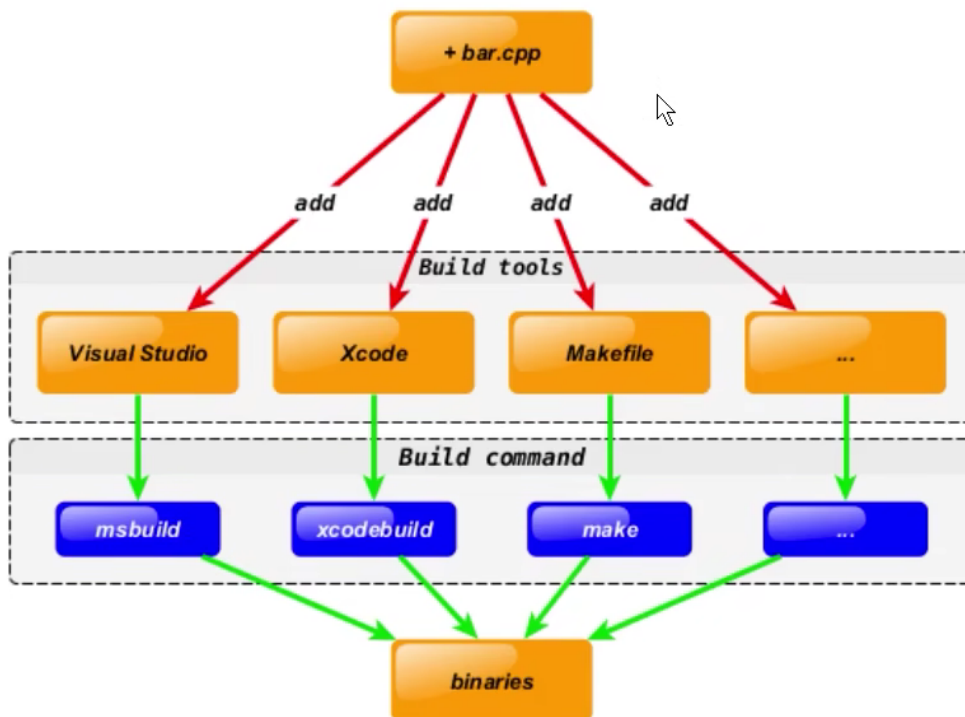
- 1> CMake是一个跨平台的安装编译工具，可以使用简单的语句来描述所有平台的安装（编译过程）
- 2> CMake可以说是已经成为大部分的C++开发项目的标配
- 3> 可以使用几行或者几十行的代码来完成非常冗长的Makefile代码

7.2 为什么要使用CMake

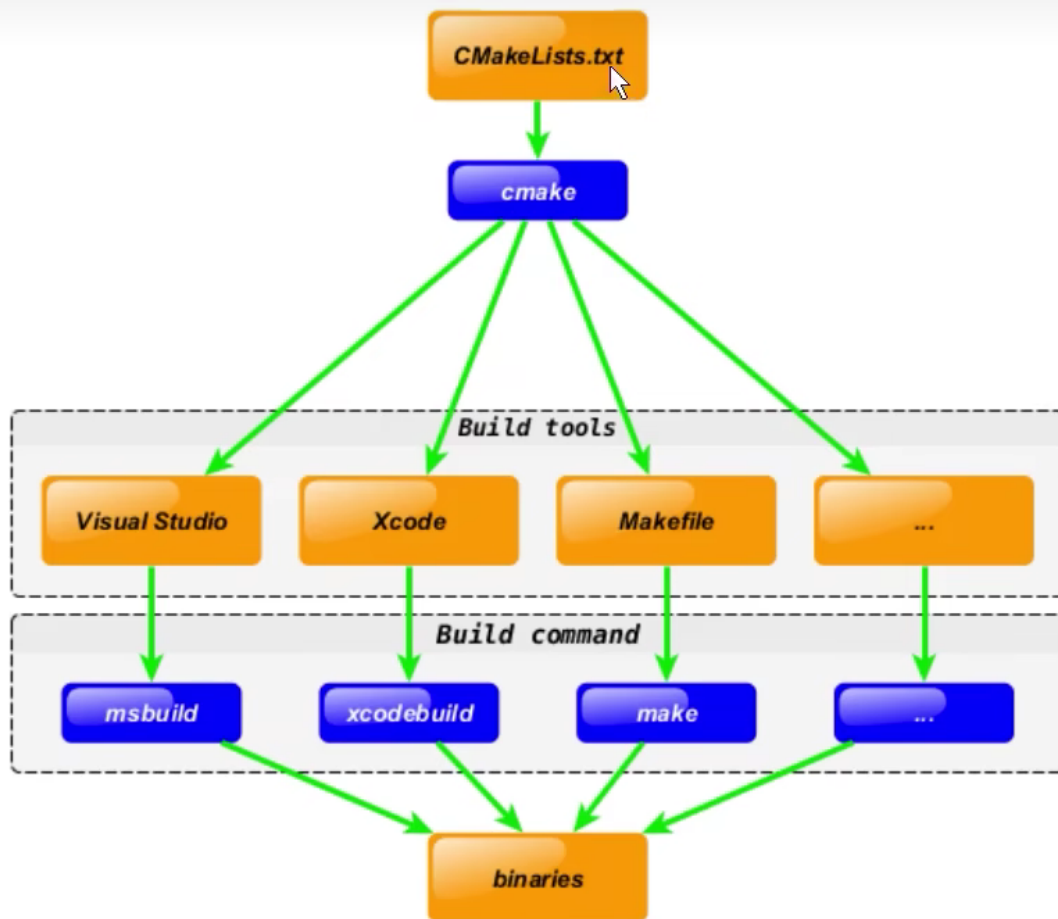
- 1> 在不使用CMake时，编译工程如下



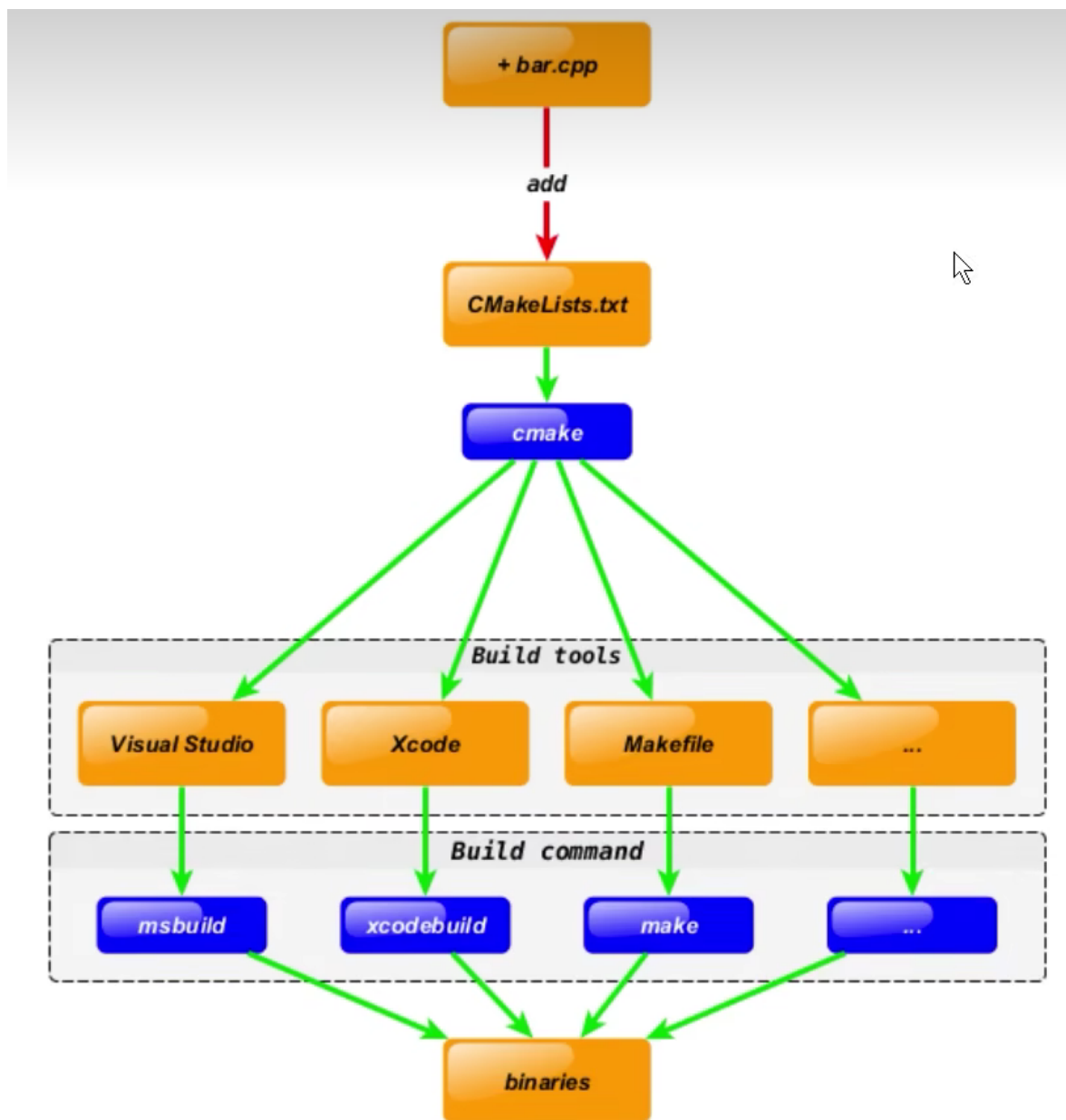
2> 在上面的机制中，工程文件中添加一个源程序 bar.cpp



3> 使用CMake来管理工程的状态



4> 使用CMake管理工程中添加一个新文件 bar.cpp



7.3 语法特性介绍

1> 基本语法：指令（参数1 参数2 ...）

参数使用括号括起来

参数之间使用空格或分号隔开

2> 注意：指令是大小写无关的，但是参数和变量是大小写相关的

```
set(HELLO hello.cpp)           # 定义一个变量名叫HELLO 变量的值为hello.cpp
add_executable(hello main.cpp hello.cpp)  # 通过main.cpp 和hello.cpp 编译生成
hello可执行程序
ADD_EXECUTABLE(hello main.cpp ${HELLO})  # 作用同上
```

3> 变量使用\${}进行取值，但是在if控制语句中，是直接使用变量名的

if(HELLO) 是正确的

if(\${HELLO}) 是不正确的

4> 语句不以分号结束

7.4 重要的指令

1> `cmake_minimum_required`:指定CMake的最小版本支持,一般作为第一条cmake指令

```
# CMake设置最小支持版本为 2.8
cmake_minimum_required(VERSION 2.8)
```

2> `project`: 定义工程的名称, 并可以指定工程支持的语言

```
# 指定工程的名称为HELLOWORLD
project(HELLOWORLD CXX)      # 表示工程名为HELLOWORLD 使用的语言为C++
```

3> `set`:显式定义变量

```
# 定义变量 SRC 其值为 sayhello.cpp hello.cpp
set(SRC sayhello.cpp hello.cpp)
```

4> `add_executable`:通过依赖生成可执行程序

```
# 编译main.cpp 生成main的可执行程序
add_executable(main main.cpp)
```

5> `include_directories`:向工程添加多个特定的头文件搜索路径吗, 类似于g++编译指令中的 `-I`

```
# 将/usr/lib/mylibfolder 和 ./include添加到工程路径中
include_directories(/usr/lib/mylibfolder ./include)
```

6> `link_directories`:向工程中国添加多个特定的库文件搜索路径, 类似于g++编译指令的 `-L`选项

```
# 将/usr/lib/mylibfolder 和 ./lib添加到库文件搜索路径中
link_directories(/usr/lib/mylibfolder ./lib)
```

7> `add_library`:生成库文件 (包括动态库和静态库)

```
# 通过SRC 变量中的文件, 生成动态库
add_library(hello SHARED ${SRC})      # 该语句生成的是动态库
add_library(hello STATIC ${SRC})      # 该语句生成的是静态库
```

8> `add_compile_options`:添加编译参数

```
# 添加编译参数: -Wall -std=c++11
add_compile_options(-Wall -std=c++11)
```

9> target_link_libraries:为target添加需要链接的共享库,类似于g++编译中的 -l 指令

```
# 将hello 动态库文件链接到可执行程序main中
target_link_libraries(main hello)
```

7.5 CMake常用变量

1> CMAKE_C_FLAGS:gcc编译选项的值

2> CMAKE_CXX_FLAGS:g++编译选项的值

```
# 在CMAKE_CXX_FLAGS编译选项后追加 -std=c++11
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11")
```

3> CMAKE_BUILD_TYPE:编译类型 (Debug、Release)

```
# 设定编译类型为Debug, 调试时需要选择该模式
set(CMAKE_BUILD_TYPE Debug)

# 设定编译类型为Release, 发布需要选择该模式
set(CMAKE_BUILD_TYPE Release)
```

7.6 CMake编译工程

CMake目录结构: 项目主目录中会放一个CMakeLists.txt的文本文档, 后期使用cmake指令时, 依赖的就是该文档

1> 包含源文件的子文件夹中包含CMakeLists.txt文件时, 主目录的CMakeLists.txt要通过add_subdirectory添加子目录

2> 包含源文件的子文件夹中不包含CMakeLists.txt文件时, 子目录编译规则, 体现在主目录中的CMakeLists.txt

7.6.1 两种构建方式

1> 内部构建: 不推荐使用

内部构建会在主目录下, 产生一大堆中间文件, 这些中间文件并不是我们最终所需要的, 和工程源文件放在一起时, 会显得比较杂乱无章

```
## 内部构建

# 在当前目录下, 编译主目录中的CMakeLists.txt 文件生成Makefile文件
cmake . # . 表示当前路径
# 执行make命令, 生成目标文件
make
```

2> 外部构建：推荐使用

将编译输出的文件与源文件放到不同的目录下，进行编译，此时，编译生成的中间文件，不会跟工程源文件进行混淆

```
## 外部构建步骤

# 1、在当前目录下，创建一个 build 文件，用于存储生成中间文件
mkdir build

# 2、进入build文件夹内
cd build

# 3、编译上一级目录中的CMakeLists.txt,生成Makefile文件以及其他文件
cmake ..          # ..表示上一级目录

# 4、执行make命令，生成可执行程序
make
```

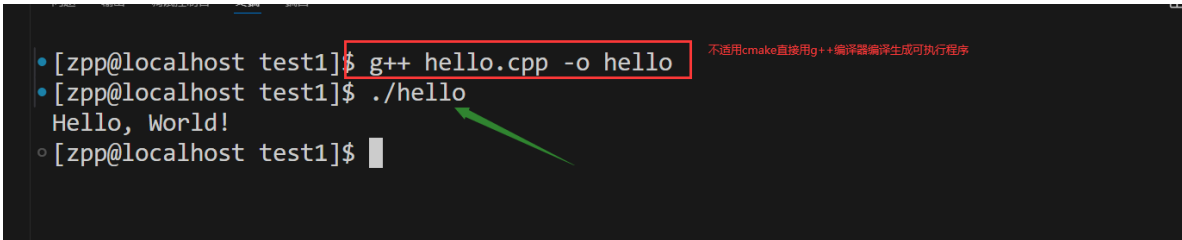
7.7 CMake代码实战

7.7.1 同一目录下的文件进行编译

1> 源文件

```
#include <myhead.h>

int main(int argc, const char *argv[])
{
    std::cout << "Hello, world!" << std::endl;    //输出一个hello world语句
    return 0;
}
```



```
[zpp@localhost test1]$ g++ hello.cpp -o hello
[zpp@localhost test1]$ ./hello
Hello, World!
[zpp@localhost test1]$
```

2> CMakeLists.txt文件

```
# 设置最小版本支持
cmake_minimum_required(VERSION 2.8)

# 项目名称
project(HELLO)

# 生成可执行程序,依赖于hello.cpp生成hello可执行程序
add_executable(hello hello.cpp)
```


3> 内部构建

```
hello.cpp 1 #include <myhead.h>
2
3 int main(int argc, const char *argv[])
4 {
5     std::cout << "Hello, World!" << std::endl;
6     return 0;
7 }
```

```
CMakeLists.txt 1
2 # 设置最小版本支持
3 cmake_minimum_required(VERSION 2.8)
4
5 # 项目名称
6 project(HELLO)
7
8 # 生成可执行程序,依赖于hello.cpp生成hello可执行程序
9 add_executable(hello_cmake hello.cpp)
```

```
[zpp@localhost test1]$ cmake .
-- The C compiler identification is GNU 4.8.5
-- The CXX compiler identification is GNU 4.8.5
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Configuring done
-- Generating done
```

```
hello.cpp 1 #include <myhead.h>
2
3 int main(int argc, const char *argv[])
4 {
5     std::cout << "Hello, World!" << std::endl;
6     return 0;
7 }
```

```
CMakeLists.txt 1
2 # 设置最小版本支持
3 cmake_minimum_required(VERSION 2.8)
4
5 # 项目名称
6 project(HELLO)
7
8 # 生成可执行程序,依赖于hello.cpp生成hello可执行程序
9 add_executable(hello_cmake hello.cpp)
```

```
[zpp@localhost test1]$ make
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/zpp/CMake/test1
[zpp@localhost test1]$ make
Scanning dependencies of target hello_cmake
[100%] Building CXX object CMakeFiles/hello_cmake.dir/hello.cpp.o
Linking CXX executable hello_cmake
[100%] Built target hello_cmake
[zpp@localhost test1]$ ./hello_cmake
Hello, World!
[zpp@localhost test1]$
```

4> 外部编译

```
hello.cpp 1 #include <myhead.h>
2
3 int main(int argc, const char *argv[])
4 {
5     std::cout << "Hello, World!" << std::endl;
6     return 0;
7 }
```

```
CMakeLists.txt 1
2 # 设置最小版本支持
3 cmake_minimum_required(VERSION 2.8)
4
5 # 项目名称
6 project(HELLO)
7
8 # 生成可执行程序,依赖于hello.cpp生成hello可执行程序
9 add_executable(hello_cmake hello.cpp)
```

```
[zpp@localhost test1]$ mkdir build
[zpp@localhost test1]$ cd build
[zpp@localhost build]$ pwd
/home/zpp/CMake/test1/build
[zpp@localhost build]$ cmake ..
-- The C compiler identification is GNU 4.8.5
-- The CXX compiler identification is GNU 4.8.5
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
```

```
1 #include <myhead.h>
2
3 int main(int argc, const char *argv[])
4 {
5     std::cout << "Hello, World!" << std::endl;
6     return 0;
7 }
```

```
1
2 # 设置最小版本支持
3 cmake_minimum_required(VERSION 2.8)
4
5 # 项目名称
6 project(HELLO)
7
8 # 生成可执行程序, 依赖于hello.cpp生成hello可执行程序
9 add_executable(hello_cmake hello.cpp)
```

```
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/zpp/CMake/test1/build
[zpp@localhost build]$ make
Scanning dependencies of target hello_cmake
[100%] Building CXX object CMakeFiles/hello_cmake.dir/hello.cpp.o
Linking CXX executable hello_cmake
[100%] Built target hello_cmake
[zpp@localhost build]$ ./hello_cmake
Hello, World!
```

7.7.2 分文件编译

1> 头文件

```
#ifndef SWAP_H
#define SWAP_H

#include<myhead.h>

//声明一个自定义交换类
class My_swap
{
private:
    int a;
    int b;           //两个成员变量
public:
    //定义构造函数
    My_swap(int a, int b)
    {
        this->a = a;
        this->b = b;
    }

    //声明交换函数
    void run();

    //声明打印函数
    void printInfo();
};

#endif
```

2> 源文件

```
#include "swap.h"

//对交换函数的定义
void My_swap::run()
{
    int temp = a;
    a = b;
    b = temp;
}

//对打印函数的定义
void My_swap::printInfo()
{
    cout << "a = " << a<< endl;
    cout << "b = " << b<< endl;
}
```

3> 测试文件

```
#include "swap.h"

int main(int argc, const char *argv[])
{
    //声明一个交换类对象,调用有参构造完成
    My_swap my_swap(520, 1314);

    //输出交换前的结果
    my_swap.printInfo();          //520 1314

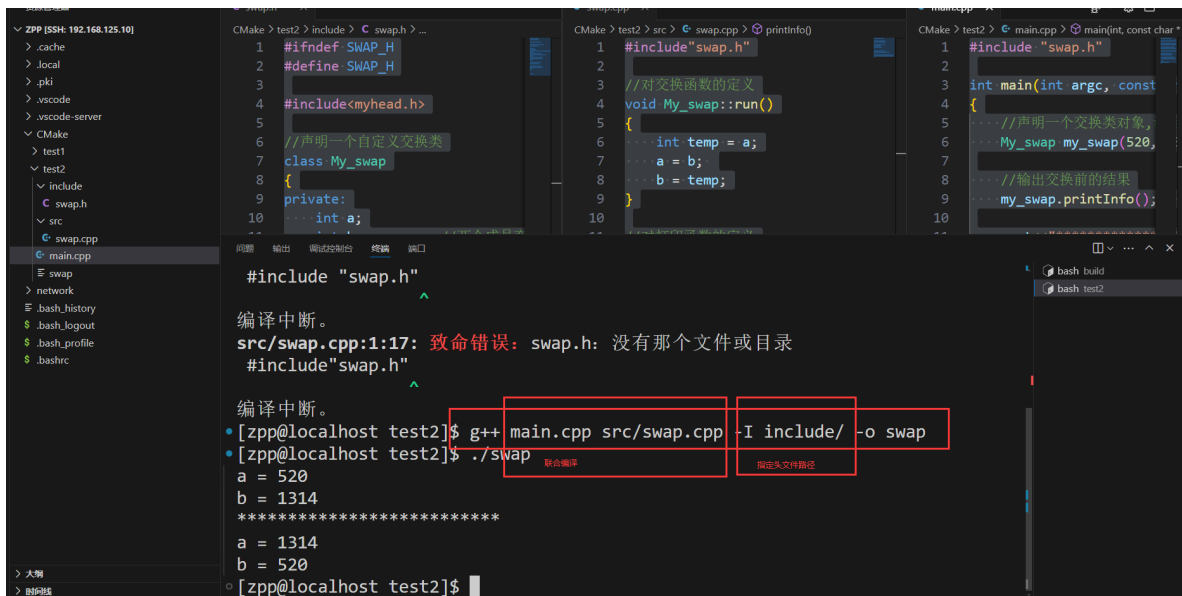
    cout<<"*****"<<endl;

    //调用交换函数
    my_swap.run();

    //输出交换后的结果
    my_swap.printInfo();

    return 0;
}
```

4> 分文件编译使用g++编译器生成可执行程序



```
1 #ifndef SWAP_H
2 #define SWAP_H
3
4 #include<myhead.h>
5
6 //声明一个自定义交换类
7 class My_swap
8 {
9 private:
10     int a;
11
12 public:
13     My_swap(int a):a(a){}
14     void printInfo() const
15     {
16         cout<<"a="<<a<<"b="<<b<<endl;
17     }
18     void swap() const
19     {
20         int temp = a;
21         a = b;
22         b = temp;
23     }
24 };
25
26 #endif
27
28 #include "swap.h"
29
30 int main(int argc, const char * argv[])
31 {
32     My_swap my_swap(520);
33     my_swap.printInfo();
34     my_swap.swap();
35     my_swap.printInfo();
36     return 0;
37 }
```

```
编译中断。
src/swap.cpp:1:17: 致命错误: swap.h: 没有那个文件或目录
#include "swap.h"
^
编译中断。
[zpp@localhost test2]$ g++ main.cpp src/swap.cpp -I include/ -o swap
[zpp@localhost test2]$ ./swap
a = 520
b = 1314
*****
a = 1314
b = 520
[zpp@localhost test2]$
```

5> 创建工程管理文件 CMakeLists.txt

指定最小编译版本

```
cmake_minimum_required(VERSION 2.8)
```

执行工程名称

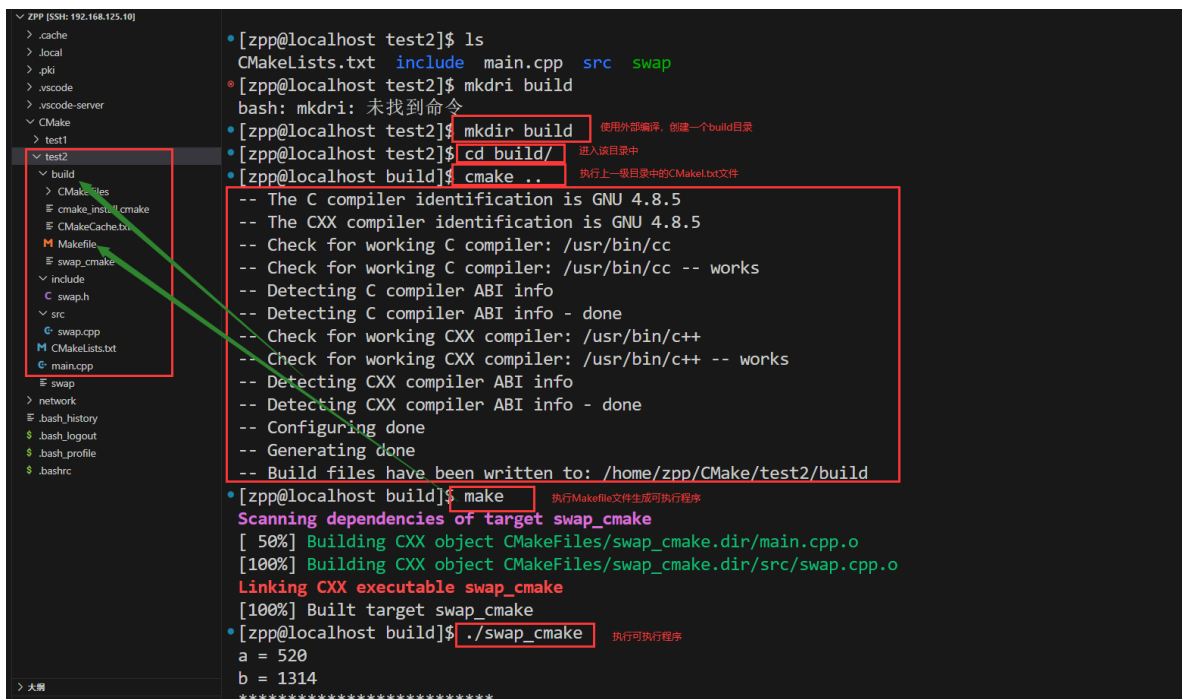
```
project(SWAP)
```

指定头文件路径 就是 g++ 编译器的 -I 选项

```
include_directories(include)
```

生成可执行程序

```
add_executable(swap_cmake main.cpp src/swap.cpp)
```



```
[zpp@localhost test2]$ ls
CMakeLists.txt include main.cpp src swap
[zpp@localhost test2]$ mkdir build
bash: mkdir: 未找到命令
[zpp@localhost test2]$ mkdir build
使用外部编译, 创建一个build目录
[zpp@localhost test2]$ cd build/
进入该目录中
[zpp@localhost build]$ cmake ..
执行上一级目录中的CMakeList.txt文件
-- The C compiler identification is GNU 4.8.5
-- The CXX compiler identification is GNU 4.8.5
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/zpp/CMake/test2/build
[zpp@localhost build]$ make
执行Makefile文件生成可执行程序
Scanning dependencies of target swap_cmake
[ 50%] Building CXX object CMakeFiles/swap_cmake.dir/main.cpp.o
[100%] Building CXX object CMakeFiles/swap_cmake.dir/src/swap.cpp.o
Linking CXX executable swap_cmake
[100%] Built target swap_cmake
[zpp@localhost build]$ ./swap_cmake
a = 520
b = 1314
*****
```

